Kubernetes as a Virtual Machine Management System

A Comparative Study of KubeVirt and Traditional Approaches

EMIL KARLSSON

Master in Computer Science
Date: May 7, 2024
Supervisors: Han Fu
Examiner: Cyrille Artho
School of Electrical Engineering and Computer Science
Host company: Net Insight AB
Swedish title: Swedish title TBD

Abstract

Virtual Machines (VMs) and containers both offer a computing environment for a wide range of workloads, providing their respective benefits such as strong isolation or being lightweight. However, they have traditionally been managed in separate management environments which has led to a fragmented infrastructure for organizations that use both VMs and containers. KubeVirt was created to unify the management of VMs and containers by integrating VMs into Kubernetes, a widely adopted platform originally designed for container orchestration. This project aims to evaluate Kubernetes with KubeVirt as a VM management system both by comparing it with traditional approaches and by investigating the implications of moving into the Kubernetes ecosystem. The project uses a mixed-methods approach including a literature study, a qualitative comparison in the form of a usability analysis and a quantitative comparison in the form of a performance analysis. The results show that managing VMs in Kubernetes using KubeVirt comes with a steeper learning curve than traditional approaches and that there is additional overhead. However, the results also show that KubeVirt can provide performance and operational efficiencies that are not possible with traditional approaches.

Sammanfattning

Contents

1 Introduction **3**
1.1 Research Questions 6
1.2 Problem 7
1.3 Purpose 7
1.4 Goals 7
1.5 Research Method 8
1.6 Limitations 8
1.7 Structure of the thesis 9

2 Background **10**
2.1 Virtualization Fundamentals 10
2.2 Understanding Hypervisors 13
2.3 VM Management Essentials 17
2.3.1 Comparison of VM Management Systems 20
2.4 Containers and Kubernetes: A Paradigm Shift 24
2.4.1 Containers Explained 25
2.4.2 Orchestration Using Kubernetes 27
2.5 Containers vs. Virtual Machines 30
2.5.1 Comparative Overview 30
2.5.2 Decision Factors 33
2.5.3 Towards Stronger Container Isolation 34
2.6 Extending Kubernetes with KubeVirt 36
2.6.1 KubeVirt Technical Overview 37
2.6.2 KubeVirt and GitOps 38
2.6.3 Comparison with Traditional Approaches 38
2.7 Related Work 38
2.7.1 Current challenges 38
2.7.2 KubeVirt challenges 39
2.7.3 Usability of Kubernetes and KubeVirt 39

2.7.4 Performance Evaluations 41

3 Performance Analysis 43
3.1 Setup 43
3.2 Building a Framework 46
3.2.1 Design 46
3.2.2 Implementation 50

4 Usability Analysis 54
4.1 Setup 54
4.2 Task Design 55
4.3 Conducting the Usability Analysis 56

5 Results 57
5.1 Performance Analysis 58
5.2 Usability Analysis 69

6 Discussion 71
6.1 Limited Features and Result Intepretation 71
6.2 Kubernetes Overhead and Induced Load 73
6.3 Jagged Disk Utilization 75
6.4 Mixed Opinions Using KubeVirt 76
6.5 Concluding Research Questions 77

7 Conclusions and Future Work 80

Bibliography 82

A Performance Analysis Components 94
A.1 VM Management System Interface 94
A.2 Metrics Collector 94
A.3 Metrics Collector Daemon 96

B Performance Analysis Results 97

C Usability Tasks 122

D Usability Analysis Answers 135
D.1 Participants 135
D.2 Task 1/3 135
D.3 Task 2/3 138

D.4 Task 3/3 139
D.5 Summary 141

List of Abbreviations

API Application Programming Interface.

AZ Availability Zone.

CaaS Container as a Service.

CLI Command Line Interface.

CNCF Cloud Native Computing Foundation.

CRD Custom Resource Definition.

CRUI Checkpoint-restore in Userspace.

GUI Graphical User Interface.

HPC High-Performance Computing.

IaaS Infrastructure as a Service.

JVM Java Virtual Machine.

KVM Kernel-based Virtual Machine.

OS Operating System.

PaaS Platform as a Service.

QEMU Quick EMUlator.

SaaS Software as a Service.

SAN Storage Area Network.

SSH Secure Shell.

vCPU Virtual CPU.

VM Virtual Machine.

VMM Virtual Machine Monitor.

VNC Virtual Network Computing.

Chapter 1

Introduction

Virtualization is a technique that either splits or combines computing resources into one or more operating environments. This is achieved by creating a software layer that surrounds or underlies existing hardware or software, which enables resources to be decoupled from the underlying hardware. Virtualization comes in various forms including the virtualization of storage, networks and servers. Virtualizing networks and storage is achieved by creating an abstraction layer on top of the physical network and storage resources, respectively, to provide a more flexible interface to interact with. Server virtualization, on the other hand, is used to create virtual computing environments that can be used for various workloads. By combining virtualized resources into a single operating environment, virtualization allows heterogeneous hardware resources to be used homogeneously [1, 2, 3].

Server virtualization is achieved by using a hypervisor, a software layer that operates between the hardware and the Operating System (OS) and is responsible for creating virtual computing environments called virtual machines (VMs) [3, 4, 5]. VMs are emulations of physical computers that are perceived as real computers by applications running inside them and are therefore used as complete and independent environments. Such environments are particularly useful when running legacy applications and applications that require a high level of isolation. For this reason, VMs have long been the core part of IT infrastructure [1, 2].

In parallel with the evolution of VMs, containers have emerged as an alternative to VMs. Container virtualization is not based on a hypervisor and instead operators higher level allowing multiple computing environments on a single OS instance. As such, containers are more lightweight, have less overhead and have faster startup times than VMs [6]. For this reason, containers

have become a popular choice for scenarios in which scaling and portability are important, such as in a microservice architecture, which are core components in modern cloud-native applications [7]. Figure 1.1 visualizes the differences between how VMs and containers are virtualized, and how the virtualization layer is placed on a lower level for VMs than for containers.

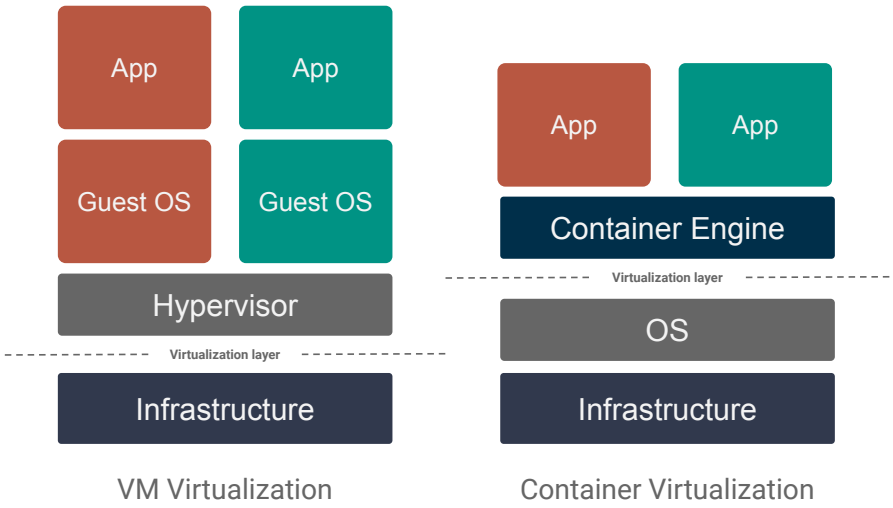

Figure 1.1: Overview of VMs and containers. The virtualization layer is placed on a lower level for VMs than for containers, illustrating the difference in their respective virtualization techniques.

However, while containers provide a lightweight alternative to VMs, they lack the strong isolation properties VMs have. Sharing the OS kernel with the host and other containers opens up a larger attack surface, which implies that containers are more prone to security vulnerabilities. While there exist technologies that strive for the benefits of both VMs and containers, such as Kata Containers, gVisor and Firecracker, it remains a fact that both containers and VMs have their separate use cases [8, 9].

Due to their different use cases, VMs and containers have traditionally been managed separately. VMs are often managed by a VM management system that provides tools and processes for life-cycle management. This makes a VM management system act as a higher-level abstraction layer that unifies hypervisors and provides interfaces to interact with the underlying VMs, typically through a Command Line Interface (CLI) or a web-based Graphical User Interface (GUI). Figure 1.2 illustrates this in an overview of a VM management system's role where clients interact with the system to manage VMs distributed across several hosts. Moreover, a VM management system is often part of a

larger infrastructure, such as an Infrastructure as a Service (IaaS), that extends its role to also include resources such as networking and storage. Examples of such systems include OpenStack [10], CloudStack [11], and OpenNebula [12]. Containers, on the other hand, are often managed by a container orchestration platform. These platforms provide similar tools and processes as a VM management system but for containers, with some additional tools for container-centric operation such as scaling. Examples of such platforms include Kubernetes [13], Nomad [14] and Docker Swarm [15], with the former being the most widely adopted [16]. As such, while VMs and containers have their respective use, they also share aspects regarding how they provide virtual computing environments. For this reason, there has been an ongoing convergence in how VMs and containers are managed [17].

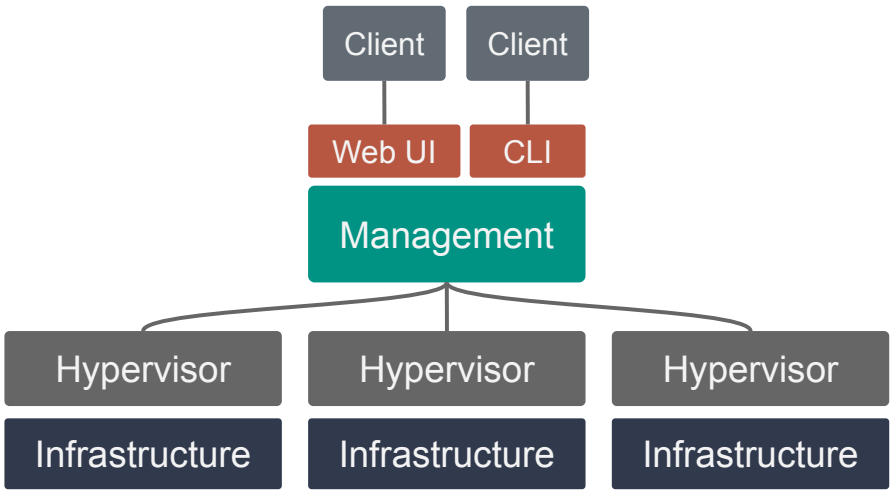

Figure 1.2: Brief overview of a monolithic VM management system where clients communicate with a management node through a set of interfaces to manage VMs distributed across several hosts. The indirect communication allows a client to manage VMs without having to interact with the hosts directly.

Kubernetes, originally designed to manage containerized applications at scale, was recently extended to accommodate VMs through the KubeVirt project. KubeVirt is a Kubernetes extension that enables VMs to run inside a Kubernetes environment alongside containers, creating a common environment for both virtualization technologies [18]. Figure 1.3 depicts a high-level overview of how VMs and containers coincide in a Kubernetes environment using KubeVirt, where traffic flows natively between the two types of resources. How this is achieved on a technical level is discussed later in 2.6.1.

The approach of including VMs in the Kubernetes ecosystem not only

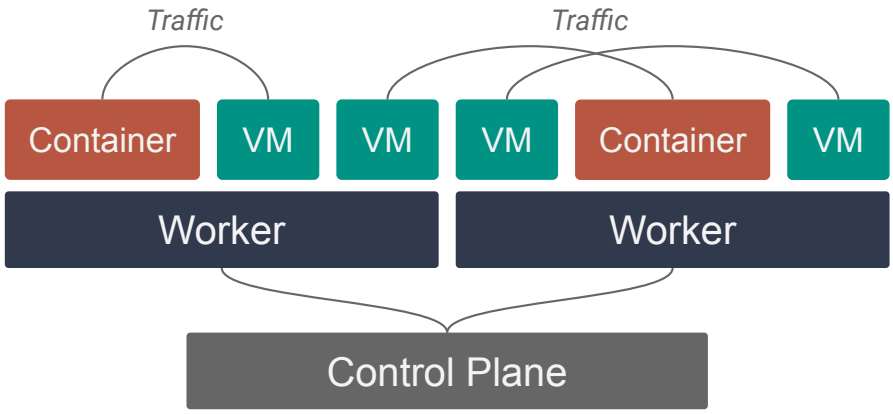

Figure 1.3: Overview of VMs and containers managed by using KubeVirt. The figure illustrates how VMs and containers communicate natively in a single environment.

benefits from the mature Kubernetes infrastructure but also from tools and processes that are designed to integrate with the life cycle of Kubernetes resources. This means by using KubeVirt, VMs can be managed more similarly to containers, which creates a more cloud-native approach to managing VM [7]. Additionally, consolidating the management of VMs and containers into a single platform could offer more benefits since the system architecture could be simplified and the required skill set in an organization could be reduced [19, 20].

The integration of VMs into Kubernetes using KubeVirt represents a significant milestone toward a more agile, efficient and unified IT infrastructure [20]. However, KubeVirt's practicality in a real-world scenario is not yet fully explored. As such, this project aims to evaluate Kubernetes with KubeVirt as a VM management system and compare it with traditional approaches to understand its practicality and limitations.

1.1 Research Questions

What performance, scalability, and operational efficiencies can be gained by integrating VMs in Kubernetes?

This could stem from using a single resource scheduler. Several metrics will be used to evaluate this, as described in the background and related work.

What are the current challenges and limitations in managing VMs and containerized applications in separate environments?

This is concerned with scenarios in which a traditional VM management system is operated alongside Kubernetes.

How can system administration be simplified by running VMs in Kubernetes using KubeVirt?

This is focused on the practical implications of using KubeVirt and Kubernetes instead of traditional VM management systems. For instance, the number of steps required to deploy a VM.

1.2 Problem

VMs and containers are similar technologies but with different use cases. This has led to separate tools and processes for managing them, and entire systems built solely for one of them. However, both VMs and containers are necessary parts of the modern IT infrastructure that many organizations depend on. This has led to a fragmented infrastructure and a larger skillset among operators to manage both types. Furthermore, many technologies today integrate well with Kubernetes's declarative approach to resource management, and thus, by managing a resource outside of Kubernetes, these benefits are not fully utilized.

1.3 Purpose

The purpose of this project is to evaluate whether Kubernetes with KubeVirt is a viable option for VM management. This evaluation is important in the field of cloud computing since it can provide insight into how an existing system could benefit from integrating VMs management in Kubernetes, as well as consolidating the management of VMs and containers to a single platform.

1.4 Goals

The goal of this project is to provide a transparent evaluation with quantifiable results regarding the applicability of managing VMs inside the Kubernetes ecosystem. The results could be used as a fundamental argument for or

against Kubernetes as a management platform, and potentially result in a more efficient management system. Potential results that argue for the use of KubeVirt in terms of performance could be used as an argument for future research regarding sustainability.

1.5 Research Method

This project will use a mixed-methods approach to investigate the research questions. A literature study will initially be conducted to thoroughly explore the reasons that led to KubeVirt being created as well as the current challenges regarding VM management. To evaluate KubeVirt itself both a quantitative and qualitative analysis will be conducted. The quantitative analysis will be based on a performance analysis that compares KubeVirt against a traditional approach and benchmarks the performance of the system when performing tasks such as creating VMs. The qualitative aspects will be based on a usability analysis, which involves having system administrators perform certain common tasks using KubeVirt. Together, these three methods capture an image of using KubeVirt for managing VMs.

1.6 Limitations

1. This project will not investigate any solutions for shortcomings in KubeVirt itself, such as finding alternatives for features that exist in existing virtual machine management systems but not in KubeVirt. As such, this project acknowledges that KubeVirt might not be a complete replacement for traditional virtual machine management tools since it might not offer all the features of an IaaS system. Therefore, only features that exist in KubeVirt and the compared system will be considered when testing.

2. This project will primarily focus on the practical implications of moving into a KubeVirt-based system. For this, there are mainly two scenarios that will be considered:

    - **Replacement**: A system uses only a traditional virtual management system. By using KubeVirt, the system will be replaced by a Kubernetes-based system.

- **Unifying**: A system uses both traditional management and Kubernetes. By using KubeVirt, the system will be unified into a single platform for both VMs and containers.

1.7 Structure of the thesis

This report is divided into three main parts. The first part is concerned with background information and the relevant concepts and technologies that are necessary to understand the problem presented in this project. It aims to provide the fundamentals to understand why VM management and container orchestration have traditionally been separated as well as why KubeVirt was created. This part also includes related work that is used as a base for the methods used in the project.

The second part describes the research method used in the project and is divided into two branches, performance analysis and usability analysis. These branches cover both the quantitative and qualitative aspects of the project.

The third and last part involves result evaluation and discussions regarding the interpretation of the results. It combines the quantitative and qualitative aspects to provide a more comprehensive understanding of the results, as well as answer the research questions and conclude the project as a whole.

Chapter 2

Background

This chapter will discuss the relevant background information as to why KubeVirt has become an alternative to traditional VM management systems. It will cover the fundamentals of virtualization, hypervisors and VMs, before transitioning into container technologies, and then finally introduce Kubernetes and KubeVirt.

2.1 Virtualization Fundamentals

Virtualization is the technology that enables hardware to be split or grouped into logical units. Each unit runs with its own allocated resources and is separated from other units running on the same host. Virtualization can thus occur in various forms including virtualizing networks, storage and servers [2]. The two former are a simpler type of virtualization that involves creating an abstraction layer on top of existing physical resources to provide a logical interface defined in software. By defining storage and network in software, a more flexible environment is created, where an application interacts with interfaces that indirectly point to a group of hardware, instead of directly communicating with the hardware. Server virtualization, on the other hand, is the more common form of virtualization and is also known as hypervisor-based virtualization. It involves splitting a server into multiple virtual computing environments [1]. Hypervisors will be discussed in more detail in the next section, see **??**

A virtual computing environment, more commonly referred to as a VM, is a complete system that operates as an isolated environment. The environment is perceived as if it were an ordinary computer from the perspective of the application running inside it, even though the physical resources are shared with other VMs running on the same host. Being a complete system is particularly

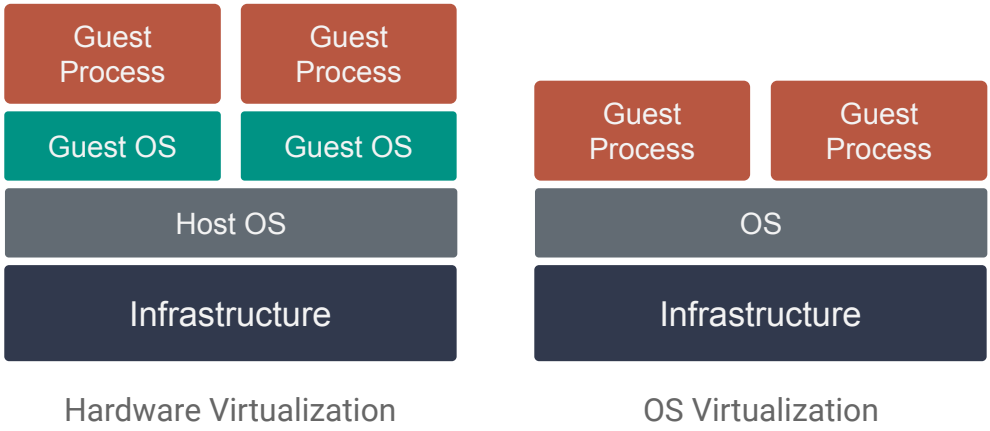

Figure 2.1: Overview of the different layers of virtualization for hardware-level and OS-level virtualization. The figure illustrates how hardware virtualization inhibits several internal OSs from running on the same hardware, while OS virtualization keeps a single OS for each process.

useful when running legacy applications that are dependent on a certain OS or driver version. This could be taken a step further where a piece of hardware could be emulated to suit the needs of a legacy application. Additionally, the isolated environments are suitable for scenarios where there are strict security requirements for the application running inside it [2, 3].

Virtualization can be achieved in different layers depending on the required level of abstraction. The three major levels are hardware level, OS-level and application level. For instance, storage and network virtualization both fall under the hardware level category since they create an abstraction layer on top of the physical resources. However, in other scenarios, virtualization is not required to be as low-level. Application virtualization is concerned with encapsulating an application in a more lightweight environment, such that the application thinks it is executed on a specific platform. A common example is Java Virtual Machine (JVM) that enables programs written in the programming language Java to be run on multiple operating systems [21]. Figure 2.1 illustrates this as an overview of the differences between the two more common layers of virtualization; hardware and OS virtualization. It is important to note that the virtualization layers are not mutually exclusive and that using them in combination leads to a unique use case. For instance, a guest process (OS level) can run in a guest OS (hardware level). This specific use case is referred to as nested virtualization [2].

Virtualization can be divided into two categories, full virtualization and

paravirtualization. Full virtualization is the older and traditional approach where the entire underlying hardware is virtualized to its core. This creates an image of the virtual system that is indistinguishable from a real one from the perspective of the application running inside it. As such, full virtualization allows the VM to be completely decoupled from the underlying hardware and for the guest OS to run without any modifications. [2, 22, 23]. However, while virtualizing on such as low level is practical due to the flexibility it provides, it comes with a cost of performance. For this reason, paravirtualization was created to reduce the number of resources that are virtualized, effectively raising the level of abstraction slightly. With paravirtualization, a VM is presented with an image that is similar, but not identical, to the underlying hardware, and thus the guest OS is aware it is running inside a VM [22, 23].

Benefits and Challenges

Many of the benefits of virtualization stem from the fact that there exists an abstraction layer above the physical hardware. The abstraction layer decouples software from hardware, allowing for the use of vendors with different I/O in the same environment. This creates a more flexible and scalable computing environment as a whole where hardware is turned into a simple pool of computing resources [3]. As such, there are numerous benefits of virtualization, a few of which are listed below.

- **Isolation**
  VMs are complete and independent systems and are thus able to run in isolation from one another. This includes isolating the OS, filesystem and application stack. Isolation is particularly useful for applications with strict security requirements, such as applications managing sensitive or confidential data. Isolation is also beneficial in scenarios that require reproducibility including testing and development environments [24].

- **Flexibility**
  VMs are not tied to specific hardware* and can be moved between different physical resources. Flexibility is an important factor for many practical tasks when managing computing environments, such as moving workloads to a different machine when performing maintenance.

  * As described previously, this is not always true for paravirtualization since it does not virtualize every resource that makes up a VM.

- **Resource utilization**
  Virtualization allows for better utilization of hardware resources since workloads can be consolidated into fewer physical resources that might not have been fully utilized, and reduce the need to acquire additional hardware. The flip side of this is larger points of failure since multiple computing environments depend on the same physical resource. However, this risk is minimized considering the benefits described in *Flexibility* [2].

- **Management**
  VMs can be created and destroyed on demand and can be managed using software tools [2].

- **Virtual hardware**
  A VM can be configured to use hardware that does not exist in the physical environment. This allows for better support for legacy applications or certain applications that are dependent on a specific hardware configuration. For instance, a VM can be configured with a virtual network card that mimics a physical network card that could not be acquired [2].

However, the portability made possible by virtualization comes at a cost of performance. Since a VM is a complete but virtualized system, many or all instructions issued are interpreted in software. This means that despite the level of abstraction layer used, there is an inescapable added overhead when using virtualization. A common hypervisor such as Kernel-based Virtual Machine (KVM) (see 2.2) introduces an overhead of 5 to 10% [25], and the overhead can be even larger when using a full virtualization approach.

2.2 Understanding Hypervisors

A hypervisor is a software layer that enables running multiple VMs on a single physical machine. It achieves this by allowing multiple operating systems (called *guests*) to co-exist and share hardware [1, 5]. Hypervisors, often referred to as Virtual Machine Monitor (VMM) are responsible for creating and managing VMs and for providing the necessary abstractions to the VMs.

Hypervisors can be categorized into two types, *type 1* and *type 2*, based on their operational context and how they interact with the underlying hardware. While both type 1 and type 2 hypervisors create virtual computing environments, they operate on different abstraction levels and have different use cases.

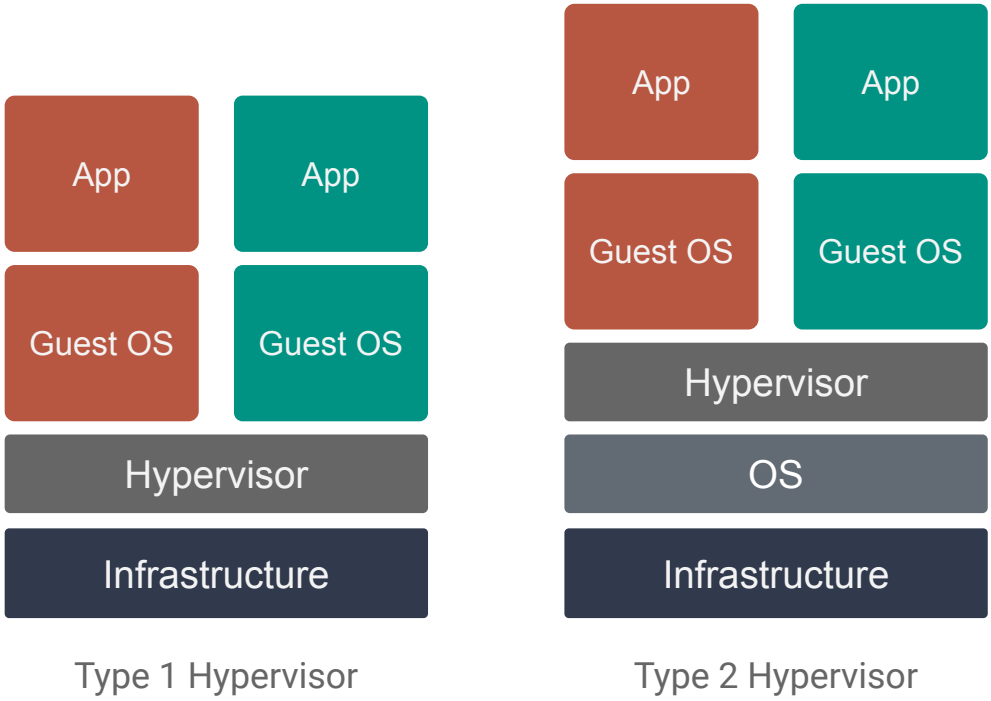

Figure 2.2: Overview of type 1 and type 2 hypervisors. The figure illustrates how a type 1 hypervisor does not use an underlying OS and instead runs directly on the hardware, while a type 2 hypervisor relies on an intermediary OS to manage the underlying hardware.

Type 1 hypervisors, commonly referred to as *bare-metal* hypervisors, run directly on top of the physical hardware and do not require an OS underneath. This enables type 1 hypervisors to run with little to no overhead and scale up to terabytes of RAM and hundreds of CPU cores [26]. However, it also means the hypervisor is entirely responsible for scheduling and ensuring that a VM is allocated the required system resources. For this reason, type 2 hypervisors, also known as *hosted hypervisors*, were created. A type 2 hypervisor is installed on top of an OS and uses OS-level features to manage the resources for a . This makes the hypervisor more portable and simpler to use for an end-user since it can be installed on virtually any system. However, it also introduces an inevitable latency and overhead, which is why type 2 hypervisors are often used for development and testing purposes [5]. As such, type 1 and type 2 are inherently different kinds of hypervisors due to their respective trade-off with portability and performance. Figure 2.2 illustrates a high-level overview of the difference between a type 1 and type 2 hypervisor [5, 26].

Hypervisors can be further categorized into *monolithic* and *microkernelized* hypervisors [27]. Monolithic hypervisors are built as a single large software stack, managing all the VMs and their resources, and hosting drivers for all hardware access. Having all drivers reside within the hypervisor means that the hypervisor will only run on certain hardware. However, it also means no operating system is required at all, as the hypervisor acts as an operating system by supporting all the operations of its VMs. Microkernelized hypervisors, on the other hand, do not require device drivers to be present in the hypervisor. Instead, the drivers are installed in the form of a manager VM, often referred to as parent VM. Each VM created by such a hypervisor is thus referred to as a child VM since it uses the parent VM's drivers to access the hardware. A microkernelized approach is more flexible than a monolithic, as it can allow the hypervisor to run on any hardware that supports the parent VM's operating system. This distinction is similar to type 1 vs type 2 hypervisor, with one being more self-sufficient and the other more modular. However, it is important to note that monolithic and micro kernelized hypervisors are more concerned with the inner technical aspects of a hypervisor rather than the usability of the end-user.

The concept of *security rings* is often used to describe the level of privilege different part of a hypervisor has. A security ring represents a hierarchical protection domain, where the innermost ring (ring 0), usually reserved for the OS kernel, has the highest privilege, and the outermost ring (ring 3), usually reserved for user applications, has the lowest privilege. The hypervisor itself is often placed in ring 0, its VM kernel space in ring 1, and the VM user space in ring 3. Security rings are thus used to define execution level to provide a system to limit the execution capabilities of VMs to regulate access to shared memory. The abstraction leads to more transparency regarding the execution level of certain applications and is key for ensuring that a VM maintains its isolation [27, 28]. Figure 2.3 illustrates this concept as a layered group of rings.

Key Hypervisors

There are several hypervisors available, each with its own set of features and capabilities, some of which are listed below.

- **VMware ESXi**
  A lightweight type 1 monolithic hypervisor often used in enterprise environments. ESXi is known for its ultra-lightweight footprint utilizing

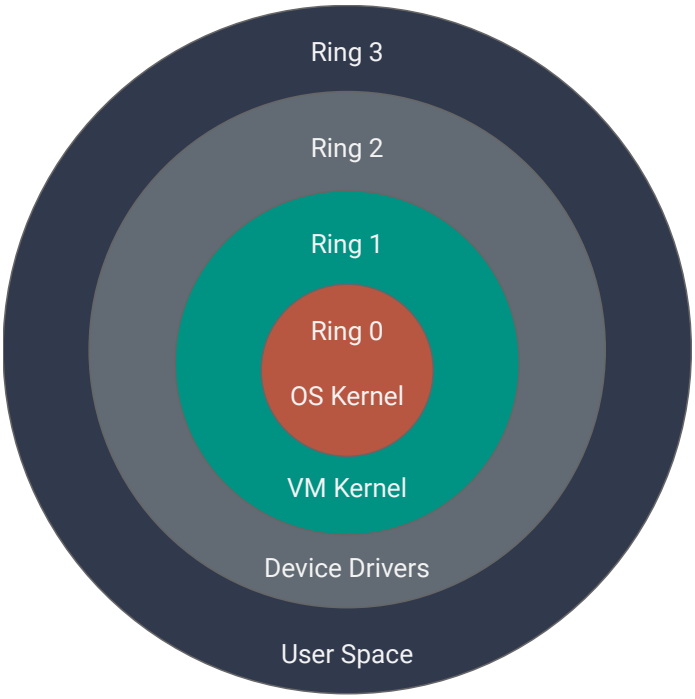

Figure 2.3: Overview of security rings in a hypervisor. The figure illustrates how the inner rings have higher privileges than the outer rings.

just a few hundred megabytes and its ability to run directly on the hardware without an underlying OS [27].

- **Microsoft Hyper-V**
  A type 1 microkernelized hypervisor that is part of the Windows Server operating system. As a microkernelized hypervisor, Hyper-V differentiates between parent and child VMs and uses the parent VM to manage hardware access for the child VMs [27, 29].

- **Citrix Hypervisor**
  Formerly known as XenServer, is a type 1 microkernelized hypervisor based on the Xen Project. It runs using a modified Linux kernel in a VM, called Dom0, that manages hardware I/O and interacts with the other VMs, called DomU guests [23, 27].

- **KVM**
  A type 1 hypervisor that is part of the Linux kernel inheriting many of its functionalities from the Linux kernel such as the scheduler and memory management. KVM is free and open-source and has thus become a popular alternative to proprietary hypervisors, such as VMware ESXi and Microsoft Hyper-V [27, 30].

2.3 VM Management Essentials

A management system for VMs has tools and processes for creating, deploying and managing VMs. The management system is responsible for managing the entire life cycle of a VM, including provisioning, deployment, monitoring and scaling. This is comparable to the role of a hypervisor that also is responsible for creating and managing VMs. However, a VM management system operates on a higher level to provide further abstraction on top of hypervisors, often supporting a range of hypervisors. A VM management system can thus be seen as a control point for the underlying hypervisors [31]. However, certain hypervisor features, such as live migration, could be have unique implementations for different hypervisors. This creates a possible trade-off for a VM management system where it can choose to implement fewer but homogeneous operations, or more tunable operations that implement hypervisor-specific features [32].

Concepts and Features

With a higher abstraction layer, a VM management system brings a set of concepts to simplify the management of VMs. Many such features and concepts exist and can be broadly grouped as clustering, availability and scalability [31].

Clustering involves grouping a set of resources, such as VMs or storage nodes, to provide an image that is perceived as a single much larger resource. By allowing multiple nodes to cooperate and share resources, clustering can allow for a system that can grow horizontally, meaning increasing the number rather than the size of each hardware resource. High-Performance Computing (HPC) is a common example of compute clustering where computing resources are grouped to gain performance greater than that of a single computer. This enables HPC clusters to process large datasets and complex calculations that would be infeasible for a single computer. As such, the computational limit is significantly lower when expanding the size of a single server rather than the number of servers. Moreover, HPC clusters are designed such that they allow for the distribution of computational tasks where larger tasks are divided into smaller subtasks. This might not be possible for all computational tasks, and HPC clusters often require a specific format on input to fully utilize the capabilities of the system [33, 34].

Availability refers to the ability to ensure a service is kept active or running. This is commonly measured by ensuring the end user can access certain applications without disruption. Availability is typically implemented using a set of Availability Zones (AZs), which represent a logical grouping of resources such that each AZ maintains a copy of the service. If the underlying hardware fails in one AZ, another AZ can continue to serve requests. This redundancy allows for outages to be isolated and maintenance to be performed without affecting the uptime of service [31, 35]. Figure 2.4 illustrates the concepts of an AZ with an overview of how 3 AZs provide redundant access to the same service.

Scalability involves technologies to ensure resources can manage an increase or decrease in computational loads. A VM management system often facilitates both scaling up and scaling out. Scaling up, also referred to as vertical scaling, involves increasing or decreasing the performance specification of a resource. For instance, a VM can be scaled up by increasing the number of CPUs or the amount of memory. Scaling out, also referred to as horizontal scaling, involves adding or removing entire resources, such as VMs. This means, that instead of upgrading a single VM, additional VMs are added to the system to handle the increased load. This approach is often regarded as more

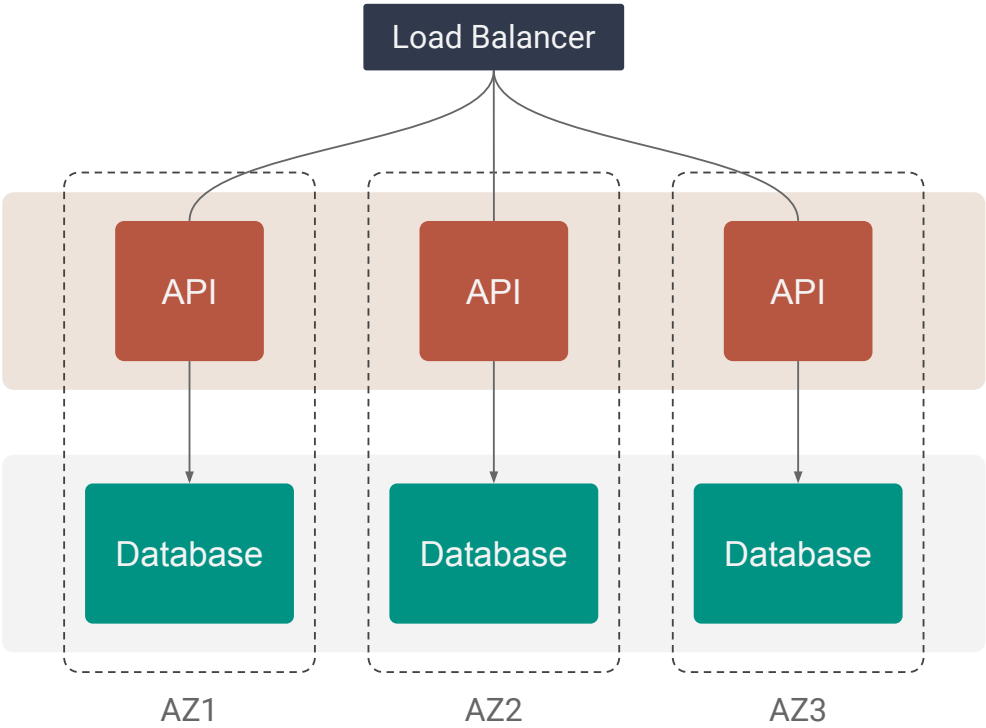

Figure 2.4: Overview of availability zones with several API gateways and databases operating in the same logical environment but in different availability zones to provide redundancy and fault tolerance.

scalable than scaling up since it does not involve any downtime. Scaling up is also in practice usually quite limited since it requires the application running inside the resource to fully utilize its resources, while scaling out expects normal operational load on each instance. The common argument against scaling out though, is that the application might not be designed to operate in several instances, such as a single-node database [36, 37].

Cloud Service Models

A VM management system is often part of a larger cloud computing infrastructure, such as an IaaS platform, which is a low-level cloud service model providing tools to create and manage virtualized resources. For example, an IaaS platform typically provides tools to create and manage VMs, their storage and virtual networks for them to reside in. As such, an IaaS platform is a foundation for building and managing cloud environments [38].

Other cloud service models include Platform as a Service (PaaS) and Soft-

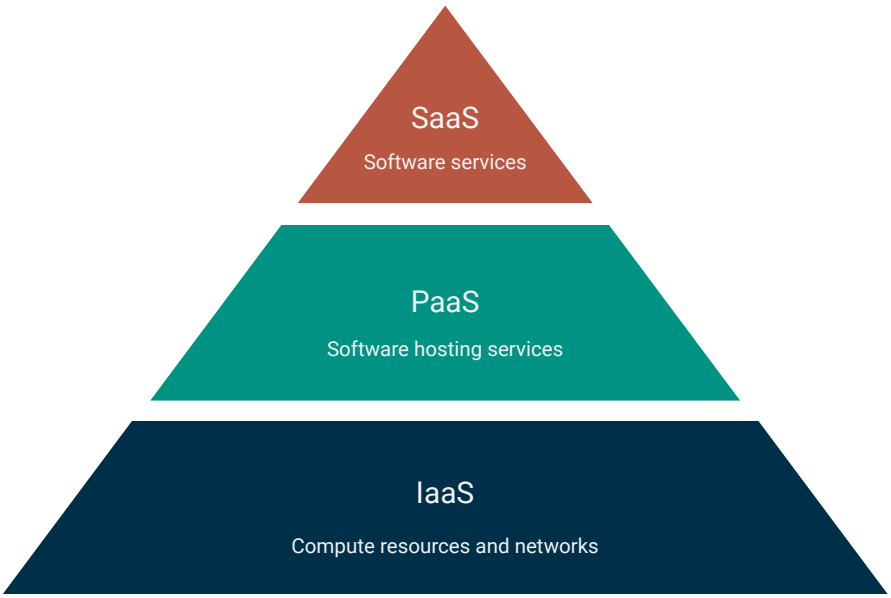

Figure 2.5: Overview of cloud service models. The figure is designed as a pyramid to illustrate the hierarchical relationship between the different cloud service models that allows a higher-level service to be built using a lower-level service.

ware as a Service (SaaS). They provide a higher level of abstraction than IaaS providing platforms and software respectively. Together, the three cloud service models form a hierarchical relationship with each other, which means a SaaS service can be built on top of a PaaS service, which in turn can be built on top of an IaaS service. Figure 2.5 illustrates an overview of the cloud service models. However, both PaaS and SaaS are out of the scope of this project [39].

2.3.1 Comparison of VM Management Systems

Managing VMs can be done in several ways depending on the required level of availability and scalability, as well as economic factors. There are numerous VM management systems available, each with its own set of features, capabilities and limitations, and they can be divided into two main groups depending on which type of system it is part of: public and private cloud. Public clouds refer to the commercial alternative maintained and operated by a third-party company. The cost of hosting resources on these platforms is usually tied to the usage and not the hardware itself. Private clouds on the other hand refer to

software that can be installed on a group of servers hosted by the organization using them. For this reason, private clouds are commonly open-source and free to use. Some of the most prevalent private cloud platforms are OpenStack [40], CloudStack [41], and OpenNebula [32], and some of the most popular public cloud platforms are Amazon, Azure, and Google Cloud. The public clouds offer far more than just VM management. However, to keep the focus of the comparison, only the VM service of each public cloud will be brought up. These are Amazon EC2 [42], Azure Virtual Machines [43] and Google Compute Engine [44].

OpenStack

OpenStack is an open-source IaaS platform that provides a large set of tools for managing virtualized resources. Its architecture is built to be modular using a component-based approach with each of the services it provides running as a separate component. The components then communicate in a distributed manner using technologies such as message queues [32] [40].

While OpenStack includes a large set of components, only the three major and most relevant to this project are brought up below.

- **Nova**
  The VM service that provides a set of tools for managing the life-cycle of VMs, such as creating, deleting and migrating VMs. Nova is a relatively large component that is split into multiple parts each with a *shared-nothing* and *messaging-based* policy, meaning they are independent and communicate with other components using message queues.

- **Swift**
  The service that manages static data in a scalable and redundant manner. This is used by *Nova* to store images and backups.

- **Glance**
  The image service that provides a catalog of VM images that can be used when creating new VMs.

In summary, OpenStack supports many features regarding VM management as well as a wide range of hypervisors. It is built to be efficient and scalable and has thus gained enormous traction in the industry [32].

CloudStack

CloudStack is an open-source IaaS platform and cloud computing software that manages virtualized resources including VMs, virtual networks and storage resources. It is used primarily to build and manage private cloud environments [41]. Architecture-wise, CloudStack is built in a monolithic fashion, encapsulating all the management features into two software packages called the management server and agent. The management server can be viewed as a controller node and is responsible for sending out instructions regarding the life-cycle of VMs, such as creating, scaling and deleting them. The agent, on the other hand, is installed on each node designated to running VMs and is responsible for low-level management tasks of the VMs on that node, such as interacting with the underlying hypervisor. The agent can thus be viewed as a intermediator between the hypervisor and the management server that enables CloudStack to be hypervisor agnostic. While CloudStack is a monolithic system, it is designed to be horizontally scalable and can be deployed in a distributed manner with multiple management servers [41] [45].

CloudStack supports a large variety of hypervisors, such as VMware, KVM, and XenServer, as well as storage services [41]. In summary, CloudStack provides an efficient public cloud environment similar to OpenStack. However, CloudStack does not scale to the same size an OpenStack due to its simple design [32].

OpenNebula

OpenNebula is an open-source IaaS platform that is mainly used to build private clouds. Contrary to OpenStack, OpenNebula is built to be simpler with as little overhead as possible, which has made OpenNebula a common choice for larger-scale datacenters [32]. Its architecture is designed to be modular and is split into three parts as described below. [46, 47].

- **Tools layer**
  The high-level layer that provides a set of interfaces such as a CLI and web-based user interface to interact with the system.

- **Core layer**
  Provides core functionalities to the system, such as host management, VM management and virtual network management.

- **Driver layer**
  The low-level layer that provides an abstraction on top of the underlying

infrastructure, including networks, storage and hypervisors. Having this layer allows OpenNebula to support a wide range of providers, as well as adapt quickly to new technologies.

In summary, OpenNebula provides a simple yet efficient approach to VM management. Its simplicity and low overhead have made it a popular choice in the industry [12].

Amazon EC2

Amazon Elastic Compute Cloud (EC2) is a web service that provides on-demand compute resources over the cloud [42, 48]. EC2 is part of the cloud platform called Amazon Web Services (AWS) and is used to host general-purpose applications. Since AWS is a public cloud platform, the technical specifications regarding hypervisor technology and such are not visible to the end user.

AWS provides a large set of templates that can be used when creating VMs, called Amazon Machine Images (AMIs). Which images is used depends on the use case and how much a user is willing to pay [42].

Azure Virtual Machines

Azure Virtual Machines is a service provided by the public cloud platform Azure to manage VMs [43] [49]. Similar to AWS, Azure provides a set of preconfigured templates that can be used when creating VMs are chosen depending on the use case, such as general-purpose or HPC. Any technical specifications on the architecture of Azure Virtual Machines are not visible to the end user [50].

Google Compute Engine

Google Compute Engine is a service provided by Google Cloud to manage computing resources in the cloud [51]. Similar to both AWS and Azure, Google Cloud also provides a set of templates when creating VMs depending on the specific workload [44].

Comparison

OpenStack, OpenNebula, and CloudStack are all examples of software that can be used to build both private and public cloud platforms. A private cloud is an on-premise cloud that is hosted within an organization and is, more often than

not, dedicated to internal use. These systems are also often free and open-source, making them a go-to tool for many organization that seeks to host internal services. The systems are also highly configurable to work on most existing infrastructure. However, being highly configurable could be seen as a double-edged sword since it leaves much of the configuration of the system to the user, such as preconfigured VM templates. [52, 53].

Amazon EC2, Azure Virtual Machines and Google Compute Engine are examples of VM management systems in public cloud platforms. They are designed with the end user in mind by providing a complete abstraction layer over the underlying hardware and hypervisors. This means that any configuration regarding the hypervisor and its settings is taken out of the equation. All three cloud platforms can be viewed as complete implementations of the previously mentioned private cloud platforms, including hardware. For this reason, the public cloud platform has seen enormous traction in the industry. However, this also creates a trade-off where the user has less control and customization but instead gains a more user-friendly and complete system [52].

Private and public clouds both provide solutions for managing virtual resources and have their respective advantages. Public clouds scale better performance-wise since they make use of large data centers. However, it inevitably involves a third party which might not be suitable for certain use cases, such as for applications managing sensitive data. Private cloud, on the other hand, comes with a larger initial cost, both in terms of purchasing hardware but also to acquire the required skillset among the personnel managing the system. Which solution to choose depends on the specific needs of the organization, such as the level of control required, customization needs and security requirements [53].

2.4 Containers and Kubernetes: A Paradigm Shift

Besides the common cloud service models (IaaS, PaaS, and SaaS), another service model has emerged, called Container as a Service (CaaS) [54]. CaaS is a container-based cloud service model that has gained significant traction due to the lightweight and portable nature of a container, allowing for simple application bundling that can run in a vast range of environments [55].

Similar to VMs, containers are often used together with a management platform, such as Kubernetes. More specifically, Kubernetes is a container orchestration platform that manages the life cycle of containers and any com-

munication between them. Kubernetes will be described in technical detail in a later section, see 2.4.2

A common use case for containers is to form the base for a microservice architecture. Microservices is a software design pattern that uses a set of small, independent and loosely coupled applications to provide a scalable service with minimal downtime [55].

2.4.1 Containers Explained

Containers are a form of virtualization that offers a lightweight alternative to VMs. Unlike VMs, containers do not require a hypervisor to run and instead run on top of an OS and share the OS kernel with other containers. By not requiring a hypervisor, containers can run with less overhead than VMs, making their life cycle relatively fast and efficient compared to VMs [6, 56]. The differences between container and hypervisor virtualization are visualized in figure 2.6.

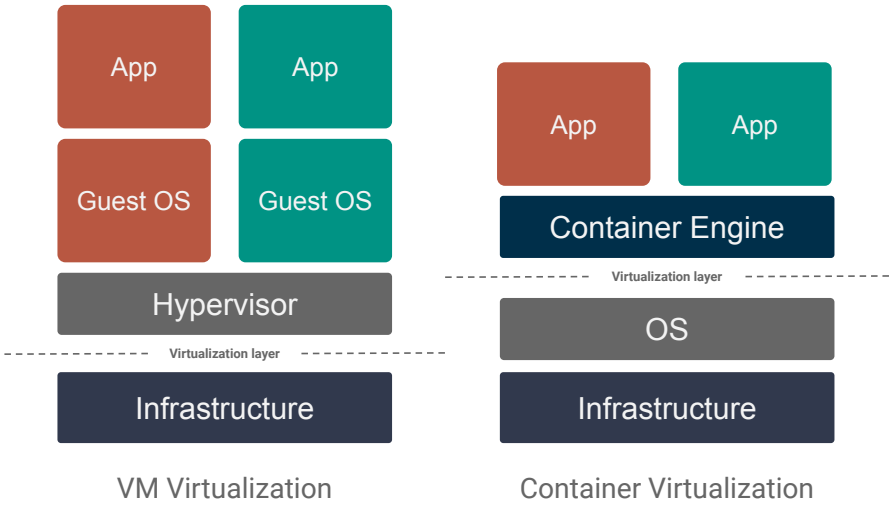

Figure 2.6: Overview of VMs and containers. The virtualization layer is placed on a lower level for VMs than for containers, illustrating the difference in their respective virtualization techniques.

A container is created from a container image, which is a lightweight package that is used when configuring the virtual environment for the container. It encapsulates the application together with any dependencies such that it runs on any platform, which makes the container itself portable and consistent. The image itself is created using a configuration file called *Dockerfile**, which de-

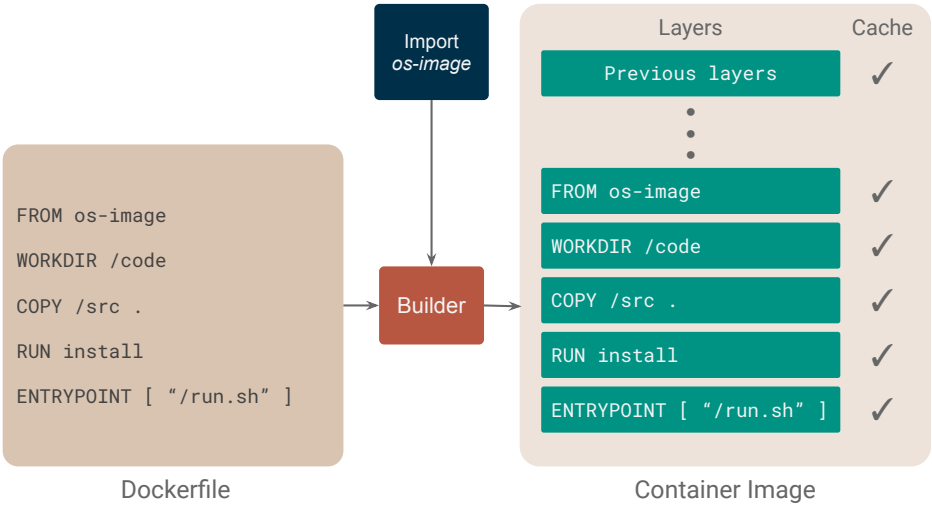

Figure 2.7: Overview of container image layers where each operation in a Dockerfile represents a layer that can be cached and reused. The layers are stacked on top of each other and appended to the parent image to form the final container image.

scribes a parent image, such as an OS image, together with several steps required to produce the container image. Once the container image is created, it can be used as a parent image in any other configuration files, creating a chain of dependencies.

To enable caching, a container image can be divided into a series of layers. Each layer represents a change to the parent image, such as adding a file or installing a package, and thus acts as a checkpoint. This enables container images to be built incrementally, where layers are only rebuilt when a change is made [57, 58, 59]. Figure 2.7 illustrates the concept of container image layers, where each operation represents a change to the parent image.

Moreover, container images are often stored in container registries, which provide central storage and are used to distribute the images. A container registry often comes with tools to manage its images, such as tools for version control and scanning for malicious images. Another common event-based feature is webhook. It acts as a message broker for predefined events, such as notifying an external system that a new image was pushed. Together these features enable a container registry to integrate well in a cloud-native environment and development pipelines [60, 61].

Originally, containers were designed to run on top of a technology developed by the company that invented containers called the *Docker Engine*.

Docker Engine is a container runtime that creates and manages containers similar to a hypervisor. However, to standardize an interface for container communication, the Open Container Initiative (OCI) was created, which defines a group of standards that aims to make containers portable across different platforms [62]. It is split into three standards called the runtime specification, image specification and distribution specification. From these specifications, the *containerd* runtime was created, which aims to provide a more transparent and open-source alternative to the Docker Engine. Since it adheres to the standards, it allows for simpler integration with tools and services built on top of it. Today, many large systems use containerd under the hood, such as Kubernetes [63]

* Recently, some movement has been made to change the name to *Containerfile*, as this would describe the content better than the company that invented the technology. However, *Dockerfile* is still the most widely used term.

2.4.2 Orchestration Using Kubernetes

Kubernetes is an open-source container orchestration platform that manages containers. It uses a set of abstraction layers to manage the life cycle of containers and all the communication between them. It provides a complete environment for the containers to run in, including scheduling, scaling and networking [6].

Kubernetes defines many abstractions to simplify container orchestration and communication, where the most basic ones are *Pods* and *Services*. A Pod is the smallest deployable unit in Kubernetes, which groups one or more containers into a single unit that can be run on nodes. This means that Kubernetes ensures all containers in a Pod run on the same host. The encapsulation allows coupled containers to reside on the same host and enables efficient communication between them. A common design pattern for containers in a Pod is the sidecar design pattern. It involves designating one or more containers to provide some additional functionality to a main container, such as logging or monitoring [64, 65]. A Service represents a communication entry point that is placed in front of one or more Pods. This means that a Service targetting a group of nodes acts as a load balancer, while a service that targets a single Pod is comparable to port-forwarding. Separating the communication into a separate layer is especially useful in scenarios where the Pods can serve client requests independently, and that the requesting client does not care which Pod it targets. Such Pods are referred to as *fungible* Pods and are more common

for cloud-native applications. Additionally, using Services in front of a group of Pods can lead to simpler deployment of updates, as the Pods can be updated one at a time using a rolling update strategy [66].

To enable a more consistent deployment of Pods, Kubernetes provides an abstraction on top of the Pods called *Deployments*. A Deployment is given a desired state in which Pods should be and actively operates towards it. This means that if a Pod fails, the number of running Pods no longer matches the desired number of Pods, and the Deployment should create a new Pod. By following a desired state, Deployments are commonly used when the load of an application is irregular. For instance, a website might face heavier traffic during the day than during the night. In this scenario, the Deployment could be adjusted to scale down the number of Pods [67, 63].

So far, none of the scenarios have included a state that should be kept between or within Pods. Containers are inherently stateless by design, meaning that any data inside a container is lost upon deletion. Kubernetes primarily provides three abstractions to manage the state of an application: *PersistentVolume*, *PersistentVolumeClaim* and *StatefulSet*. The former two are used in conjunction, where a PersistentVolume defines a storage target, such as the file path in a network storage server or on the local node, while a PersistentVolumeClaim acts as the intermediator between a PersistentVolume and a Pod. There is a one-to-one relationship between a PersistentVolume and PersistentVolumeClaim to ensure consistent access to the underlying storage, but also to provide a mechanism to notify Kubernetes that storage is no longer used [68]. A StatefulSet, on the other hand, is more comparable to a Deployment with a specific order for the Pods to start in. As previously described, a Deployment ensures that a desired number of Pods are running, but it does not guarantee the order in which they start or stop. A StatefulSet solves this by providing a more predictable strategy where each Pod has its own identity that is preserved between restarts. For this reason, a StatefulSet is often used for applications that define roles for a group of nodes, such as a database that must start a primary node before any secondary nodes [69, 63].

Figure 2.8 presents a brief overview of the Kubernetes abstractions and how they interact with one another.

In some more sophisticated scenarios, these built-in abstractions might not suffice since they might not be able to model more complex types. For such scenarios, Kubernetes provides Custom Resource Definitions (CRDs), a form of extension, that allows user-defined types with logic and validation. For instance, a CRD can define an abstraction similar to a Deployment, but it enforces certain sidecar containers to exist using custom validation rules. With

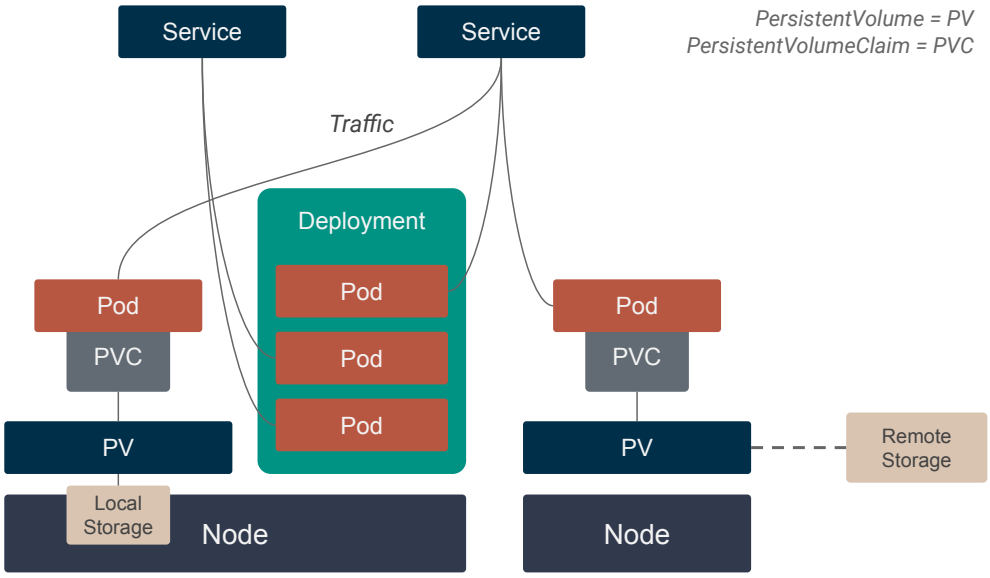

Figure 2.8: Brief overview of Kubernetes abstractions including many of the core components. The figure illustrates the communication between the different components and how storage is managed in Kubernetes.

CRDs, the Kubernetes ecosystem can be tailored for specific use cases forming a more feature-rich environment [70, 63, 71]. Kubernetes includes many more abstractions, each with its use case. However, presenting an exhaustive list falls out of the scope of this project.

The design and workflow of Kubernetes resources are inherently declarative, meaning that instead of providing a set of instructions to move a resource into a specific state, a complete state is given in the form of configuration files called manifests. The manifest defines metadata, relationships and dependencies for a resource, and Kubernetes controllers are responsible for ensuring this state is applied and up to date [72]. Depending on the kind of abstraction, different logic is applied to ensure the resource specification matches the manifest. For instance, a Pod might be rescheduled to ensure it matches the manifest [6, 63].

Architecture-wise, Kubernetes uses a client-server approach. The server, often referred to as the control plane, manages the state of the cluster, including scheduling and scaling, as well as networking. Since it consists of the API server, it can be viewed as the gateway to cluster, processing and acting upon the request of clients [73]. The client, often referred to as the worker node, receives Pod-related instructions from the control plane. Contrary to the control

plane, a worker node consists of a container runtime, which allows it to run containers [74, 63, 63, 75].

From an end user's perspective, Kubernetes resources are managed through the API server, which in turn notifies its workers. An end user can send requests using a CLI tool such as *kubectl* or using a programmatic interface through a REST API [63].

Through its abstractions, life cycle management and declarative client-server architecture, Kubernetes provides a complete and feature-rich platform for container orchestration. This has led to enormous traction in the industry and has Kubernetes a popular choice for container infrastructure [16].

2.5 Containers vs. Virtual Machines

Virtualization has brought many benefits within the realm of cloud computing by allowing hardware to be split or grouped into virtual hardware. It has been the key in enabling what is today referred to as cloud computing and cloud platforms. There are mainly two technologies that were made possible due to virtualization: VM and containers. VMs are complete and isolated systems with their own OS and containers are a lightweight alternative to VMs. This creates a trade-off where depending on the use case, a VM might be more suitable or vice versa [6, 76].

2.5.1 Comparative Overview

The main difference between VMs and containers revolves around isolation and how independently they can run. As previously described VMs provides a complete system while containers share the OS kernel with other containers. This can lead to more efficient resource sharing among the containers, but it also increases the attack surface of containers. A vulnerability found in the underlying OS might compromise the containers running on the host. A VM reduces this risk by removing the OS as a dependency [77].

Isolation is also important when considering the scenario of co-location, meaning containers and VMs running on the same host. Since containers are allocated resources from the OS, they are allocated as if they were ordinary applications. In many scenarios, this leads to efficient resource utilization, but sharing resources this way can be a double-edged sword. For instance, containers are more vulnerable than VMs to the noisy neighbor problem, since a co-located container could consume an unproportionate amount of the host's resources, which could lead to resource starvation among the other containers

running on the host. Some solutions exist for this, such as the OS-level feature *cgroups*. A cgroup is a mechanism that limits, prioritizes and isolates resource usage for a process [78, 79, 80]. VMs, on the other hand, are not as susceptible to the noisy neighbor problem since they are allocated a static amount of resources from the host. However, it is important to note that a VM does not reduce the noisy neighbor problem for all resource types. For instance, unproportionate consumption of network traffic is still an issue for both VMs and containers [76].

Overcommitment refers to a mechanism that allocates more virtual resources than what is physically available. This is a common practice when using virtualization, as hardware resources are often not fully utilized. Containers use mechanisms built into the OSto achieve resource overcommitment, which involves multiplexing and virtual memory. However, resource overcommitment is more challenging for VMs. While CPU overcommitment is achieved using Virtual CPU (vCPU), memory overcommitment is solved by more sophisticated methods since a hypervisor allocates a static amount of memory to each VM. One such method is called memory ballooning and it involves sharing a subset of each VM's memory with other VMs. It works by reclaiming memory that is not used inside a VM and allocating it to another VM. Then, when the first VM requires the now reclaimed memory, the hypervisor ensures the VM is either given back its memory or allocated a new chunk. As such, memory ballooning operates under the assumption that certain parts of the allocated memory are not used [81]. However, it is important to note that a technique like memory ballooning does not directly solve memory overcommitment, but instead reduces the need for it [76, 82]. Figure 2.9 illustrates the processes of deflating the memory of a VM and how it is used in another VM.

Live migration is a technique used to move a running resource to a new host without stopping and starting it. The technique is often used when balancing load over hosts or when performing maintenance. For VMs, live migration is a mature and widely used technique and works by copying memory pages from the source host to the destination host and then transferring the VM from the source hypervisor to the destination hypervisor. Since VM disks are often stored on shared storage, such as a Storage Area Network (SAN), the VM disk is not transferred but instead reattached after the live migration is finished. This usually requires some form of time-freezing to reduce the risk of data corruption. However, a live migration operation is usually fast enough to even notice [83]. Live migration for containers, however, is not as mature as for VMs. A container requires OS-level process migration, which is less reliable.

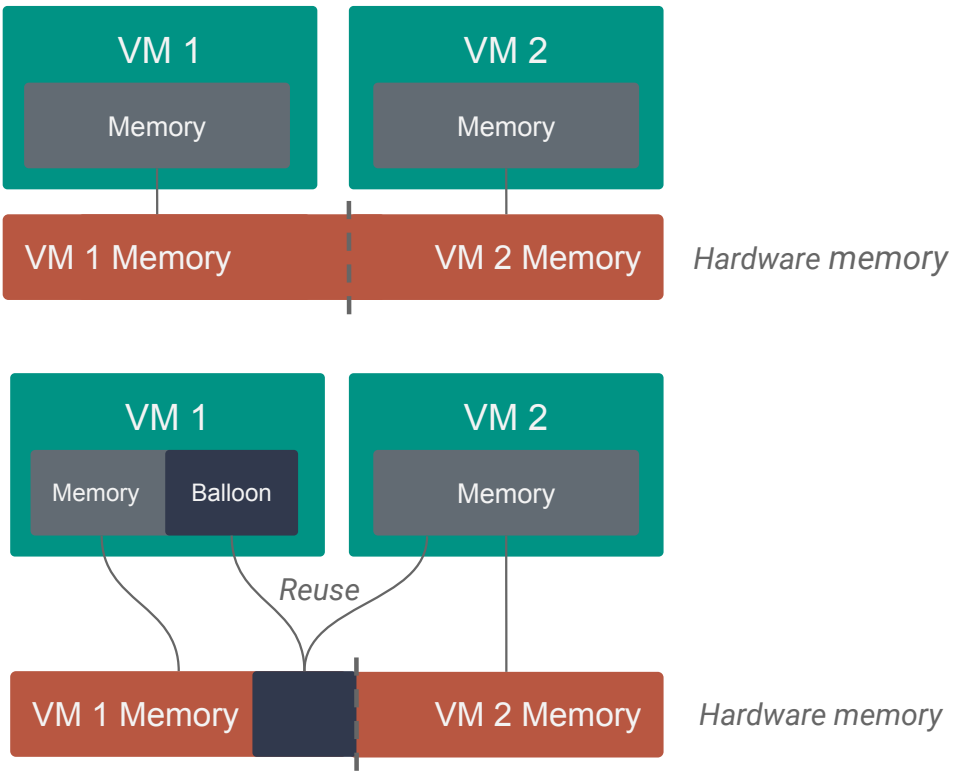

Figure 2.9: Overview of memory ballooning in two stages. The figure illustrates the memory of a VM is shrunk by the hypervisor (deflation) and then reused by another VM (inflation).

However, several projects aim to provide live migration for containers, such as Checkpoint-restore in Userspace (CRUI). CRUI works by checkpointing the state of a container, including its memory and file descriptors, and then restoring the state on another host. While this is considered as the defacto container live migration, it is more comparable to a snapshot operation than a live migration operation [84, 85, 86]. Another approach for live migrating containers involves terminating them on one host and restarting them on another. This strategy assumes the container is stateless or utilizes shared storage for its state, as is often used in Kubernetes since it does not provide container live migration. Instead, Kubernetes facilitates persistent storage through its persistent storage abstractions, such as PersistentVolumes and PersistentVolumeClaims. This means a user maps portions of the container's state to shared storage and assumes the rest can be discarded when a container is terminated [63].

2.5.2 Decision Factors

The choice between containers and VMs depends on the specific use case, and there are several factors to consider when deciding which technology to use. Some of the core factors are security, scalability and legacy support [76].

As previously described, using a container implies a larger attack surface than for a VM due to sharing the OS kernel. For this reason, VM have commonly been associated with stronger security since they are less prone to security vulnerabilities. This makes VMs an excellent candidate for application with stricter security requirements. Some examples include applications managing sensitive or confidential data [87]. Another use case that is made possible due to a VM's strong isolation is running unsafe or untrusted code. This use case is often referred to as sandbox and is useful when evaluating potentially malicious code that could otherwise risk harming the host system. Untrusted code could also involve user-submitted code, such as in GitHub pipelines that use VMs to execute actions based on some predefined event. This could be tests or some other development-related tasks created by a user [88]. Using a VM in these scenarios ensures that the untrusted code does not affect the hosting system [89, 90]

Configuration drift is another aspect to consider when deciding between using a VM or a container. It is a phenomenon that occurs when the configuration of a system changes over time, such that it could potentially lead to security risks. This is usually caused by continuous manual intervention in a system, where the summarized effects of all the changes made could be an unstable system [91, 92]. This is more common for VMs than for containers, as a

VM's state is saved indefinitely. A container mitigates this problem by having its entire state defined in the Dockerfile (see 2.4.1), where its dependencies and configuration are more transparent to the person managing it. A container state is thus discarded once it is stopped, which allows containers to have a more consistent environment than VMs. This is further supported by the fact container registries (a central location for container images, see 2.4.1) usually include image-scanning tools to find vulnerabilities in a container image itself [93].

Scalability is a factor that is concerned with how well a system adapts to an increase or decrease in load. This can be measured in many ways where one being the overhead each additional computational unit has. Containers being lightweight and sharing hardware resources more efficiently implies they can scale up and down both quicker but also with a smaller resource footprint. However, as previously described, they are more susceptible to the noisy neighbor problem, where co-located containers could interfere [80, 93]. Moreover, scalability is also related to cloud-native applications, which are applications that are designed to run in the cloud [7]. A cloud-native application is characterized by being horizontally scalable, stateless, and resilient, and is often built using microservices, see 2.4. Cloud-native applications thus require small and lightweight environments making containers a natural fit.

Legacy support is another relevant factor to consider when comparing VMs and containers. It can be measured by how well a system can keep running old software or software depending on older hardware. In both cases, VMs is a good fit since they are a complete system. A VM can simply replicate the state of a legacy system, even including the hardware that was used. This allows an organization to acquire new infrastructure while still maintaining its legacy applications. Containers are also able to support legacy applications, but not to the same extent, as they are inevitably using a modern kernel [93, 94].

2.5.3 Towards Stronger Container Isolation

So far, containers have been described as having weaker isolation than VMs due to it sharing the underlying OS kernel. However. several projects have been created to tackle this issue without completely resorting to a VM. These include Firecracker, Kata Containers and gVisor.

Firecracker is a KVM-based hypervisor that creates lightweight VMs, called microVMs. Compared to an ordinary VM, a microVM is much more lightweight and can start in a fraction of a second. Firecracker achieves such speeds

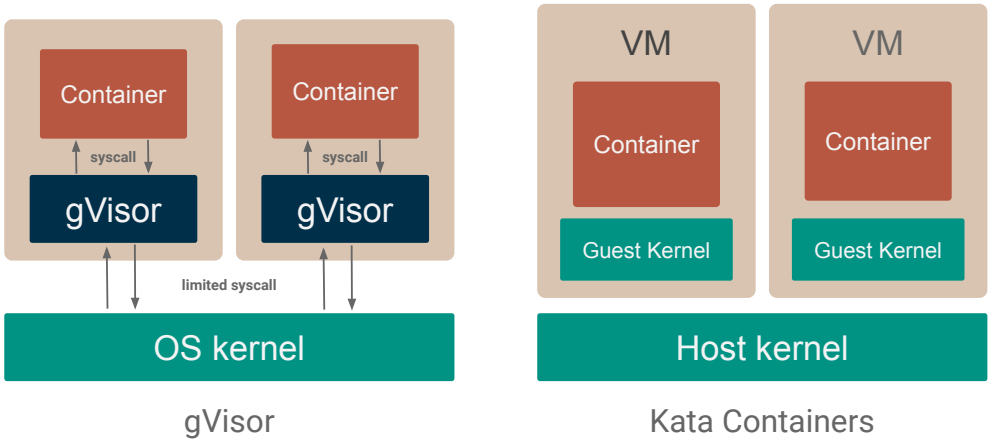

Figure 2.10: Overview of gVisor and Kata Containers and their approach to providing a more secure execution environment for containers. The figure illustrates the different approaches where gVisor operates as a proxy for system calls and file access, while Kata Containers wraps the container in a VM.

through optimizations and a heavy stripped-down guest [8, 95].

Kata Container is another VM alternative that aims to provide a lightweight environment, while still having strong isolation properties. Unlike Firecracker, a Kata Container still uses a container, but wraps it using a lightweight version of a VM, which enables Kata Container to act as a runtime, such as containerd. This means Kata Container is simpler to implement than Firecracker since it can run ordinary containers [8].

Another approach to strong container isolation is gVisor, an extension to a container that enforces stricter isolation by restricting system calls. This approach differs from both Firecracker and Kata Container in that it does not use a VM at all, and instead relies on intercepting system calls and file access requests [96].

In summary, these three technologies provide different approaches to the same problem. Firecracker is more VM-like, while gVisor is most similar to a container. However, due to the varying added overhead, they are yet to be widely adopted [8]. Figure 2.10 illustrates the differences between how Kata Containers and gVisor wrap a container to provide a more secure execution environment.

2.6 Extending Kubernetes with KubeVirt

Kubernetes was recently extended to accommodate VMs as a native resource through the KubeVirt project. KubeVirt is designed to integrate well with the Kubernetes ecosystem and provide a similar interface to VMs as any other Kubernetes resource. VMs created with KubeVirt are scheduled and communicate as ordinary Pods, allowing them to blend into the environment [97]. This means that a Kubernetes resource that integrates with a Pod can also integrate with a KubeVirt VM. For instance, Services can expose a VM externally and PersistentVolumeClaims can maintain its persistent storage. While KubeVirt uses Pods to wrap a VM, a VM abstraction is implemented as a CRD. This allows KubeVirt to perform validation that is specific to a VM, such as referencing a non-existing disk [98, 99].

KubeVirt's design pattern follows Kubernetes conventions, using a declarative approach to managing the VMs and any other resources revolving around it, such as snapshots and live migration. A VM is split into a *VirtualMachine* and *VirtualMachineInstance*, where the VirtualMachine represents a desired state and the VirtualMachineInstance describes a running state. This relationship is similar to that of a StatefulSet and its Pods [100]. Moreover, KubeVirt implements a VM controller to ensure the desired state is met, which is responsible for requesting an environment in which the VM can run, but also update the running VM when its desired configuration changes. [101, 98].

KubeVirt's declarative approach stretches to aspects that have been imperative operations in traditional VM management systems. The most prominent examples of this are live migrations and snapshots that are both requested using configuration files. As such, they are implemented as CRD that triggers live migration or snapshot operations once created [102, 103]. This is a novel take on operations that traditionally have been imperative. However, the processes of triggering an operation upon resource creation have existed in Kubernetes for a long time. For instance, a storage device is connected to a Pod when a PersistentVolumeClaim is created [104].

A consequence of being built on top of Kubernetes is that KubeVirt-related resources can be managed using the same tools used for other resources, such as the CLI tool kubectl. However, certain operations are still unique to VMs and are not yet supported by kubectl. This includes accessing a VM over Secure Shell (SSH) or Virtual Network Computing (VNC), which are inherently imperative operations. For this reason, KubeVirt provides a complementary CLI tool that implements these missing features [105].

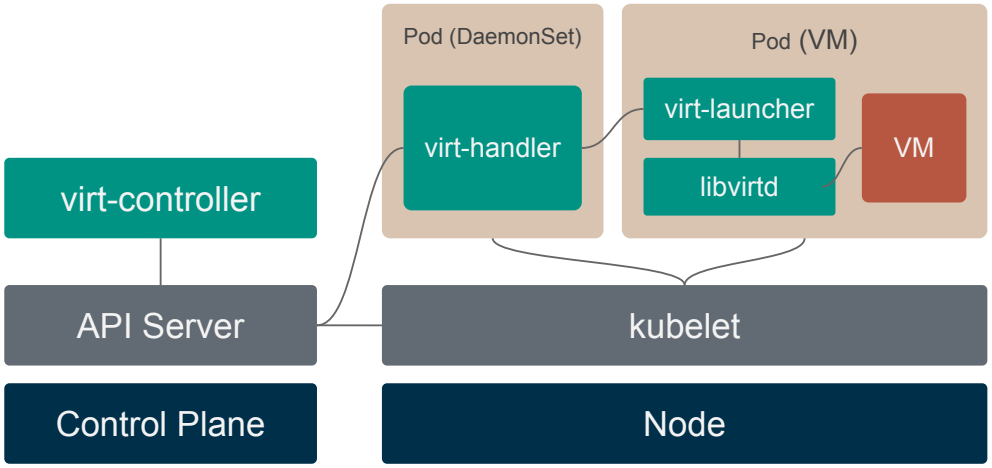

Figure 2.11: Overview of KubeVirt components, their logical place in the Kubernetes ecosystem and how they interact with each other. The green components represent the KubeVirt internal components.

2.6.1 KubeVirt Technical Overview

On a technical level, KubeVirt does not implement a new deployable unit for their VMs and was instead designed to encapsulate them using Pods (see 2.4.2). This design has allowed KubeVirt VMs to integrate well with other Kubernetes resources such as Sevices and PersistentVolumeClaims.

KubeVirt can be split into four core components: *virt-controller*, *virt-handler*, *virt-launcher* and the *libvirt daemon*. The virt-controller and virt-handler are both concerned with managing the life cycle of the VMs, where the virt-controller is responsible for creating the Pod for the VM and the virt-handler is responsible for creating the VM itself. The virt-launcher manages the runtime environment for a VM making it responsible for requesting start and stop operations, as well as monitoring. Finally, the libvirt daemon (libvirtd) provides the interface to manage the VMs hosted by the underlying KVM hypervisor. The relationship between the three components is illustrated in figure 2.11, where the virt-controller operates closest to Kubernetes API-server, virt-handler operates on every node to communicate with the underlying hypervisor, and virt-launcher operates as a sidecar container in every Pod to manage the runtime environment of the VM [98, 101].

2.6.2 KubeVirt and GitOps

GitOps is a set of practices that enables using Git as a storage for the desired state of a system. Using a storage managed by Git allows resources to be version-controlled and audited. This approach keeps developers and maintainers in focus since it allows their resources and codebase to exist in the same system. For some scenarios, the resources managed in Git might even be the same resources that host their application [106]. For this reason, GitOps becomes a natural fit for KubeVirt since VMs are managed in files [107].

2.6.3 Comparison with Traditional Approaches

So far, KubeVirt has been presented as a silver bullet for VM management. However, it is important to note that KubeVirt does not represent a new virtualization technology, but rather a unifying technology. Behind the hood, KubeVirt uses Quick EMUlator (QEMU) and KVM to create the underlying VM, which is similar to any other traditional VM management system. This means that KubeVirt should in theory provide the same level of isolation and performance as any other management system. However, KubeVirt does fall short in that the VM is encapsulated inside a Pod, as this could lead to a performance overhead. Another aspect to highlight is that KubeVirt itself is not directly comparable to existing VM management systems, but rather forms a fully-fledged system when evaluated together with Kubernetes. For example, KubeVirt's implementation does not implement its scheduler or storage solution and instead relies on Kubernetes implementation and integrations, while OpenStack has its own scheduler as well wide support for different storage solutions including data warehousing [108, 98, 109, 110].

2.7 Related Work

2.7.1 Current challenges

The Cloud Native Computing Foundation (CNCF) presents in a report some of the current challenges of managing VM using traditional approaches, such as an IaaS platform [19]. They argue that a platform such as OpenStack is overly complex and difficult to manage for an organization that only wants to manage VMs. They conclude by suggesting KubeVirt with Kubernetes as a simpler alternative to VM management.

Moreover, the CNCF report also mentions the potential benefits of a unified platform for both VMs and containers. They argue that a unified platform reduces the required skillset and management burden for personnel, as well as reducing the architectural complexity of the system. These benefits are supported in a report by Kubacki [20]. He argues that managing both a traditional VM management system and a Kubernetes could lead to unnecessary costs for an organization. Additionally, he argues that by allowing VMs and containers to exist in a single platform VM-only organizations could potentially be driven toward adopting containers for suitable workloads.

2.7.2 KubeVirt challenges

While KubeVirt is presented as a solution to many challenges related to managing VMs, it is important to consider the challenges that come with KubeVirt. A study by Trakulmaiphol and Piromsopa presents some shortcomings when moving into a KubeVirt-based system and found some issues related to how networking [111]. One of the issues was related to how IP address management differs in Kubernetes compared to traditional VM management systems. For instance, static IP addresses were difficult to assign and manage. In the study, they developed a framework to solve some of the issues on the application level, such as DNS queries, and provided a set of guidelines related to KubeVirt's unique features including the Multus Container Network Interface.

A common practice when managing Kubernetes clusters is to deploy Kubernetes nodes on VMs created by a VM management system. This means that when using KubeVirt, VMs are created inside other VMs leading to nested virtualization. Ye et al. argue in a study that nested virtualization raises security concerns since it increases the size of the [112]. A larger means that the size of the system's attack vector increases, which in turn leads to an increased risk for vulnerabilities. This study is particularly relevant as it highlights a potential limitation if KubeVirt is simply seen as another Kubernetes resource and not as a VM management system.

2.7.3 Usability of Kubernetes and KubeVirt

Using Kubernetes as a VM management system is a relatively new concept and there is limited research on the usability of KubeVirt. However, there exist several studies that have evaluated the usability of Kubernetes, which is equally relevant since KubeVirt is built on top of Kubernetes.

Kubernetes API is inherently declarative, meaning it uses a configuration

to describe a desired state it should be in. A study by Sonninen investigates many of the possible benefits of such an approach [113]. The study found that a declarative approach enables a system to use GitOps, which is a way to manage infrastructure using files in storage backed by Git. This simplifies resource management through its built-in version control and audit, as well as rollbacks (given that it is supported by the resource). Two studies exemplify these benefits when presenting their case study on when CERN migrated an internal project from a cluster of VM to Kubernetes. The first study conducted by Imran et al. describes how migrating to Kubernetes improved the overall system management compared to using a VM cluster [114]. While many of their experienced benefits came from adopting containers instead of VMs, the study still suggests Kubernetes itself provided many benefits, such as reduced operational costs. The other study conducted by Badran presents the practical benefits they experienced by using Kubernetes and is more focused on the adoption of a declarative approach for system management [115]. It describes how using Kubernetes instead of a VM cluster allowed them to create reusable configurations that enabled their entire infrastructure to be managed as code.

The benefits of moving into Kubernetes are further exemplified in a study by Gelley [116]. It presents a case study where an organization moved its application from a PaaS provided by Cloud Foundry into Kubernetes. The study not only describes how Kubernetes provides more flexible configuration and deployment compared to the previous PaaS but also how the organization was able to make use of the tools Kubernetes provides to enable auto-scaling and monitoring. Moreover, the study concludes that by moving into Kubernetes, possible future migration to other platforms was made simpler due to how widely adopted Kubernetes is.

User experience in Kubernetes was further investigated in a study by Packard et al. that collected several projects that moved into Kubernetes [117]. The study covered many aspects, including administration such as managing persistent storage and quotas, but also developer experience such as deploying applications. The study found that system administrators in general face a relatively steep learning curve when using Kubernetes since they need to learn many of the concepts that make up an application. Interestingly, it also found that many users without prior knowledge of Kubernetes could perform basic tasks without any major issues, such as deploying an application. This finding is especially useful when evaluating KubeVirt since the ability to deploy a built-in resource in Kubernetes is similar, if not identical, to deploying a KubeVirt-based resource. Another study that investigated the user experience of Kubernetes was conducted by Boronin [118]. The study presents a case

study where an organization migrated from a monolithic system design to a more cloud-native design. The study is relevant as it is not only concerned with the technical aspects of Kubernetes but also considers the organizational impact of migrating to Kubernetes. For instance, transferring skill sets among personnel and the impact on the organization's culture.

2.7.4 Performance Evaluations

Regarding performance, several studies have evaluated popular systems that can manage VMs, including open-source alternatives such as OpenStack and CloudStack, or proprietary systems such as Amazon EC2.

A study by Kang et al. compared the performance of OpenStack and Amazon EC2 using a wide range of configurations and analyzed diverse aspects based on different hypervisor, storage and networking configurations [119]. Apart from the performance comparisons, the study also investigated performance impact when using more advanced features, such as Infiniband networking instead of Ethernet. The study showed mixed results, where OpenStack outperforms EC2 in some scenarios such as when using Infiniband networking, but EC2 tends to outperform OpenStack in parallel processing. As such, the study concluded that component-wise, each system has strengths. This study brings with it relevant perspectives as it highlights many of the aspects that are important to consider when evaluating the performance of VM management systems.

Another study by Mullerikkal and Sastri compared OpenStack and CloudStack regarding implementation complexity, stability and performance [120]. Compared to the previous study, these performance tests were designed to measure disk and network I/O, and not CPU or memory usage. The study found that OpenStack was significantly more complex than CloudStack, but that it was overall more stable. The study also found that OpenStack had a slightly but overall better performance than CloudStack. These claims are further supported by another study conducted by Paradowski et al. [121] a few years later that also compared CloudStack and OpenStack. The study benchmarked the two systems in terms of practical tasks such as time to deploy a VM and resource utilization during the deployment. The study found that OpenStack was slightly faster in most operations and had a lower resource utilization. While the tests were designed to represent user interactions, their scale was relatively small and did not explore any limitations of either system.

In contrast, a later study by Amaral benchmarked KubeVirt uses a setup with a large number of VMs relative to the size of the nodes, aiming to explore

the limits of KubeVirt's control plane (scheduler etc.) [122]. The study found that, while KubeVirt was able to create all VMs, the control plane suffered a heavy load, indicating a possible large overhead when KubeVirt provisions a large number of VMs. This finding is important to consider when evaluating KubeVirt, as it indicates that KubeVirt may not yet be suitable for managing a large number of VMs.

In terms of network performance, a study conducted by Kolahdouzan investigated the networking traffic pattern in KubeVirt and OpenStack by performing networking performance tests to evaluate potential VM overhead [123]. The study found that the network overhead was significantly larger for scenarios in which VMs reside on separate nodes. However, while the study found overhead for both KubeVirt and OpenStack, it found that OpenStack had a considerably larger overhead. While this project will not focus on networking performance, it is important to consider the findings of this study when evaluating KubeVirt.

So far, only related work that evaluated higher-level systems has been presented. However, several studies have evaluated the performance of hypervisors. A study by Hwang et al. evaluated the performance of a range of hypervisors including KVM, Hyper-V and Xen [124]. The study did not manage to draw any conclusions on which hypervisor was most performant and instead concluded that it depended on the specific use case. This conclusion is interesting as it suggests that the performance of a hypervisor could be an obsolete factor when deciding which one to use. Either way, reviewing hypervisors is important when evaluating KubeVirt, as KubeVirt uses KVM as its hypervisor.

Chapter 3

Performance Analysis

As described in the literature study, Paradowski et al. presented a set of benchmarks used to evaluate CloudStack and OpenStack [121]. This comparison was used as a foundational basis for the performance analysis in this project, employing many of its approaches for evaluating the performance of a VM management system. In this project, however, OpenNebula was compared with KubeVirt. Choosing OpenNebula as the second system was based on its popularity and the fact it is open-source, but also to ensure that the tests were relevant to Net Insight's current infrastructure.

3.1 Setup

The testing process was conducted on Azure using a group of VMs that acted as hosting nodes. This approach was chosen to ensure that the tests are not affected by the underlying hardware and to facilitate reproducibility.

A set of scripts was created to automate the entire testing process. The scripts automated node creation in Azure, as well as installation and configuration of the VM management system on the nodes. Once the tests were finished, the results were collected by the scripts and the nodes were deleted in Azure. The scripts were thus used to create a complete setup for each system in a consistent manner using a single command.

Payload-wise, this project used a similar approach to the study by Paradowski et al. with predefined types for VMs with CPU ranging from 1 to 4 CPU cores, and RAM ranging from 128 MB to 4 GB (see Table 3.4 for a detailed overview). Moreover, VMs used the lightweight Cirros Linux image, since a lightweight OS image is less likely to introduce noise in measurements [125]. This also meant a VM disk could be smaller, which allowed for more VMs to

be created per node and for tests to run quicker since a lightweight image is usually faster to boot than a larger image.

Each test case used a setup of three Azure nodes, one management node and two worker nodes. Having more than one worker node was crucial for certain tests to be able to run. For instance, a live migration test requires at least two worker nodes since it involves moving VMs between nodes. The Azure nodes were configured large enough to facilitate a broad range of tests, including tests with many small VMs and tests with a few large VMs. Azure offered many different VM types, many of which were suitable in terms of CPU and memory specification. However, only a few offered virtualization support, which was required when running nested VMs. This left the Azure Dv3-series as the most suitable VM type and, specifically, the D4v3 VM type that has 4 CPU cores and 16 GB of memory [126]. The price of the D4v3 VM type was 0.2 USD per hour, which was deemed acceptable for the project [127]. Table 3.1 shows the overview of the Azure node specification used in the tests.

Role	CPU	RAM	Disk
Control Plane (Azure D4v3)	4	16GB	100GB
Worker 1 (Azure D4v3)	4	16GB	100GB
Worker 2 (Azure D4v3)	4	16GB	100GB
Scale-up Workers 3-10 (Azure D4v3)	4	16GB	100GB

Table 3.1: Node specifications used for the nodes hosting the VM management systems.

To quantify the results of the tests, several metrics were collected on each Azure node for every test. This includes measuring the utilization of a node's CPU, memory and disk, as well as the duration of the specific task, see Table 3.2. Further technical details on how metrics were collected are described later in this chapter.

The tests performed were divided into three categories: basic, limit testing, and scaling. The basic tests were designed to evaluate the basic functionality of the VM management systems, such as creating and deleting VMs. The limit testing tests were designed to evaluate how a VM management system operates when the maximum capacity is reached in the worker nodes. The scaling tests were designed to evaluate how a VM management system handles the scale-out and scale-in of worker nodes, both with and without VMs on it (that would need to be migrated). Each test was run 10 times. See table 3.3 for an overview of the tests and their purpose.

Metric	Unit	Description
CPU utilization	%	The percentage of CPU used by the system
Memory utilization	%	The percentage of memory used by the system
Disk utilization	%	The percentage of disk used by the system
Task duration	s	The duration of a specific task

Table 3.2: Performance metrics that were used to quantify the results of the tests.

Test	Purpose
Create each type of VM and delete it	Evaluate the basic functionality of the VM management systems without addressing potential load issues.
Create 20 type-1 VMs and then delete them	Evaluate how the VM management systems manage a relatively large burst of VMs.
Live migrate a type-4 VM	Evaluate how well the VM management systems execute a migration operation without addressing potential load issues.
Live migrate 10 type-1 VMs	Evaluate how well the VM management systems manage a relatively large burst of migration operations.
Scale up and down 2 to 10 workers without VMs	Evaluate how well the VM management systems scale up and down without any load.
Scale up and down 2 to 10 workers with a type-1 VM on each worker	Evaluate how the VM management systems handle scaling operations that incur migration operations.

Table 3.3: Performance tests that were used when evaluating the VM management systems. During each test, metrics were collected for each node.

Type	CPU	RAM	Disk Size
Type 1	1	128MB	1GB
Type 2	1	1GB	5GB
Type 3	2	2GB	10GB
Type 4	4	4GB	20GB

Table 3.4: VM specifications for the different types of VMS that were used in the tests.

3.2 Building a Framework

Evaluating the performance of a VM management system is a complex task that requires a structured approach due to its many components and multi-node setup. For this reason, a framework to form standardized testing environments was created. The framework was designed to be VM management system agnostic, meaning that it can be used to evaluate any VM management system as long as the system supports the set of features defined in the framework. Consequently, this means that the framework can be extended to evaluate any other VM management system, such as the previously mentioned OpenStack and CloudStack, and not only OpenNebula and KubeVirt. However, it is important to note that the framework was designed to evaluate VM management systems that can be hosted on-premise since it quantifies the performance of a VM management system by measuring the utilization of nodes hosting the system. As such, the framework does not consider cloud-based VM management systems, such as Amazon EC2, Google Compute Engine or Azure Virtual Machines.

3.2.1 Design

The framework was designed to be modular and loosely coupled with external libraries. This design choice was made to ensure that the framework can be easily extended and modified and that it does not rely on external dependencies. To interact with a VM management system an interface was created that defines the methods that a VM management system must implement to be compatible. It defines methods for creating and deleting VMs, migrating VMs between nodes and scaling up and down the number of worker nodes, as well as methods for installing control and worker nodes. Each VM management system implements these methods in their respective packages. The full definition of the interface can be found in the appendix A.1. To host

each VM management system, the framework created a set of nodes in Azure, which were completely installed and configured before any performance analysis was conducted to form a fair starting point for every VM management system. However, any dependencies required by the VM management system were intentionally left to the installation, as unnecessary dependencies could introduce noise in the measurements.

Metrics were collected by running a daemon on each Azure node that collects utilization metrics, including CPU, memory and disk utilization. The daemon operated by fetching metrics every second and storing them in a JSON file locally. However, to only capture metrics when a test was running, the daemon only started collecting metrics when a test was initiated and stopped when the test was finished. This was controlled by placing an empty file called *run* where the daemon was running. The daemon checked for the file's existence periodically and started collecting metrics when it noticed the file was present. As such, each node timestamped its metrics locally, which meant that the timestamp of the data points could vary between the nodes. However, this distributed approach was chosen to reduce the number of network requests to a minimum and to ensure that no skew was introduced in the measurements due to network latency.

The utilization metrics were collected by a set of tools that were installed on each Azure node. Since the tools were required to be installed on every node, they were installed directly after creation. Table 3.5 shows an overview of the tools used by the daemon. Another important aspect considered was asynchronous execution. Each of the tools used by the daemon was executed in parallel to minimize the time it took to collect the metrics for a given timestamp, essentially allowing shorter intervals between each data point. The structure of the output varied between the tools and was parsed using built-in tools in Linux such as *awk* and *sed* before being stored in a JSON file. The final output file was fetched over SSH using *scp* and parsed by the framework to extract the metrics. The daemon was written in Bash to ensure that it could run on any Linux distribution and did not require any external dependencies. The code for the daemon can be found in the appendix A.2 and the framework design is visualized in Figure 3.1.

The workflow of the framework was split into four phases to form a structured approach that allows for modularity and caching. Each phase was responsible for a specific part of the workflow and, in most cases, independent of each other. For instance, the installation phase can be omitted if the nodes are already installed and configured from a previous run. However, if the teardown phase was executed in one run, the next run after it must start with the

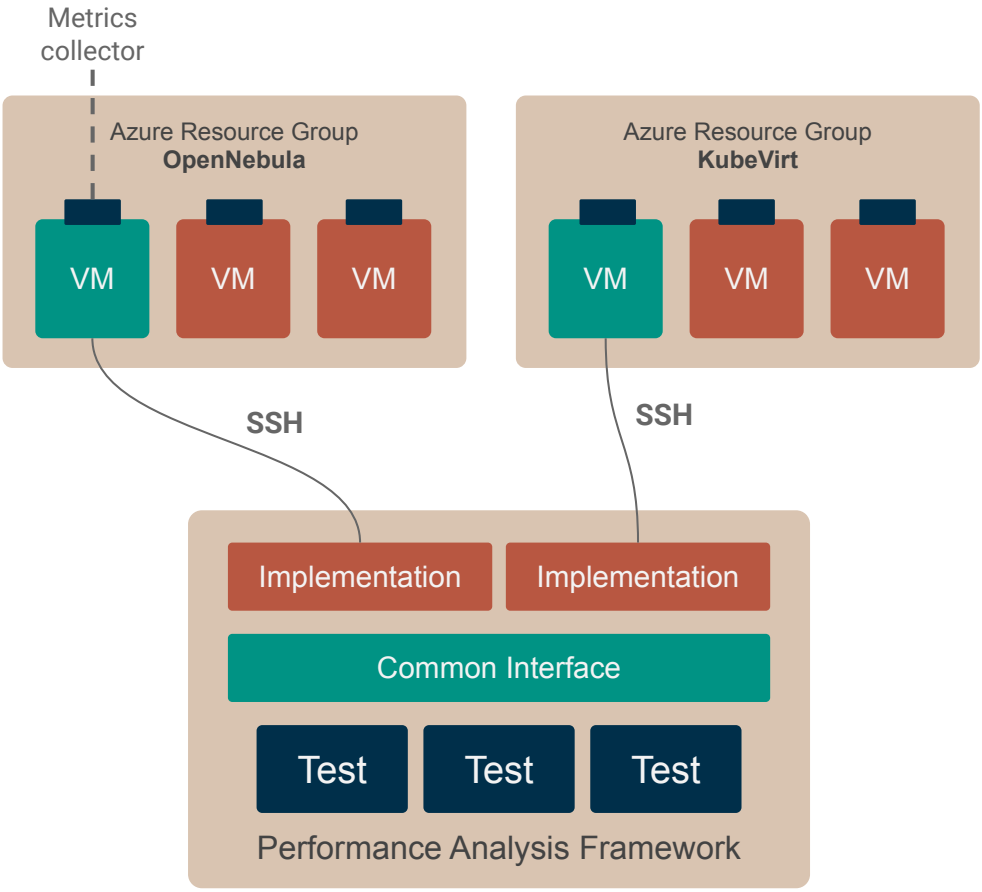

Figure 3.1: Overview of the framework designed with a framework communicating with the VM management systems through SSH. The metric collector was installed on each node to collect utilization metrics. The green VM figure represents the VM management system control node and the orange VM figures represent the worker nodes.

Tool	Utilization	Description
mpstat	CPU	Reports activities for each available processor
free	Memory	Displays the total amount of free and used memory in the system
iostat	Disk	Reports CPU utilization and disk I/O statistics

Table 3.5: Tools used by the metrics collector daemon. Each tool was installed on each Azure node during the installation phase.

setup phase. The design choice of dividing the workflow into phases was made to allow for more efficient testing, both when developing the scripts and when executing them. This was especially important considering that each test came with a cost in Azure. The entire workflow is illustrated in Figure 3.2 and is described below:

1. **Setup**: The setup phase is responsible for creating the nodes and networking in Azure. It is also responsible for installing base packages that every node requires, such as *sysstat* to collect utilization metrics and *jq* to parse JSON output.

2. **Installation**: The installation phase is responsible for installing and configuring the VM management system on the Azure nodes. Each VM management system implements an installation method that is responsible for installing and configuring the system with one control node and two worker nodes such that it can run a VM. The installation method also installs several standby workers that will be used when scaling up and down the number of worker nodes. By having standby workers, the framework can independently measure the cost of scaling up and down without the cost of downloading dependencies. While larger dependencies can be seen as a metric on their own, it is important to note that production environments usually generate OS images with prepared dependencies, rendering the cost of downloading dependencies irrelevant.

3. **Benchmark**: The benchmark phase is responsible for running the performance tests. The tests are described in 3.3 and are designed to evaluate the general performance of the VM management system. Each test is a combination of the methods the framework interface offers, such as creating VMs in different sizes, migrating VMs between nodes and

scaling up and down the number of worker nodes. This phase generates and saves the results of the tests in a JSON file.

4. **Teardown**: The teardown phase is responsible for deleting the Azure nodes and ensuring that no resources are left behind. Having this as a separate phase allows for tests to run sequentially without the time cost of creating and deleting nodes for each test.

3.2.2 Implementation

The framework was implemented in the programming language Go, a language popular for its performance and concurrency features. Concurrency was a particularly important factor as several tests involved running multiple operations in parallel, such as creating or migrating several VMs at the same time. As such, settling for Go reduced the risk of introducing noise in the measurements and could run each test more efficiently.

The most important part of the framework was arguably to provide fair and reproducible comparisons. As described previously, each VM management system implemented an interface that defined methods for creating and deleting VMs. While this provided uniform operations from the framework's point of view, it also made the method's implementation crucial: KubeVirt's implementation of creating a VM should be as close to OpenNebula's implementation as possible to ensure that the comparisons are fair and that no unnecessary overhead are introduced. One example of this was that any network-related configuration had to be omitted since network-related metrics were not considered in the project. Another example was that the scheduling configuration had to be edited to ensure that VMs were created in as similar a manner as possible. However, all features that a VM management system offered and that were beneficial for the specific operation were considered. For instance, OpenNebula did not allow a live migration action to be performed on a VM that is not running, while KubeVirt allowed the operation to be requested despite the state of a VM. This allowed KubeVirt to omit the waiting time for a VM to be in the correct state before requesting a migration action.

Below is an extraction of OpenNebula's and KubeVirt's implementations for creating a VM.

KubeVirt

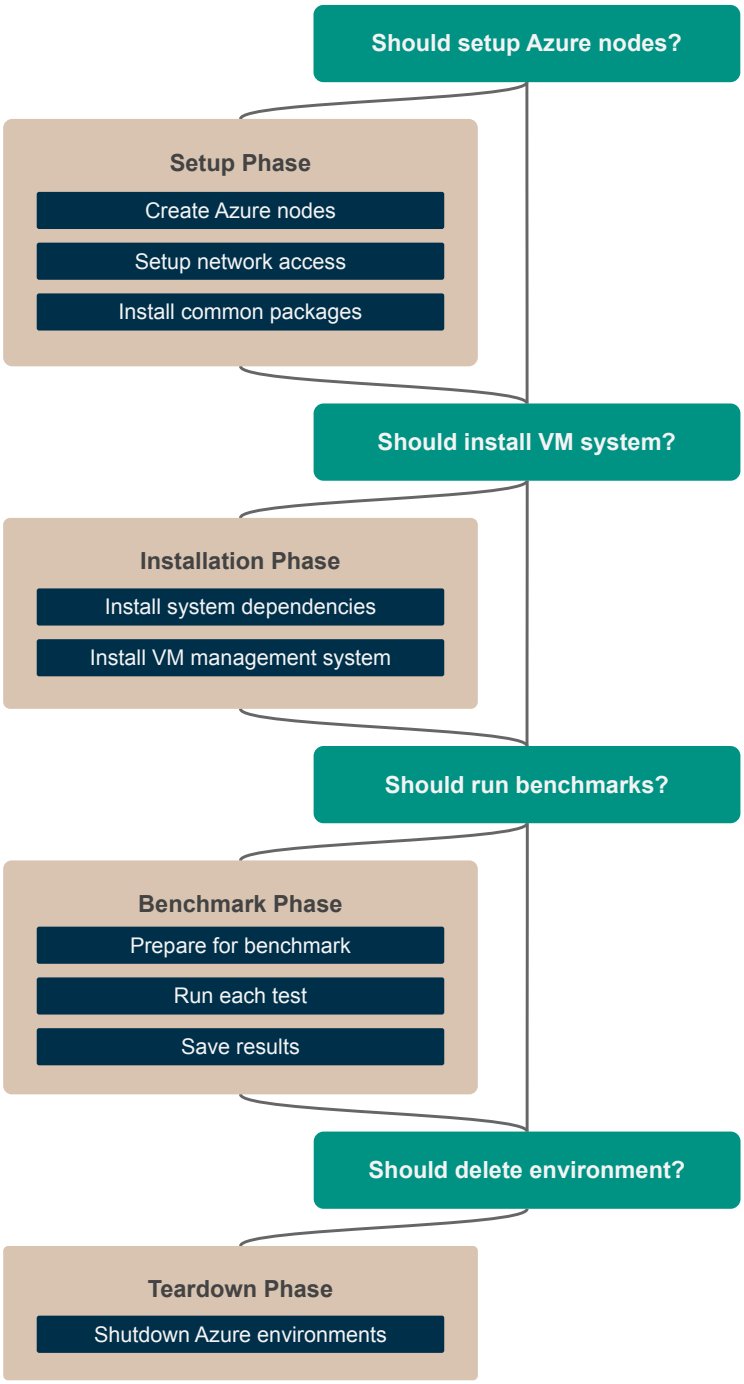

Figure 3.2: Overview of the workflow of the framework. The framework was divided into four phases that were responsible for different parts of the workflow. Each phase could be run independently of each other and disabled if not needed.

```
kubectl apply -f <<EOF
    apiVersion: kubevirt.io/v1
    kind: VirtualMachine
    metadata:
        name: $NAME
    spec:
        running: true
        template:
            spec:
            domain:
                devices:
                    rng: {}
                    disks:
                    - disk:
                        bus: virtio
                        name: containerdisk
                    - disk:
                        bus: virtio
                        name: emptydisk
                    resources:
                    requests:
                        cpu: 100m
                        memory: $MEMORYMi
                    limits:
                        cpu: $CPU_CORES
                        memory: $MEMORYMi
            volumes:
            - name: emptydisk
                emptyDisk:
                capacity: $DISK_SIZEGi
            - name: containerdisk
                containerDisk:
                image: $OS_IMAGE
EOF
```

OpenNebula

```
onetemplate instantiate $TEMPLATE_ID --name $NAME <<EOF
    CPU="0.1"
```

```
    VCPU="$CPU_CORES"
    MEMORY="$MEMORY"
    DISK=[
        SIZE="$DISK_SIZE"
        IMAGE="$OS_IMAGE"
    ]
EOF
```

The entire framework is open-source and available at `INSERTURLHERE`. The GitHub repository contains all the code required to run the framework, including documentation on how to set up and run the framework.

Chapter 4

Usability Analysis

To accurately measure the usability of KubeVirt, a set of tasks was crafted in cooperation with a system administrator at Net Insight. The tasks were designed to be representative of the tasks that a system administrator would perform when managing VMs. The tasks were designed to be performed using the Kubernetes CLI tool *kubectl* and KubeVirt's complementary CLI tool *virtctl*. This approach was chosen since KubeVirt did not directly offer a GUI, and using any third-party solution could risk introducing biases in the results.

4.1 Setup

To host an environment that was easily accessible to the participants, an AKS cluster was set up. Alternatives included installing a Kubernetes cluster locally using tools such as *Minicube* or *k3d*, but this approach was deemed less suitable since it could be time-consuming and require more resources from the participants. Moreover, a cloud-based solution ensured that the participants could access the environment anywhere reducing possible barriers to participation. AKS specifically was chosen since it included preinstalled storage classes that enabled plug-and-play persistent storage functionality. KubeVirt's snapshot-restore API required a CSI storage class, which made AKS a suitable choice. Similar to the performance analysis, the cluster included one control-plane node and two worker nodes to ensure that the participants could perform the tasks without any issues. Having two worker nodes was crucial for the live migration task, as it required moving a VM between two nodes. Separating the control node from the worker nodes was not necessary for the tests, but it is a common practice since it allows workers to be recreated without affecting the cluster's control components.

4.2 Task Design

The analysis was designed as a step-by-step guide that would take the participants through a set of tasks that were divided into three groups: basic usage, maintenance and debugging. The focus of the analysis was not to evaluate the participant's ability to perform the tasks or their Kubernetes knowledge but rather to expose them to KubeVirt and its workflows. As such, the tasks were designed to be as straightforward as possible, except for a debugging task that was designed to evaluate finding and resolving runtime issues. Moreover, the order of the tasks was designed to gradually increase in complexity to ensure a structured approach and allow the participants to familiarize themselves with KubeVirt. Below is a description of each group of tasks:

- **Basic usage**
  This task was designed to act as a starting point for the usability analysis. The participants were asked to create a single VM configured with a *containerDisk*, connect to it using SSH and VNC and then delete it. The purpose of this task was to evaluate how well KubeVirt supports the most basic operations and whether the participants deem the process intuitive.

- **Maintenance**
  The second task was designed to evaluate how well KubeVirt supports recurring tasks that a system administrator could face when managing VMs and the platform hosting them. The analysis divided this task into two parts: live migration and snapshotting. The participants were first asked to create a single VM and start it on a specific host, then migrate the VM to another host without any downtime. A lightweight VM configured with a *containerDisk* was chosen for this task to keep the focus on the live migration operation. The second part of the task was to create a snapshot of a VM and then restore it. A VM configured with a *DataVolume* disk was chosen for this task since a container disk cannot be snapshotted. Both live migration and snapshotting were included in the usability analysis since they are inherently different in KubeVirt compared to traditional VM management systems due to the declarative nature of Kubernetes, see 2.6.

- **Debugging**
  The third task was designed to evaluate how well KubeVirt supports debugging and how intuitive the process of resolving runtime issues is. While KubeVirt does enforce correct specification upon creation with

API validation, see 2.6, it is still possible that a VM does not start if the misconfiguration affects the runtime. For this reason, the participants were presented with a VM that was not accessible due to a cloud-init misconfiguration that prevents SSH access to the VM by having invalid password syntax and missing authorized SSH public keys. This means that the participants could solve the issue either by resolving the password formatting issue or supplying a valid SSH public key. The participants were presented with a basic problem statement and were encouraged to use the official documentation if needed. For this task, a lightweight VM configured with a containerDisk was used to keep the focus on the debugging operation.

After each group of tasks was finished, the participants were asked to reflect on the experience and provide feedback on the tasks. The participants were encouraged to use a free-form approach but could use a set of open-ended questions as a reference.

4.3 Conducting the Usability Analysis

The usability analysis was performed together with a group of system administrators at Net Insight. The participants were presented with a set of tasks as described in the previous section. Each participant was given a document containing all the tasks as well as an introduction to KubeVirt and each of the operations they would perform. For each task, the participant was encouraged to use the official KubeVirt documentation to aid them in completing the tasks. Together with the document containing the tasks, each participant was given a *kubeconfig* and the name for their Kubernetes namespace in which they would perform their tasks. They were given separate namespaces to ensure that none of the tasks would interfere with the results of the other participants. Once finished, the reflection and feedback were collected from each participant. They were given two weeks to complete the tasks, and while this was a relatively long time, it was chosen to ensure that the participants were given enough time to potentially explore features in KubeVirt that were not part of the given tasks. The entire usability analysis can be found in the appendix C.

Chapter 5

Results

This section contains the results of the performance analysis and usability analysis. Each test in the performance analysis was run 10 times, and the result visualizations were averaged using a rolling mean with a window size of 5% of the total number of data points. The usability analysis received feedback in different forms, either by answering questions or providing a free-form response.

5.1 Performance Analysis

This section contains the results of the performance analysis with KubeVirt and OpenNebula. Only the most significant results are presented in this section, and the complete list of results can be found in the appendix B.

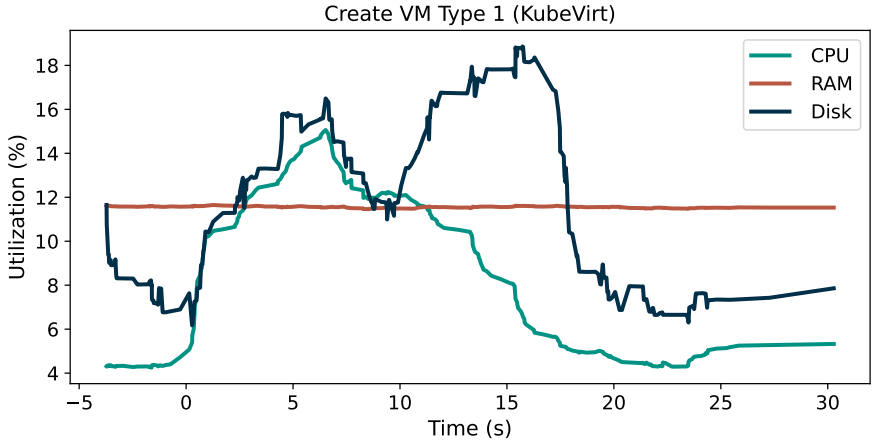

Figure 5.1: Resource utilization on the control node when creating a VM type 1 in KubeVirt. CPU and disk utilization follow a similar pattern nearly reaching 20% while memory utilization remains constant at 12% for the duration of the test.

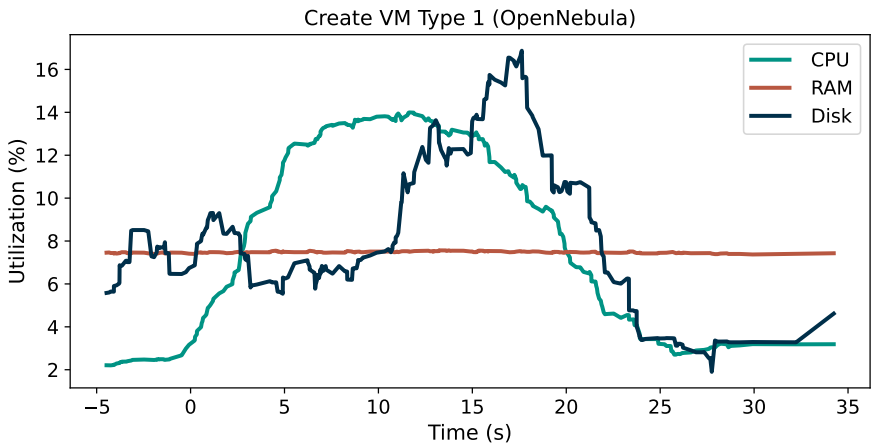

Figure 5.2: Resource utilization on the control node when creating a VM type 1 in OpenNebula. CPU and disk utilization both increase and peak after half the duration of the test. Memory utilization remains constant below 8% for the duration of the test.

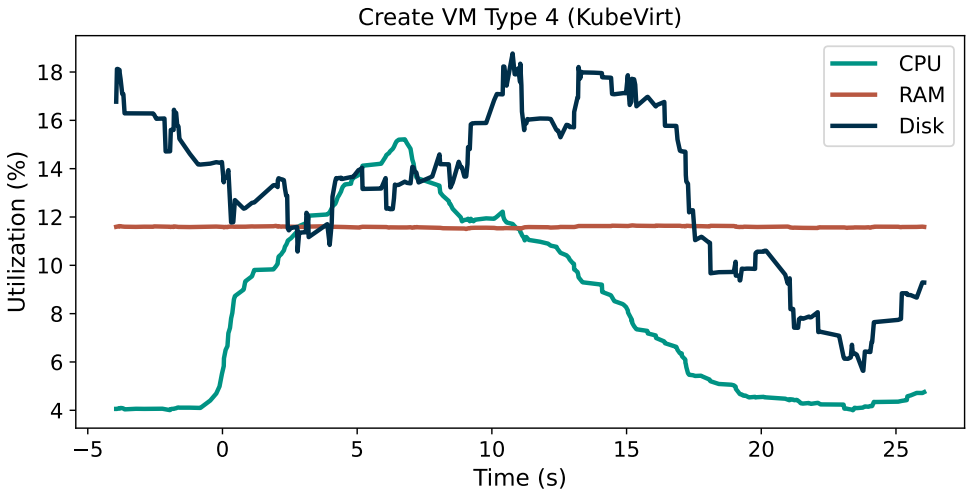

Figure 5.3: Resource utilization on the control node when creating a VM type 4 in KubeVirt. CPU utilization peaks quickly while disk utilization is sporadic. The sporadic behavior is not in line with the operations performed during the test. Memory utilization remains constant at 12% for the duration of the test.

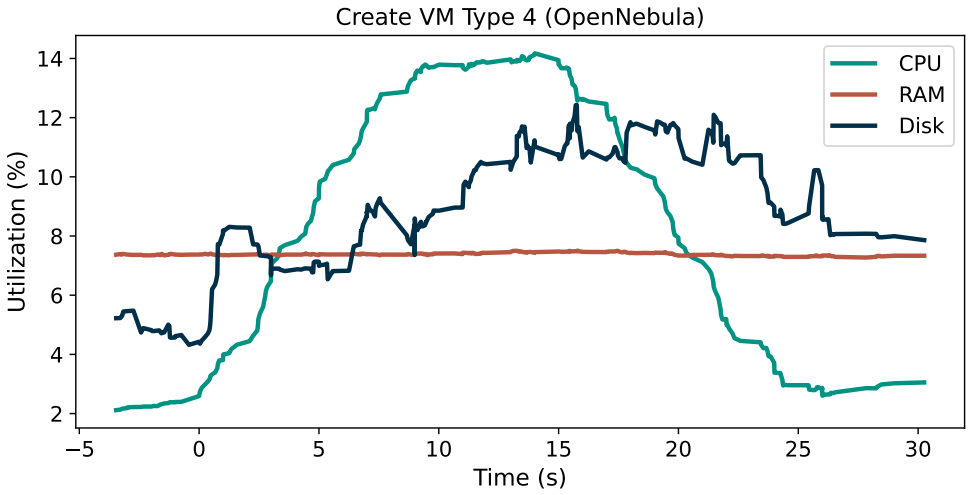

Figure 5.4: Resource utilization on the control node when creating a VM type 4 in OpenNebula. CPU utilization is the most prominent among the resources plateauing at nearly 15% utilization. Memory utilization remains constant at 8% for the duration of the test.

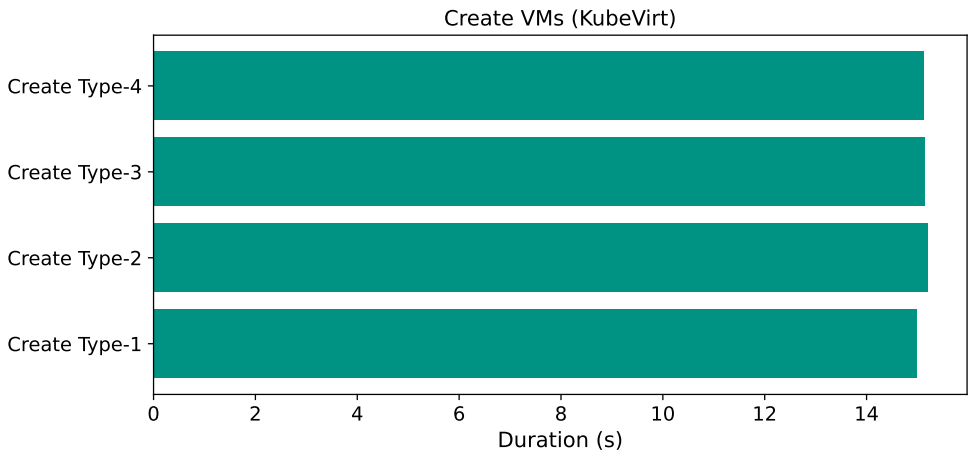

Figure 5.5: Duration of creating VMs in KubeVirt. The duration for all types remains constant for each VM type reaching slightly above 14 seconds

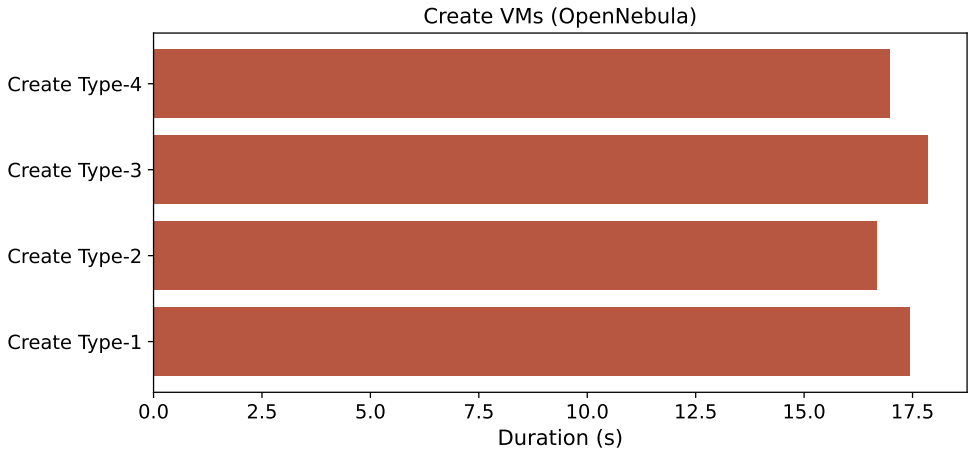

Figure 5.6: Duration of creating VMs in OpenNebula. The duration for all types remains constant for each VM type reaching nearly 18 seconds.

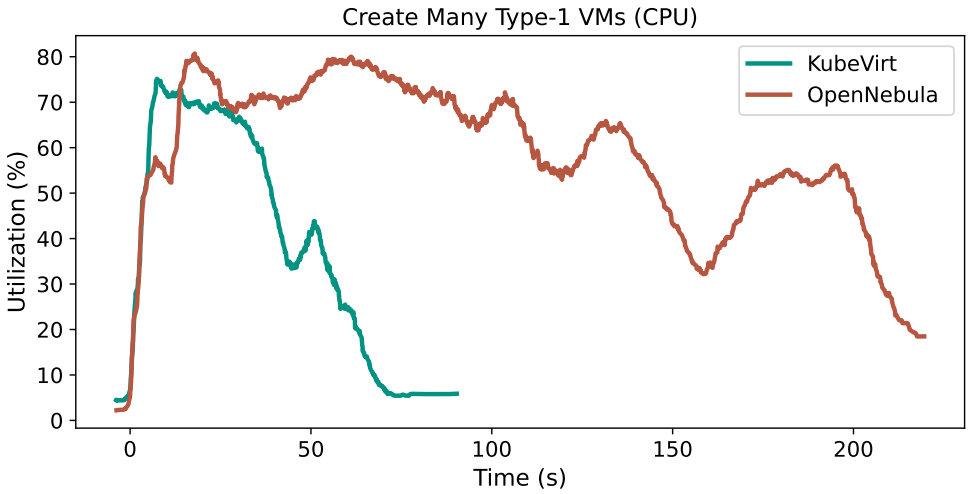

Figure 5.7: CPU utilization on the control node when creating 20 type-1 VMs. The CPU utilization peaks quickly for both KubeVirt and OpenNebula due to the burst of requests. KubeVirt reaches a slightly lower peak than OpenNebula and subsides significantly faster. OpenNebula does not manage to return to a utilization level of an idle state within the duration of the test, as evidenced by the utilization not reaching base utilization in the end.

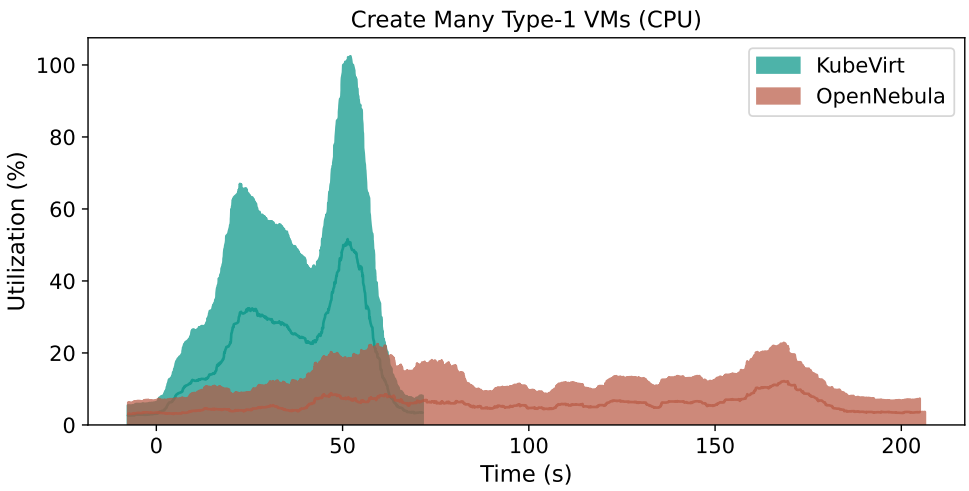

Figure 5.8: Cumulative CPU utilization among worker nodes when creating 20 type-1 VMs. The CPU utilization among the worker nodes is vastly different between KubeVirt and OpenNebula, where KubeVirt experiences a short-lived large burst of CPU utilization while OpenNebula barely reaches a cumulative 20% utilization.

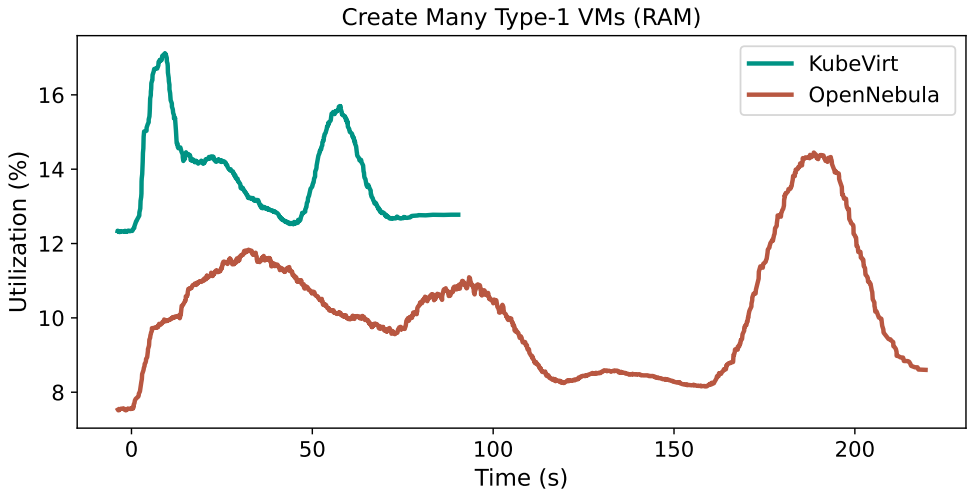

Figure 5.9: Memory utilization on the control node when creating 20 type-1 VMs. KubeVirt experiences a burst of memory utilization in the beginning, while OpenNeubla has a more gradual increase before spiking quickly toward the end of the test. While KubeVirt has a larger base memory utilization, OpenNebula experiences a larger induced load from the test, as evidenced by its longer slopes.

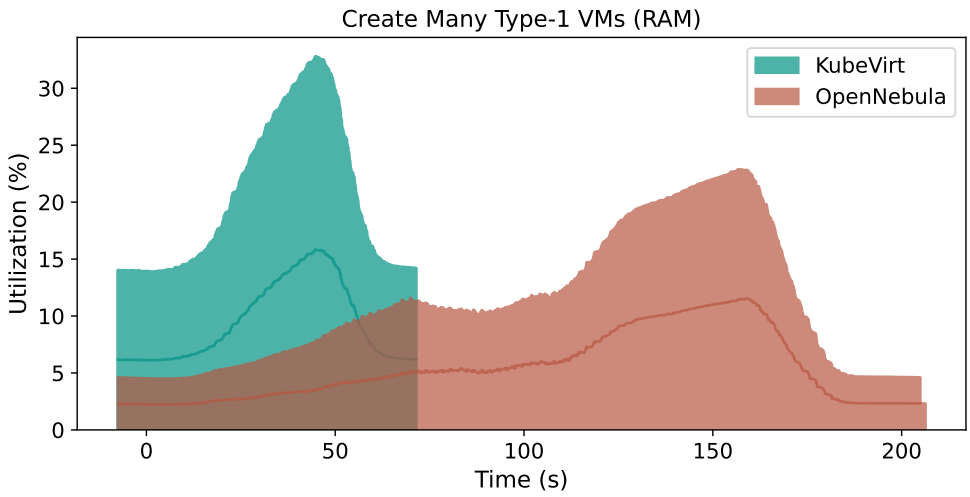

Figure 5.10: Cumulative memory utilization among worker nodes when creating 20 type-1 VMs. While KubeVirt experiences a larger memory utilization due to a larger base, both systems have a similar magnitude of the induced load, albeit KubeVirt's load is more short-lived.

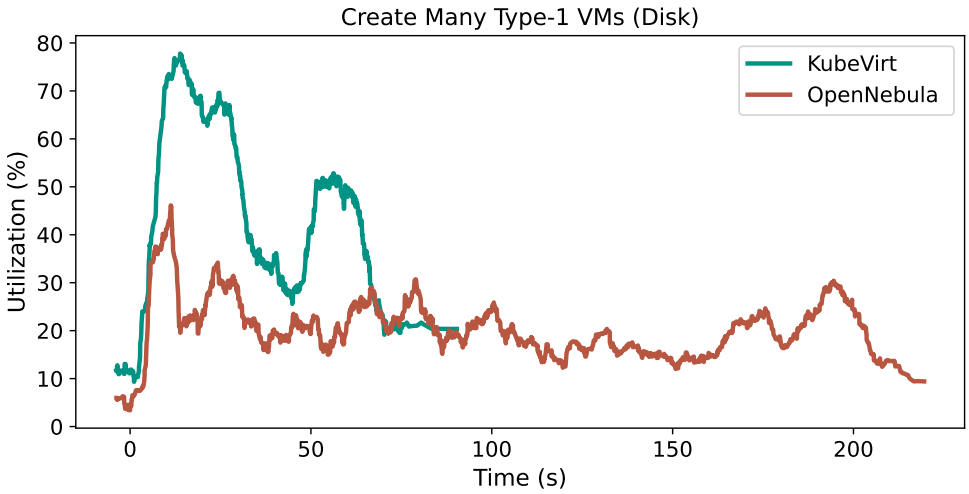

Figure 5.11: Disk utilization on the control node when creating 20 type-1 VMs. KubeVirt experiences a larger burst of disk utilization than OpenNebula, while OpenNebula maintains a steady disk utilization throughout the test.

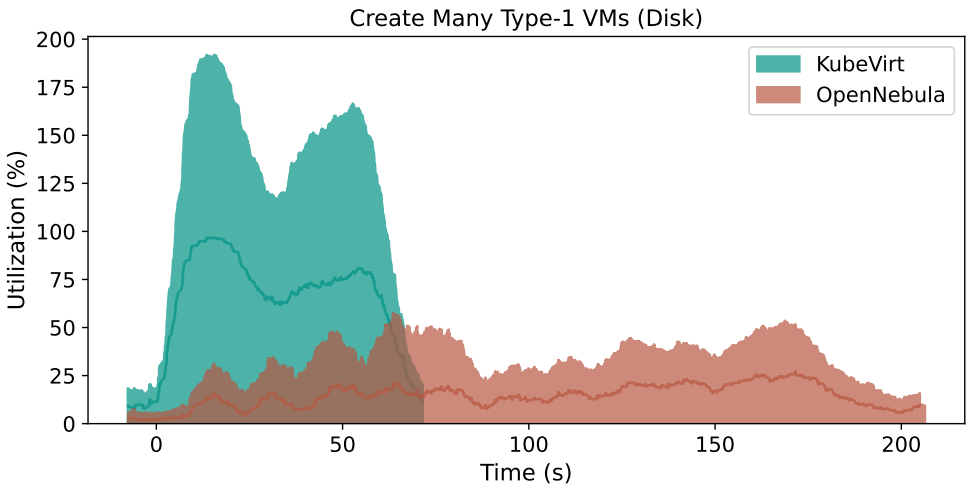

Figure 5.12: Cumulative disk utilization among worker nodes when creating 20 type-1 VMs. KubeVirt experiences a larger burst of disk utilization than OpenNebula, while OpenNebula maintains a low and steady disk utilization throughout the test.

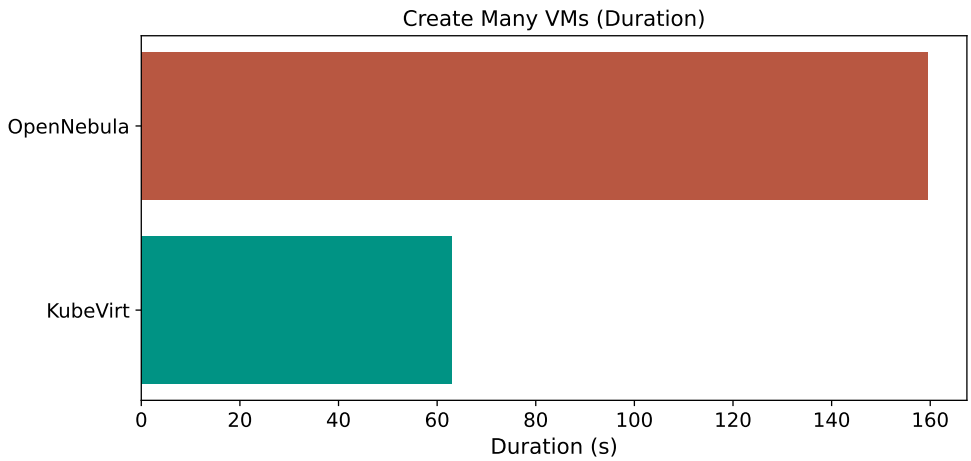

Figure 5.13: Duration of creating 20 type-1 VMs. KubeVirt is significantly faster to create 20 type-1 VMs than OpenNebula, taking 67 seconds compared to OpenNebula's 192 seconds.

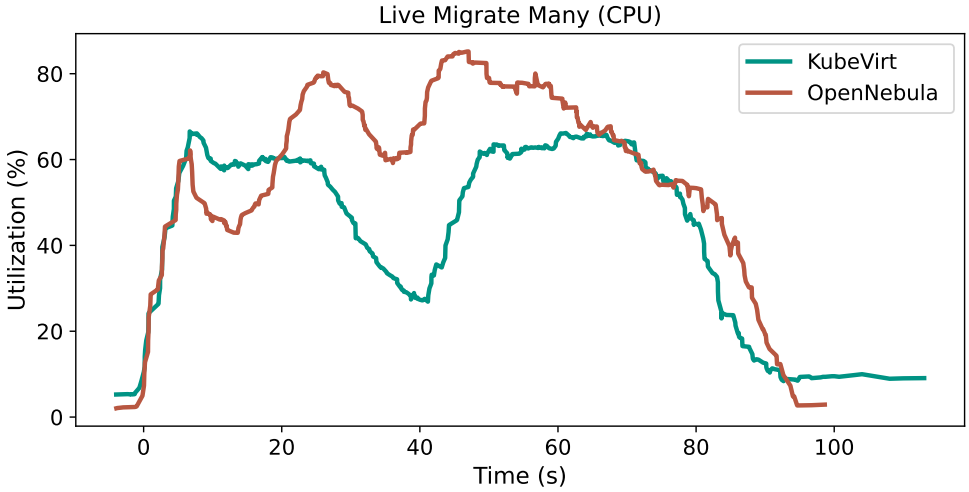

Figure 5.14: CPU utilization on the control node when live migrating 10 type-1 VMs. Both systems follow a similar pattern where KubeVirt has a slightly larger base utilization than OpenNebula, but the induced load is larger for OpenNebula.

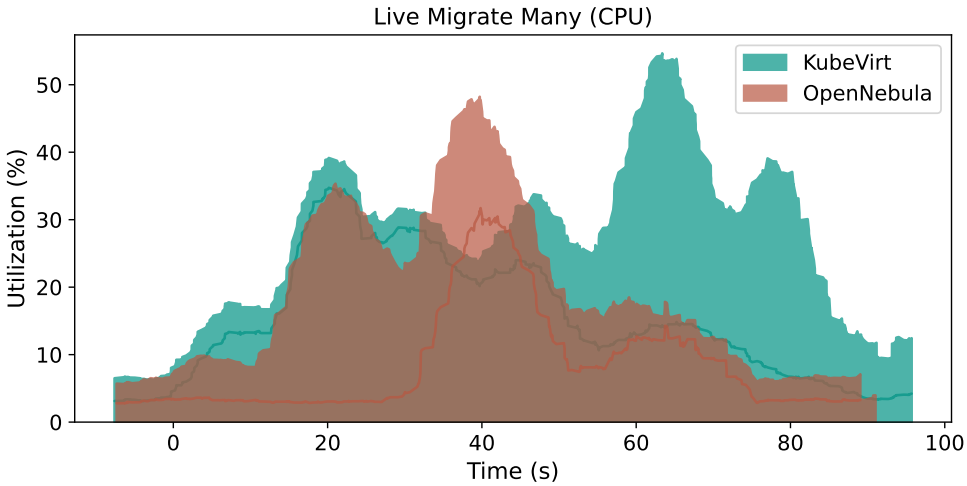

Figure 5.15: Cumulative CPU utilization among worker nodes when live migrating 10 type-1 VMs. Both systems experience a similar load for the first half of the test, but KubeVirt experiences a significantly larger load in the second half of the test.

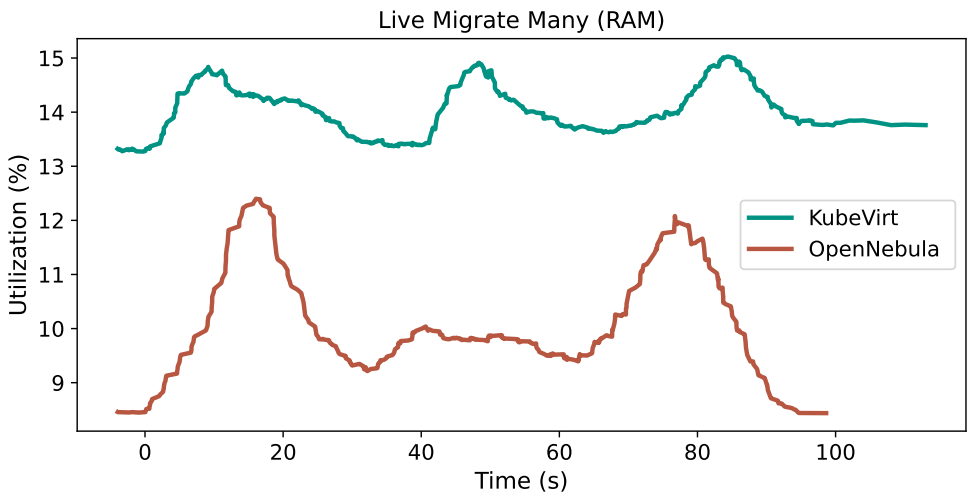

Figure 5.16: Memory utilization on the control node when live migrating 10 type-1 VMs. KubeVirt has a larger base memory utilization than OpenNebula, but the induced load is more than double for OpenNebula.

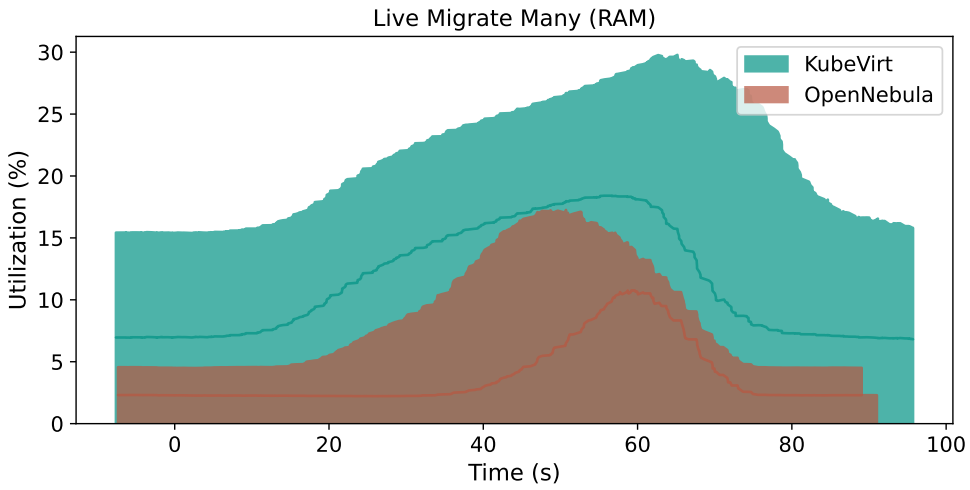

Figure 5.17: Cumulative memory utilization among worker nodes when live migrating 10 type-1 VMs. KubeVirt has a larger base memory utilization than OpenNebula, but both systems experience a similar induced load. OpenNebula returns to an idle state faster than KubeVirt.

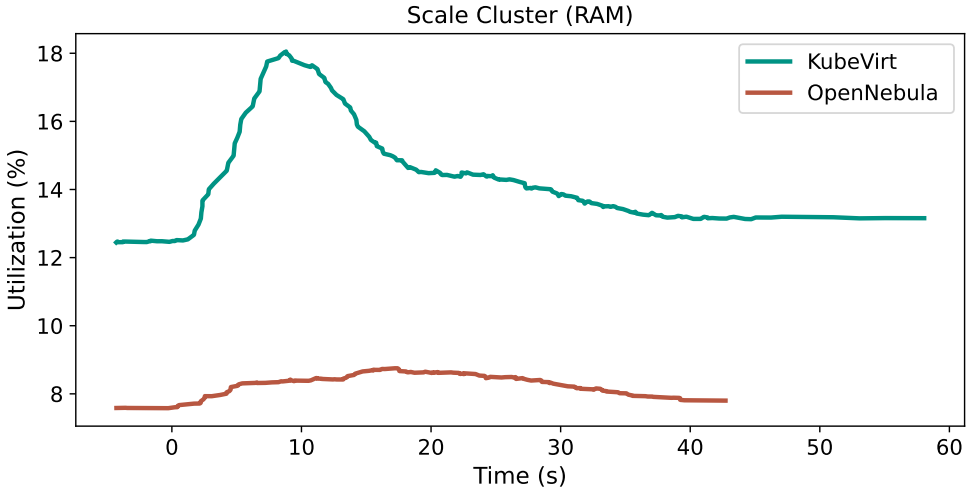

Figure 5.18: Memory utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a significantly larger memory utilization than OpenNebula. OpenNebula's memory utilization is barely affected by the scaling operation.

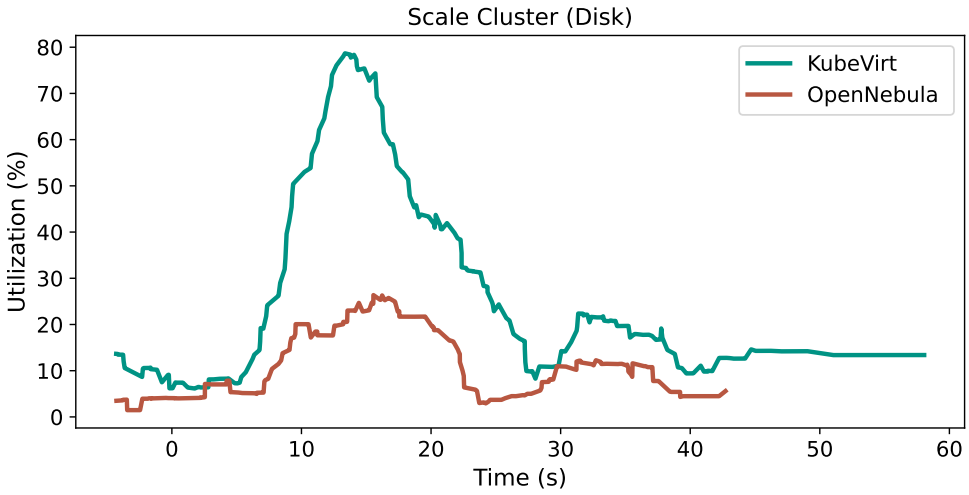

Figure 5.19: Disk utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a rapid increase in disk utilization, reaching a peak of over 70% utilization. OpenNebula has a less aggressive increase reaching barely 30% utilization.

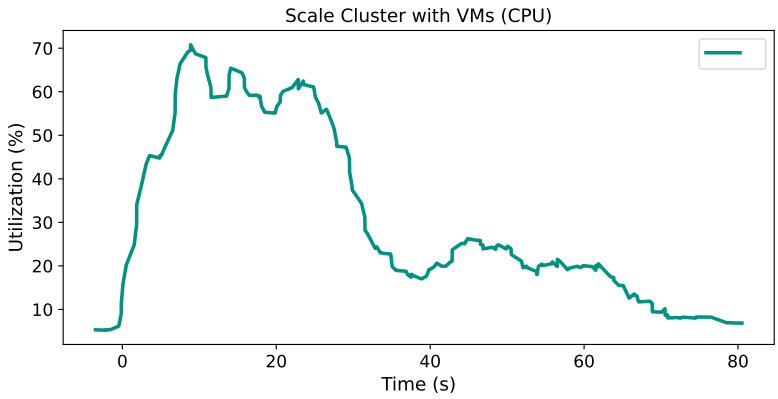

Figure 5.20: CPU utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences an initial burst of CPU utilization for the scale-up operation, but a significantly lower utilization for the scale-down operation. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

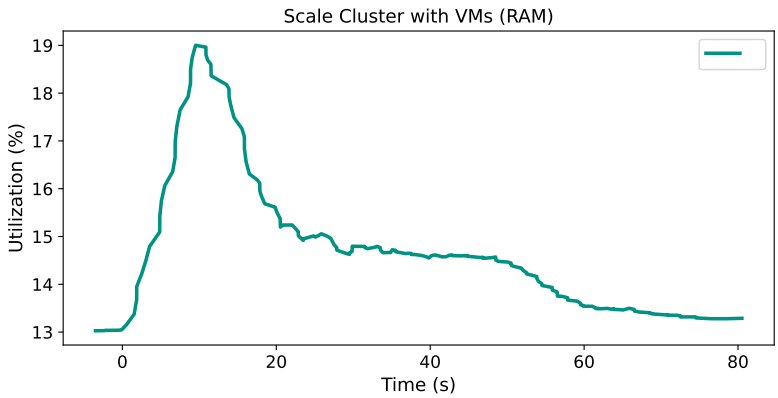

Figure 5.21: Memory utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences a large burst of memory utilization for the scale-up operation, but a significantly lower utilization for the scale-down operation that gradually decreases towards the end. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

5.2 Usability Analysis

The collected answers from the participants are summarized below. Some participants chose a free-hand approach when answering the questions, while other participants chose to answer the questions directly in a structured manner. The full answers from every participant can be found in the appendix D.

Basic Usage

The participants found the process of creating a VM in KubeVirt to be mostly straightforward.

- Participant 1: *The act of doing kubectl apply -f and kubectl delete is quite neat.*
- Participant 2: *Very easy! I like that it is as simple as creating a YAML file and then applying it.*

However, participants expressed some confusion when connecting to the VM using SSH, as it was not clear which SSH public key was used to connect to the VM, and that conflicts with previously created VMs could occur. Moreover, participant 1, in particular, found the VM manifest definition complex, and that the process of creating it was not intuitive.

- Participant 1: *It would be quite difficult for me to figure out I need to use bus: virtio without having someone else tell me I need to.*

KubeVirt's complementary CLI tool *virtctl* was found to be useful when connecting to the VM, as it provided a single tool for both VNC and SSH.

- Participant 2: *Very simple! Simply doing virtual ssh cirros@my-vm and then typing in the password and then I was in!*

Maintenance

The maintenance tasks received a broad range of feedback. Some of the participants found the process of live migration to be simple, while other participants deemed the process to be overly complex and non-intuitive. Participant 1 in particular regarded many of the operations as flawed, such as the live migration object not being deleted after the migration was completed, but also that

KubeVirt lacked proper documentation around the practical aspects of how to perform a live migration. Similar to the basic usage tasks, the participants experienced issues with connecting to VMs using SSH, as it was not clear which SSH public key was used to connect to the VM.

- Participant 2: *I had to delete the entry from /.ssh/kubevirt_known_hosts each time I restarted the VM because virtual ssh complained about a key mismatch.*

Debugging

Compared to both the basic tasks and maintenance, the debugging tasks received the most negative feedback. The participants found the process of debugging to be complex and unclear. Finding the correct documentation was another issue some participants faced, as both the official documentation and the *virtctl –help* command did not provide the necessary information to solve the issue.

- Participant 1: *I did not manage to solve the key mismatch issue as there is very little in the way of debugging virtctl ssh.*

However, while debugging the issue presented in the task was found to be complex, participant 3 regarded the YAML files in general as a positive aspect of debugging.

- Participant 3: *Debugging the issue was easy because the manifest uses standard constructs that are easily googleable.*

Overall

The feedback to KubeVirt was in general mixed. While the workflow was found to be intuitive due to its similarities with ordinary Kubernetes resources. This was particularly highlighted by Partcipant 4 who already mangaged other Kubernetes manifests in Git.

- Participant 4: *It is primarily because I'm already storing all my Kubernetes manifest in Git and have the CI setup for that, so why not also create virtual machines in a similar way.*

Being similar to Kubernetes resources was also found to be a negative aspect as it leads to a steep learning curve. The documentation was found to be lacking practical examples of how to get started outside the scope of the given tasks and instead focused more on the operational aspects of KubeVirt.

Chapter 6

Discussion

The research questions involved investigating potential efficiencies that could be gained by integrating VMs in Kubernetes using KubeVirt. This project has evaluated KubeVirt using two different approaches: a performance analysis and a usability analysis. The discussion aims to explore deeper into the results of the project to provide insight into how feasible KubeVirt is as a VM management system.

6.1 Limited Features and Result Intepretation

The goal of the performance analysis was to find factual and provable claims regarding the performance of a VM management system. This included creating a framework that could be used to craft environments that were both reusable and fair among the systems being evaluated. The framework also provided a common interface for which an implementation was created per VM management system. By definition, such a system is only as good as the implementations are, which is why the implementations were designed to be as similar as possible to ensure a fair comparison. However, certain features and settings were not able to be tuned in the same way, such as the scheduling configuration where OpenNebula provides only a single option for scheduling: the number of VMs that can be dispatched concurrently. Not being able to accurately configure the systems identically in every aspect leaves the interpretation of the results open for discussion. This project has chosen the approach where if the system does not provide a specific feature to be tuned, the system is considered to be as optimized as possible. This approach was chosen as it favors the VM management system that provides the most features and flexibility, and as such, the results should be interpreted with this in

mind. Having this aspect in mind is particularly important when interpreting the *Create Many Type-1 VMs* tests, see figures 5.7, 5.9, 5.11 and 5.13. KubeVirt, using the scheduler built into Kubernetes, was able to create 20 type-1 VMs in 67 seconds, while it took OpenNebula 192 seconds on average. However, the results also suggest that KubeVirt is more resource-intensive, as the utilization of CPU, memory and disk was significantly higher than OpenNebula, which can viewed as a trade-off between the duration to complete a task and the resources required to complete it. This project has chosen the duration to be the more important metric, as it indicates that the system is capable of handling a burst of requests efficiently and using its allocated resources to its fullest potential.

It is not always the case that KubeVirt's larger resource utilization is short-lived compared to OpenNebula, where some results suggest that KubeVirt has a significantly larger disk utilization than OpenNeubla for a similar period. This is particularly evident in figure 5.19 where KubeVirt and OpenNebula follow a similar characteristic when scaling the cluster but KubeVirt disk utilization is significantly more intense. For this specific test, the result suggests that KubeVirt does not provide a trade-off between the duration to complete the task and the resources required to complete it. This could be due to the way KubeVirt implements adding additional nodes to the cluster, which could be a resource-intensive operation. Another explanation could be that the specific Kubernetes distribution used in the tests is not optimized for scaling the cluster. However, exploring solutions for this is outside the scope of this project.

The final test used in this project included scaling the cluster with VMs running on each node. It involved scaling the cluster from 2 worker nodes up to 10 worker nodes, then creating a type-1 VM on each worker node before scaling it down to 2 worker nodes. This caused the VM management system to migrate the VMs to available nodes. This test posed a challenge for the scheduler of both systems since a VM might be scheduled to the wrong host if the scale-down operation was not handled immediately. The test was also an extension to the *Scale Cluster* test that involved the same steps but without the VMs. The result shows that OpenNebula was not able to complete the test due to some VMs not being migrated to the correct host, and instead remained on the host that was requested to be removed. Since the test was designed to disconnect the worker and then wait for each host to be empty, the OpenNebula implementation hung indefinitely. A possible explanation for this could be that OpenNebula attempted to move a VM from a host that disconnected to another host that was just about to disconnect, but hadn't yet, and could cause the receiving host to end up in a bad state. Another explanation could be that

OpenNebula requires additional control plane nodes to handle such bursts for migration operation since the test environment only provided a single control plane node to reduce the cost of the test since the nodes were hosted in Azure. However, this is purely speculative and exploring the cause of the issue is outside the scope of this project. The issue of VMs stuck on hosts could be resolved by manual intervention, but since this was deemed unfair to the other system, OpenNebula was disqualified from the scale-with-VMs test. As such, the result for scaling with VMs was omitted for OpenNebula and only KubeVirt was evaluated for this test.

6.2 Kubernetes Overhead and Induced Load

The results of the performance analysis indicate that both KubeVirt and OpenNebula have a consistent overhead and that it differs between the systems. Having a larger overhead implies the system has more ongoing computational work in the background while not performing a requested operation. This is particularly evident in the tests creating different VM-types, see figures 5.1, 5.2, 5.3 and 5.4. KubeVirt CPU utilization hovers around 4% with a memory utilization of just below 12%. OpenNebula, on the other hand, has a CPU utilization just above 2% and a RAM utilization of 8%. This suggests that OpenNebula has a lower required overhead to maintain the system compared to KubeVirt. The overhead is also evident in the tests including live migration, see 5.16. OpenNebula starts the test utilizing just above 8% of the available memory while KubeVirt uses over 13%. However, while this would suggest that KubeVirt is less resource-efficient than OpenNebula, it is important to first consider the surrounding factors. Firstly, since Kubernetes was originally designed to manage containers, it has a different set of requirements compared to a traditional VM management system such as OpenNebula. Certain operations enabling container orchestration might not be necessary for a VM management system, which could manifest as an overhead. Secondly, the overhead shown in the results is not necessarily connected to KubeVirts implementation but rather Kubernetes itself, which could in turn be tied to the specific Kubernetes distribution used in the tests. However, exploring possible optimizations to reduce such overhead or the source of the overhead is outside the scope of this project. Lastly, as described in 2.6.1, KubeVirt achieves VM hosting in Kubernetes by using custom resource and controllers that wraps a VM instance inside a Pod. While this approach allows for seamless integration of VMs in Kubernetes, it also introduces an unavoidable overhead. In conclusion, the overhead is a trade-off for the flexibility and features that KubeVirt provides, and the results

suggest that KubeVirt is more resource-intensive than OpenNebula when not performing operations.

The flip side of this is that many of the results suggest the opposite regarding induced load. Induced load is the added load on the system when performing a specific operation and is calculated as the difference between the total utilization and the base utilization. This can be difficult to interpret since a large induced load over a short period is not necessarily worse than a small induced load over a longer period. However, this project has chosen a short-lived burst of utilization to be favorable over a longer period of low to medium utilization since it suggests that the system is capable of using its allocated resources to its fullest extent. The results suggest that KubeVirt has a lower induced load in general than OpenNebula, where in many cases the induced load is more than double for OpenNebula compared to KubeVirt. This is particularly evident in figure 5.7 where OpenNebula both take twice as long to complete the test and have twice as large induced load from the operation, around 6% compared to KubeVirt's 3%. The difference in induced load is further illustrated in figure 5.14. Not only does it show that OpenNebula reaches an induced load of 76% while KubeVirt reaches just above 50%, but also that induced load is more prominent in the total utilization for both systems. This suggests that induced load is a more significant factor when evaluating the performance of VM management systems and, as such, should be considered when interpreting the results. Figure 6.1 further illustrates this point where for OpenNebula, the induced load is a significant part of the total utilization. Having a smaller induced load suggests that the system is better at scale while having a smaller base overhead suggests that the system is more efficient when the system is idle or performing few operations. However, whether this is true for all operations a VM management system offers remains to be investigated.

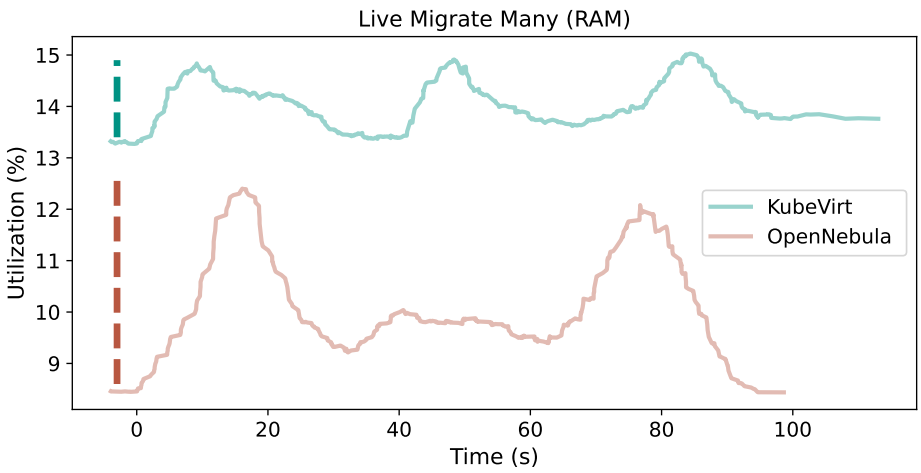

Figure 6.1: Induced memory utilization on the control node when live migrating 10 type-1 VMs. The induced load is a more significant part of the total utilization for OpenNebula than for KubeVirt. The figure is based on the results from figure 5.16.

6.3 Jagged Disk Utilization

Throughout the performance analysis results, there is a recurring and notable pattern specifically tied to disk utilization. The disk utilization graphs are jagged and, in many cases, do not follow the expected pattern of the specific operation performed in the test. This is particularly evident in the *Create VM Type 4* tests, see figures 5.3 and 5.4. The disk utilization for both Kubevirt and OpenNebula is initially larger than expected and does not return to the same state after the test is finished. Since the tests were run multiple times in no particular order, this likely means that noise was introduced in the measurement that caused each test to generate data points in different ranges. When the induced disk utilization was low for a particular test, background disk utilization from sources such as OS-level operations, was more prominent in the results. By then applying a rolling mean smoothing function to the dataset, the noise was reduced, but due to the variability, some jagged features remained. This is further supported by the fact that the disk utilization is less jagged for tests with a larger expected disk utilization, such as the *Create Many Type-1 VMs* tests since these tests generate a larger induced load. While the jagged disk utilization is not a problem in itself, it is important to consider when interpreting the results, especially those where disk utilization is small, as it could introduce bias unrelated to the operation performed.

6.4 Mixed Opinions Using KubeVirt

The results of the usability analysis broadly indicate that the participants experienced KubeVirt differently. Some participants found KubeVirt intuitive and acknowledged many of the benefits discussed throughout this report, while other participants made strong claims against the technology as a whole. To understand this discrepancy, it is important to consider what the positive or negative feedback targeted specifically to understand if the opinions were contradictory or if they were based on different aspects of the technology. The positive feedback was mostly tied to the idea of using a well-known CLI tool, *kubectl*, and using declarative configuration files to define the VMs. This formed a foundation where the participants were comfortable and appeared to find similarities with their existing knowledge of Kubernetes and, as such, basic tasks received well. Mixed opinions appeared once the tasks became more complex. For instance, the live migration task was found, contradictorily, to be both intuitive and confusing. This could be due to the different workflow KubeVirt presents compared to OpenNebula, where KubeVirt uses a declarative approach to define the live migration operation, while OpenNebula uses an imperative action-based approach. This is supported by the feedback from participants 1 and 2, where Participant 2 found the process of creating a resource for the live migration to be straightforward, while Participant 1 expressed confusion and argued that the process was unclear regarding clean-up and when the operation was completed. Structuring a live migration operation as a resource, where the action is performed once the resource is created is a common pattern in Kubernetes, but the results of the usability analysis suggest this approach could be confusing.

Other negative feedback was more concerned with exploring KubeVirt outside the scope of the given tasks. This is a particularly prevalent issue for one participant who deemed KubeVirt's official documentation to be lacking and missing practical examples of how to get started. Since exploring documentation was not part of the usability analysis, and the participants were not given any guidance on how to find the necessary information, it is difficult to determine if the participants were unable to find the information due to the lack of documentation or guidance. Either way, having issues with finding correct documentation is a significant problem for a technology that involves many user interactions and operations.

The mixed responses in the usability analysis indicate both possible biases among the participants regarding previous knowledge and experience with Kubernetes, but also that KubeVirt could be a controversial approach to managing

resources. Kubernetes is known for its steep learning curve due to its declarative approach to resource management and verbose configuration files, and since KubeVirt follows the same idea, it is not surprising that the participants found the technology complex. Biases could thus have been caused by previous VM management systems being more approachable than KubeVirt, which could have affected the participant's opinions on KubeVirt. Exploring whether previous knowledge of a VM management system affects the perceived usability of KubeVirt is a possible future research direction but it is outside the scope of this project.

6.5 Concluding Research Questions

The results from the performance analysis indicate both strengths and weaknesses for KubeVirt. As discussed in previous sections, KubeVirt inhibits a larger base utilization without performing any operation requested by a user compared to OpenNebula. However, the induced load is in general significantly lower for KubeVirt than for OpenNebula. This suggests that KubeVirt is more efficient when performing operations and thus could be more scalable than OpenNebula. Furthermore, the usability analysis indicates that KubeVirt in many cases benefits from the declarative approach inherited by Kubernetes. This section aims to answer the research questions presented in the introduction.

What performance, scalability, and operational efficiencies can be gained by integrating VMs in Kubernetes?

KubeVirt provides an efficient way to manage VMs in Kubernetes with a lower induced load than OpenNebula. KubeVirt appears to be more scalable than OpenNebula, as evidenced by the lower induced load and the shorter duration to complete tasks. However, KubeVirt has a larger base utilization than OpenNebula, which argues for the consideration of each specific use case, such as whether KubeVirt is a viable option when each hosting node is relatively small in terms of resources. Operational efficiencies were acknowledged by some participants, but argued against by others. This suggests that KubeVirt could provide operational efficiencies, but that it requires a strong familiarity with Kubernetes and its concepts to be fully utilized. It also suggests that KubeVirt might lack in certain aspects regarding usability and that it requires further development. In summary, KubeVirt provides performance and scalability efficiencies when managing many VMs, but might be a more resource-intensive

solution when idle or performing few operations. Operational efficiencies are gained when the system administrators are familiar with KubeVirt, but it could lead to confusion regarding the workflow and the technology itself.

What are the current challenges and limitations in managing VMs and containerized applications in separate environments?

The literature study suggests that VMs and containers share many aspects including what they provide for a user, such as an environment that allows for computational workloads, and are thus often used in the same context. However, the literature study also described how both VMs and containers have their own use cases due to inherent properties, including the level of isolation they provide. Having VMs and containers in separate environments not only implies that the system administrators are required to have a broader skillset to manage both environments but also that benefits from one environment cannot be utilized in the other. This is particularly evident in the performance analysis where KubeVirt appears to benefit from the mature and efficient resource scheduler built into Kubernetes. Furthermore, Kubernetes provides a vast ecosystem of tools and resources that could be utilized by KubeVirt, including monitoring, logging and networking, as well as third-party integrations for storage, security and observability. In summary, this project not only presents the benefits of integrating VMs and containers into a unified environment but also the benefits of integrating VMs into Kubernetes.

How can system administration be simplified by running VMs in Kubernetes using KubeVirt?

The usability analysis highlights several aspects of how a system administrator can harness the benefits of moving into a more declarative approach to managing resources. This is particularly evident by one participant who found that the configuration files were easy to debug and "googleable" due to how contained the information was. This suggests that error handling and debugging could be simplified by being less time-consuming. Furthermore, the literature study presents other benefits of managing VMs in KubeVirt. Since the resources are managed declaratively it opens up the possibility of managing the resources using GitOps, which could provide a more secure and reliable way of managing resources since it removes the burden of version control and auditing from the system administrator. However, it is important to note that the results of usability analysis suggest that KubeVirt comes with a steep learning curve and that the technology could be perceived as complex. As such, the

benefits found in the usability analysis and literature study might not be fully utilized if the system administrators are not familiar with Kubernetes and its concepts.

Chapter 7

Conclusions and Future Work

This project has successfully evaluated Kubernetes as a VM management system through a literature study, by benchmarking the performance of KubeVirt against a traditional VM management system and by performing a usability analysis with system administrators. The literature study presented the benefits of adopting a Kubernetes approach to managing resources, including how the ecosystem of tools and services could utilized for a more efficient environment. The findings suggest that moving into Kubernetes enables integration with existing cloud-native tools such as GitOps. The results of the performance analysis suggest that the performance of both VM management systems are similar, but that KubeVirt is more efficient in large-scale environments due to its lower induced load. However, the overhead is not negligible and could be a limiting factor for certain use cases, especially those where the hosting nodes are relatively weak in terms of resources. Limitations among the VM management system, including available configuration and tuning options and the performance analysis framework itself were discussed, and the results were interpreted with these in mind. The usability analysis suggests that KubeVirt can provide operational efficiencies such as simplified debugging and resource management. However, it also found that the technology could be perceived as complex and require familiarity with Kubernetes, its concepts and design philosophy. Potential biases among the participants as well as flaws in the technology itself were discussed, including whether the previous VM management system used affects the usability of KubeVirt.

Further work could involve exploring the source of the overhead discrepancy found in the performance analysis, including why OpenNebula inhibits a significantly lower baseline overhead than KubeVirt. It could also involve exploring if KubeVirt's apparent lower induced load is a result of the Kuber-

netes scheduler, the Kubernetes distribution or KubeVirt's implementation of VM management. Additionally, further work could explore the source of the potential biases found in the usability analysis and answer if the adoption of KubeVirt differs depending on the previous VM management systems used, as this project has only concluded that previous Kubernetes experience affects the usability of KubeVirt positively. Lastly, further work could investigate KubeVirt's applicability in environments with strict security requirements and whether Kubernetes offers the necessary tools and resources to meet these requirements, as this could further support the claims made in this project regarding the benefits of Kubernetes integration.

In summary, the project has shown that while OpenNebula outperforms KubeVirt in some scenarios due to its simplicity and low overhead, KubeVirt appears more scalable in larger-scale deployments. KubeVirt also provides operational efficiencies given that the system administrators are familiar with both the core concepts of Kubernetes and KubeVirt itself.

Bibliography

[1] Awodele Oludele et al. "On the evolution of virtualization and cloud computing: A review". In: *Journal of Computer Sciences and Applications* 2.3 (2014), pp. 40–43.

[2] Susanta Nanda Tzi-cker Chiueh and Stony Brook. "A survey on virtualization technologies". In: *Rpe Report* 142 (2005).

[3] M. Rosenblum and T. Garfinkel. "Virtual machine monitors: current technology and future trends". In: *Computer* 38.5 (2005), pp. 39–47. DOI: 10.1109/MC.2005.176.

[4] IBM. *Virtualization*. Feb. 2024. URL: https://www.ibm.com/topics/virtualization.

[5] Ankita Desai et al. "Hypervisor: A survey on concepts and taxonomy". In: *International Journal of Innovative Technology and Exploring Engineering* 2.3 (2013), pp. 222–225.

[6] Vitor Goncalves da Silva, Marite Kirikova, and Gundars Alksnis. "Containers for Virtualization: An Overview." In: *Appl. Comput. Syst.* 23.1 (2018), pp. 21–27.

[7] Dennis Gannon, Roger Barga, and Neel Sundaresan. "Cloud-native applications". In: *IEEE Cloud Computing* 4.5 (2017), pp. 16–21.

[8] Ramzi Debab and Walid Khaled Hidouci. "Containers runtimes war: a comparative study". In: *Proceedings of the Future Technologies Conference (FTC) 2020, Volume 2*. Springer. 2021, pp. 135–161.

[9] Xingyu Wang, Junzhao Du, and Hui Liu. "Performance and isolation analysis of RunC, gVisor and Kata Containers runtimes". In: *Cluster Computing* 25.2 (2022), pp. 1497–1513.

[10] OpenStack Foundation. *OpenStack*. Accessed 2024-03-06. Mar. 2024. URL: https://www.openstack.org/.

[11] Apache CloudStack. *Apache CloudStack*. Accessed 2024-03-06. Mar. 2024. URL: https://cloudstack.apache.org/.

[12] OpenNebula. *OpenNebula*. Accessed 2024-03-06. Mar. 2024. URL: https://opennebula.io/.

[13] Kubernetes. *Kubernetes*. Accessed 2024-03-06. Mar. 2024. URL: https://kubernetes.io/.

[14] HashiCorp. *Nomad*. Accessed 2024-03-06. Mar. 2024. URL: https://www.nomadproject.io/.

[15] Docker. *Docker Swarm*. Accessed 2024-03-06. Mar. 2024. URL: https://docs.docker.com/engine/swarm/.

[16] Subrota Kumar Mondal et al. "Kubernetes in IT administration and serverless computing: An empirical study and research challenges". In: *The Journal of Supercomputing* (2022), pp. 1–51.

[17] Guoqing Li et al. "The Convergence of Container and Traditional Virtualization: Strengths and Limitations". In: *SN Computer Science* 4.4 (May 2023). https://doi.org/10.1007/s42979-023-01827-9, p. 387. ISSN: 2661-8907. DOI: 10.1007/s42979-023-01827-9.

[18] The KubeVirt.io website CNCF. *Kubevirt.io*. Jan. 2024. URL: https://kubevirt.io/.

[19] Zou Nengren September 22, Ambassador post by Zou Nengren, and September 22. *Best practices for transitioning VM clusters to KubeVirt 1.0*. Sept. 2023. URL: https://www.cncf.io/blog/2023/09/22/best-practices-for-transitioning-vm-clusters-to-kubevirt-1-0/.

[20] Marcin Kubacki. *what is Kubevirt*. Feb. 2024. URL: https://storware.eu/blog/what-is-kubevirt/.

[21] Fred Douglis and Orran Krieger. "Virtualization". In: *IEEE Internet Computing* 17.2 (2013), pp. 6–9. DOI: 10.1109/MIC.2013.42.

[22] Hasan Fayyad-Kazan, Luc Perneel, and Martin Timmerman. "Full and para-virtualization with Xen: a performance comparison". In: *Journal of Emerging Trends in Computing and Information Sciences* 4.9 (2013), pp. 719–727.

[23] Tim Abels, Puneet Dhawan, and Balasubramanian Chandrasekaran. "An overview of xen virtualization". In: *Dell Power Solutions* 8 (2005), pp. 109–111.

[24] Carl Boettiger. "An introduction to Docker for reproducible research". In: *SIGOPS Oper. Syst. Rev.* 49.1 (Jan. 2015). https://doi.org/10.1145/2723872.2723882, pp. 71–79. ISSN: 0163-5980. DOI: 10.1145/2723872.2723882.

[25] Lars Rasmusson and Diarmuid Corcoran. "Performance overhead of KVM on Linux 3.9 on ARM cortex-a15". In: *SIGBED Rev.* 11.2 (Sept. 2014). https://doi.org/10.1145/2668138.2668143, pp. 32–38. DOI: 10.1145/2668138.2668143.

[26] Stephen J. Bigelow. *What's the difference between type 1 vs. type 2 hypervisor?: TechTarget*. Jan. 2021. URL: https://www.techtarget.com/searchitoperations/tip/Whats-the-difference-between-Type-1-vs-Type-2-hypervisor.

[27] Hasan Fayyad-Kazan, Luc Perneel, and Martin Timmerman. "Benchmarking the performance of microsoft hyper-v server, vmware esxi and xen hypervisors". In: *Journal of Emerging Trends in Computing and Information Sciences* 4.12 (2013), pp. 922–933.

[28] Jordan Shropshire. "Analysis of monolithic and microkernel architectures: Towards secure hypervisor design". In: *2014 47th Hawaii International Conference on System Sciences*. IEEE. 2014, pp. 5008–5017.

[29] Hasan Fayyad-Kazan, Luc Perneel, and Martin Timmerman. "Benchmarking the performance of microsoft hyper-v server, vmware esxi and xen hypervisors". In: *Journal of Emerging Trends in Computing and Information Sciences* 4.12 (2013), pp. 922–933.

[30] Pankaj Kadam and Nilesh Alone. "Review on KVM Hypervisor". In: *International Journal of Recent Technology and Engineering (IJRTE) ISSN* (2014), pp. 2277–3878.

[31] David Freet et al. "Open source cloud management platforms and hypervisor technologies: A review and comparison". In: *SoutheastCon 2016* (2016), pp. 1–8.

[32] Xiaolong Wen et al. "Comparison of open-source cloud management platforms: OpenStack and OpenNebula". In: *2012 9th International Conference on Fuzzy Systems and Knowledge Discovery*. IEEE. 2012, pp. 2457–2461.

[33] Google Cloud. *High Performance Computing*. Accessed 2024-03-19. Mar. 2024. URL: https://cloud.google.com/discover/what-is-high-performance-computing.

[34] Oracle. *High Performance Computing*. Accessed 2024-03-19. Mar. 2024. URL: https://www.oracle.com/se/cloud/hpc/what-is-hpc/.

[35] Microsoft. *Availability Zones*. Accessed 2024-03-19. Mar. 2024. URL: https://learn.microsoft.com/en-us/azure/reliability/availability-zones-overview?tabs=azure-cli.

[36] Maged Michael et al. "Scale-up x scale-out: A case study using nutch/lucene". In: *2007 IEEE International Parallel and Distributed Processing Symposium*. IEEE. 2007, pp. 1–8.

[37] IBM. *Cloud scalability: Scale-up vs. scale-out*. Accessed 2024-03-19. Mar. 2024. URL: https://www.ibm.com/blog/cloud-scalability-scale-up-vs-scale-out/.

[38] Sunilkumar S Manvi and Gopal Krishna Shyam. "Resource management for Infrastructure as a Service (IaaS) in cloud computing: A survey". In: *Journal of network and computer applications* 41 (2014), pp. 424–440.

[39] Chnar Mustafa Mohammed and Subhi RM Zeebaree. "Sufficient comparison among cloud computing services: IaaS, PaaS, and SaaS: A review". In: *International Journal of Science and Business* 5.2 (2021), pp. 17–30.

[40] Tiago Rosado and Jorge Bernardino. "An overview of openstack architecture". In: *Proceedings of the 18th International Database Engineering & Applications Symposium*. 2014, pp. 366–367.

[41] Rakesh Kumar et al. "Apache cloudstack: Open source infrastructure as a service cloud computing platform". In: *Proceedings of the International Journal of advancement in Engineering technology, Management and Applied Science* 111 (2014), p. 116.

[42] Anurag Choudhary, Pradeep Kumar Verma, and Piyush Rai. "A walkthrough of Amazon Elastic Compute Cloud (Amazon EC2): A Review". In: *International Journal for Research in Applied Science and Engineering Technology* 9.11 (2021), pp. 93–97.

[43] Marshall Copeland et al. "Getting started with Azure virtual machines". In: *Microsoft Azure: Planning, Deploying, and Managing Your Data center in the Cloud* (2015), pp. 147–176.

[44] SPT Krishnan et al. "Google compute engine". In: *Building Your Next Big Thing with Google Cloud Platform: A Guide for Developers and Enterprise Architects* (2015), pp. 53–81.

[45] Apache CloudStack. *CloudStack Live Migration*. Feb. 2024. URL: https://svn.apache.org/repos/asf/cloudstack/docsite/html/docs/en-US/Apache_CloudStack/4.2.0/html/Admin_Guide/manual-live-migration.html.

[46] Daniel Moritz et al. "Enhancing a virtual security lab with a private cloud framework". In: *Proceedings of 2013 IEEE International Conference on Teaching, Assessment and Learning for Engineering (TALE)*. IEEE. 2013, pp. 314–320.

[47] Younes Khair, Abdeslam Dennai, and Youssef Elmir. "An experimental performance evaluation of OpenNebula and eucalyptus cloud platform solutions". In: *Artificial Intelligence and Heuristics for Smart Energy Efficiency in Smart Cities: Case Study: Tipasa, Algeria*. Springer. 2022, pp. 450–457.

[48] Amazon Web Services. *Amazon EC2*. Feb. 2024. URL: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html.

[49] Rawan Aljamal, Fahed Jubair, and Ali El-Mousa. "Analyzing the Cost-Effectiveness of Running HPC Applications on Microsoft Azure Virtual Machines". In: *2021 IEEE Jordan International Joint Conference on Electrical Engineering and Information Technology (JEEIT)*. IEEE. 2021, pp. 78–82.

[50] Microsoft. *Azure Virtual Machines*. Feb. 2024. URL: https://learn.microsoft.com/en-us/azure/virtual-machines/overview.

[51] TechTarget. *Google Compute Engine*. Feb. 2024. URL: https://www.techtarget.com/searchaws/definition/Google-Compute-Engine#:~:text=Google%20Compute%20Engine%20(GCE)%20is,compute%20clusters%20for%20that%20purpose..

[52] Solanke Vikas et al. "Private vs public cloud". In: *International Journal of Computer Science & Communication Networks* 3.2 (2013), p. 79.

[53] Guruh Aryotejo, Daniel Y Kristiyanto, et al. "Hybrid cloud: bridging of private and public cloud computing". In: *Journal of Physics: Conference Series*. Vol. 1025. 1. IOP Publishing. 2018, p. 012091.

[54] Khouloud Boukadi et al. "From VM to container: A linear program for outsourcing a business process to cloud containers". In: *On the Move to Meaningful Internet Systems. OTM 2017 Conferences: Confederated International Conferences: CoopIS, C&TC, and ODBASE 2017, Rhodes, Greece, October 23-27, 2017, Proceedings, Part I*. Springer. 2017, pp. 488–504.

[55] Nane Kratzke. "About microservices, containers and their underestimated impact on network performance". In: *arXiv preprint arXiv:1710.04049* (2017).

[56] Arnaldo Pereira Ferreira and Richard Sinnott. "A performance evaluation of containers running on managed kubernetes services". In: *2019 IEEE international conference on cloud computing technology and science (CloudCom)*. IEEE. 2019, pp. 199–208.

[57] Docker. *Layers*. Feb. 2024. URL: `https://docs.docker.com/build/guide/layers/`.

[58] Docker. *Dockerfile reference*. Feb. 2024. URL: `https://docs.docker.com/reference/dockerfile/`.

[59] Carl Boettiger. "An introduction to Docker for reproducible research". In: *ACM SIGOPS Operating Systems Review* 49.1 (2015), pp. 71–79.

[60] Steve Buchanan et al. "Container registries". In: *Introducing Azure Kubernetes Service: A Practical Guide to Container Orchestration* (2020), pp. 17–34.

[61] Red Hat. *What is a container registry?* Feb. 2024. URL: `https://www.redhat.com/en/topics/cloud-native-apps/what-is-a-container-registry`.

[62] Open Container Initiative. *Open Container Initiative*. Feb. 2024. URL: `https://opencontainers.org/`.

[63] Arun Gurta. "Kubernetes". In: *Kubernetes. Retrieved May* 24 (2019), p. 2019.

[64] Kubernetes. *Pods*. Accessed 2024-03-19. Mar. 2024. URL: `https://kubernetes.io/docs/concepts/workloads/pods/`.

[65] Kubernetes. *Sidecar containers*. Accessed 2024-04-20. Mar. 2024. URL: `https://kubernetes.io/docs/concepts/workloads/pods/sidecar-containers/`.

[66] Kubernetes. *Services*. Accessed 2024-03-19. Mar. 2024. URL: https://kubernetes.io/docs/concepts/services-networking/service/.

[67] Kubernetes. *Deployments*. Accessed 2024-03-19. Mar. 2024. URL: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/.

[68] Kubernetes. *Persistent Volumes*. Accessed 2024-03-19. Mar. 2024. URL: https://kubernetes.io/docs/concepts/storage/persistent-volumes/.

[69] Kubernetes. *StatefulSets*. Accessed 2024-03-19. Mar. 2024. URL: https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/.

[70] Kakoli Banerjee et al. "A Survey on Kubernetes Policy Report Custom Resource Definition Kube-Bench Adapter". In: *Advances in Data and Information Sciences: Proceedings of ICDIS 2022*. Springer, 2022, pp. 315–322.

[71] Kubernetes. *Custom Resource Definitions*. Feb. 2024. URL: https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/.

[72] Kubernetes. *Controllers*. Accessed 2024-03-20. Mar. 2024. URL: https://kubernetes.io/docs/concepts/architecture/controller/.

[73] Kubernetes. *Kubernetes Components*. Accessed 2024-04-20. Mar. 2024. URL: https://kubernetes.io/docs/concepts/overview/components/.

[74] Kubernetes. *kubelet*. Accessed 2024-03-20. Mar. 2024. URL: https://kubernetes.io/docs/reference/command-line-tools-reference/kubelet/.

[75] Kubernetes. *kube-proxy*. Accessed 2024-03-20. Mar. 2024. URL: https://kubernetes.io/docs/reference/command-line-tools-reference/kube-proxy/.

[76] Prateek Sharma et al. "Containers and virtual machines at scale: A comparative study". In: *Proceedings of the 17th international middleware conference*. 2016, pp. 1–13.

[77] Alessandro Randazzo and Ilenia Tinnirello. "Kata containers: An emerging architecture for enabling mec services in fast and secure way". In: *2019 Sixth International Conference on Internet of Things: Systems, Management and Security (IOTSMS)*. IEEE. 2019, pp. 209–214.

[78] Shashank Mohan Jain and Shashank Mohan Jain. "Cgroups". In: *Linux Containers and Virtualization: A Kernel Perspective* (2020), pp. 45–80.

[79] Amazon Web Services. *Noisy Neighbor*. Feb. 2024. URL: `https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/noisy-neighbor.html`.

[80] Prateek Sharma et al. "Containers and virtual machines at scale: A comparative study". In: *Proceedings of the 17th international middleware conference*. 2016, pp. 1–13.

[81] Jinchun Kim et al. "Dynamic memory pressure aware ballooning". In: *Proceedings of the 2015 International Symposium on Memory Systems*. 2015, pp. 103–112.

[82] Ishan Banerjee et al. "Memory overcommitment in the ESX server". In: *VMware Technical Journal* 2.1 (2013), pp. 2–12.

[83] Christopher Clark et al. "Live migration of virtual machines". In: *Proceedings of the 2nd conference on Symposium on Networked Systems Design & Implementation-Volume 2*. 2005, pp. 273–286.

[84] Andrey Mirkin, Alexey Kuznetsov, and Kir Kolyshkin. "Containers checkpointing and live migration". In: *Proceedings of the Linux Symposium*. Vol. 2. 2008, pp. 85–90.

[85] Radostin Stoyanov and Martin J Kollingbaum. "Efficient live migration of linux containers". In: *High Performance Computing: ISC High Performance 2018 International Workshops, Frankfurt/Main, Germany, June 28, 2018, Revised Selected Papers 33*. Springer. 2018, pp. 184–193.

[86] CRIU. *CRIU*. Feb. 2024. URL: `https://criu.org/`.

[87] Red Hat. *Containers vs VMs*. Feb. 2024. URL: `https://www.redhat.com/en/topics/containers/containers-vs-vms`.

[88] GitHub. *Self-hosted runner security*. Accessed 2024-04-20. Mar. 2024. URL: https://docs.github.com/en/actions/hosting-your-own-runners/managing-self-hosted-runners/about-self-hosted-runners#self-hosted-runner-security.

[89] Kong. *What Are Virtual Machines (VMs)?* Feb. 2024. URL: https://konghq.com/learning-center/service-mesh/virtual-machines.

[90] Chris Greamo and Anup Ghosh. "Sandboxing and virtualization: Modern tools for combating malware". In: *IEEE Security & Privacy* 9.2 (2011), pp. 79–82.

[91] Tripwire. *What is Configuration Drift?* Feb. 2024. URL: https://www.tripwire.com/state-of-security/what-is-configuration-drift.

[92] Shared Rotlevi. *Configuration Drift Explained*. Feb. 2024. URL: https://www.wiz.io/academy/configuration-drift.

[93] Sysdig. *Containers vs. Virtual Machines – Making an Informed Decision*. Feb. 2024. URL: https://sysdig.com/learn-cloud-native/kubernetes-security/containers-vs-virtual-machines-making-an-informed-decision/.

[94] Microsoft. *Virtual machines: virtual computers within computers*. Feb. 2024. URL: https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-a-virtual-machine.

[95] Anjali, Tyler Caraza-Harter, and Michael M Swift. "Blending containers and virtual machines: a study of firecracker and gVisor". In: *Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*. 2020, pp. 101–113.

[96] Anjali, Tyler Caraza-Harter, and Michael M Swift. "Blending containers and virtual machines: a study of firecracker and gVisor". In: *Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*. 2020, pp. 101–113.

[97] Kubernetes. *kube-scheduler*. Accessed 2024-02-29. Feb. 2024. URL: https://kubernetes.io/docs/reference/command-line-tools-reference/kube-scheduler/.

[98] KubeVirt. *KubeVirt Architecture*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/architecture/.

[99] KubeVirt. *API Validation*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/api_validation/.

[100] KubeVirt. *Virtual Machine Instances*. Accessed 2024-04-21. Mar. 2024. URL: https://kubevirt.io/user-guide/virtual_machines/virtual_machine_instances/.

[101] Kubernetes. *Getting to know KubeVirt*. Accessed 2024-03-20. Mar. 2024. URL: https://kubernetes.io/blog/2018/05/22/getting-to-know-kubevirt/.

[102] KubeVirt. *Snapshot Restore API*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/snapshot_restore_api/.

[103] KubeVirt. *Live Migration*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/live_migration/.

[104] Kubernetes. *Persistent Volumes*. Accessed 2024-02-29. Feb. 2024. URL: https://kubernetes.io/docs/concepts/storage/persistent-volumes/.

[105] KubeVirt. *virtctl*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/virtctl_client_tool/.

[106] Florian Beetz and Simon Harrer. "Gitops: The evolution of devops?" In: *IEEE Software* 39.4 (2021), pp. 70–75.

[107] KubeVirt. *GitOps*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/gitops/.

[108] KubeVirt. *Comparing KubeVirt to other technologies*. Accessed 2024-02-29. Feb. 2017. URL: https://kubevirt.io/2017/technology-comparison.html.

[109] KubeVirt. *Network Binding Plugins*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/virtual_machines/network_binding_plugins/.

[110] KubeVirt. *Disks and Volumes*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/virtual_machines/disks_and_volumes/.

[111] Supakorn Trakulmaiphol and Krerk Piromsopa. "An Implementation of DNS Operator and Network Guideline for Migrating Virtual Machine to KubeVirt". In: *2022 26th International Computer Science and Engineering Conference (ICSEC)*. 2022, pp. 84–89. DOI: 10.1109/ICSEC56337.2022.10049369.

[112] Mengmei Ye et al. "Free the Turtles: Removing Nested Virtualization for Performance and Confidentiality in the Cloud". In: *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*. 2023, pp. 275–281. DOI: 10.1109/CLOUD60044.2023.00039.

[113] Oskar Sonninen. "Perceived benefits of declarative software deployment: an exploratory case study". In: (2020).

[114] Muhammad Imran et al. "Migration of CMSWEB cluster at CERN to Kubernetes: a comprehensive study". In: *Cluster Computing* 24.4 (2021), pp. 3085–3099.

[115] Rukaiah NJH Badran. *Automation and improvement in CMSWEB Kubernetes clusters*. Tech. rep.

[116] Sravanthi Gelley. "Migrate Cloud Foundry Application to Kubernetes". In: (2022).

[117] Michael Packard et al. "Real-World, Self-Hosted Kubernetes Experience". In: *Practice and Experience in Advanced Research Computing*. 2021, pp. 1–5.

[118] Mikhail Boronin. *Hybrid Cloud Migration Challenges. A case study at King*. 2020.

[119] Mikyung Kang et al. "A comparison of system performance on a private openstack cloud and amazon ec2". In: *2017 IEEE 10th International Conference on Cloud Computing (CLOUD)*. IEEE. 2017, pp. 310–317.

[120] Jaison Paul Mullerikkal and Yedhu Sastri. "A comparative study of OpenStack and CloudStack". In: *2015 Fifth International Conference on Advances in Computing and Communications (ICACC)*. IEEE. 2015, pp. 81–84.

[121] Aaron Paradowski, Lu Liu, and Bo Yuan. "Benchmarking the Performance of OpenStack and CloudStack". In: *2014 IEEE 17th International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing*. 2014, pp. 405–412. DOI: 10.1109/ISORC.2014.12.

[122] Marcelo Amaral. *Kubevirt scale test by creating 400 vmis on a single node for FOSDEM 2022: IBM Research*. Feb. 2022. URL: https://research.ibm.com/publications/kubevirt-scale-test-by-creating-400-vmis-on-a-single-node.

[123] RISSO FULVIO GIOVANNI OTTAVIO and Alireza Kolahdouzan. "Network Automation, Orchestration, Cloud: Tools and Technologies". In: (2019).

[124] Jinho Hwang et al. "A component-based performance comparison of four hypervisors". In: *2013 IFIP/IEEE International Symposium on Integrated Network Management (IM 2013)*. IEEE. 2013, pp. 269–276.

[125] Robert-Steve Schmoll et al. "Comparing and evaluating application-specific boot times of virtualized instances". In: *2019 IEEE 2nd 5G World Forum (5GWF)*. IEEE. 2019, pp. 602–606.

[126] Microsoft. *Dv3 and Dsv3 series*. Accessed 2024-03-20. Mar. 2024. URL: https://learn.microsoft.com/en-us/azure/virtual-machines/dv3-dsv3-series.

[127] Microsoft. *Azure Pricing Calculator*. Feb. 2024. URL: https://azure.microsoft.com/en-us/pricing/calculator/.

Appendix A

Performance Analysis Components

A.1 VM Management System Interface

```
type VmManagementSystem interface {
    Setup() error
    Get\Gls{vm}(name string) *models.\Gls{vm}
    List\Glspl{vm}() []models.\Gls{vm}
    Create\Gls{vm}(vm *models.\Gls{vm}, hostIdx ...int)
       error
    Delete\Gls{vm}(name string) error
    Migrate\Gls{vm}(name string, hostIdx int) error
    ConnectWorker(workerIdx int) error
    DisconnectWorker(workerIdx int) error
    WaitForRunning\Gls{vm}(name string) error
    DeleteAll\Glspl{vm}() error
}
```

A.2 Metrics Collector

```
#!/bin/bash

interval=1
OUTPUT_FILE="./output.json"
```

```
cpu_tmp_file=$(mktemp)
ram_tmp_file=$(mktemp)
disk_tmp_file=$(mktemp)

while true; do
    if [ ! -f "./run" ]; then
        sleep $interval
        continue
    fi

    (mpstat -P ALL "$interval" 1 | awk '/^[0-9]/␣{if␣(NR
       >1)␣print␣$3/100}' | jq -R -s -c 'split("\n")
       [1:-1]' | jq '[.[]␣|␣gsub(",";␣".")␣|␣tonumber]'
       > "$cpu_tmp_file") &

    (free | awk '/Mem:/␣{print␣$3/$2}' | sed 's/,/./' >
       "$ram_tmp_file") &

    (iostat -dx "$disk" 1 2 | awk '/Device/{flag=1;␣next
       }␣flag␣&&␣/^[^␣]/␣{util=$NF}␣END{print␣util/100}'
        | sed 's/,/./' > "$disk_tmp_file") &
    timestamp=$(date -u +"%Y-%m-%dT%H:%M:%SZ")

    wait

    name=$(hostname)
    cpu_core_utilization=$(cat "$cpu_tmp_file")
    cpu_utilization_avg=$(echo "$cpu_core_utilization" |
        jq 'add/length')
    ram_utilization=$(cat "$ram_tmp_file")
    disk_utilization=$(cat "$disk_tmp_file")

    # Create the JSON object for the current datapoint
    datapoint=$(jq -n \
                  --argjson cpuCores "
                     $cpu_core_utilization" \
                  --argjson cpuAverage "
                     $cpu_utilization_avg" \
                  --arg name "$name" \
                  --arg ramUtilization "$ram_utilization
                     " \
                  --arg diskUtilization "
```

```
                    $disk_utilization" \
                 --arg timestamp "$timestamp" \
                 '{name:␣$name,␣cpu:␣{cores:␣$cpuCores,
                    ␣average:␣$cpuAverage},␣ram:␣(
                    $ramUtilization␣|␣tonumber),␣disk:␣
                    ($diskUtilization␣|␣tonumber),␣
                    timestamp:␣$timestamp}')

    if [ ! -f "$OUTPUT_FILE" ]; then
        echo "[]" > "$OUTPUT_FILE"
    fi

    jq --argjson newData "$datapoint" '.␣+=␣[$newData]'
       "$OUTPUT_FILE" > tmp.$$.json && mv tmp.$$.json "
       $OUTPUT_FILE"

    sleep $interval
done
```

A.3 Metrics Collector Daemon

```
[Unit]
Description=Metrics scraping
[Service]
WorkingDirectory=/home/user
ExecStart=/bin/bash /home/user/metrics.sh
Restart=always
[Install]
WantedBy=multi-user.target
```

Appendix B

Performance Analysis Results

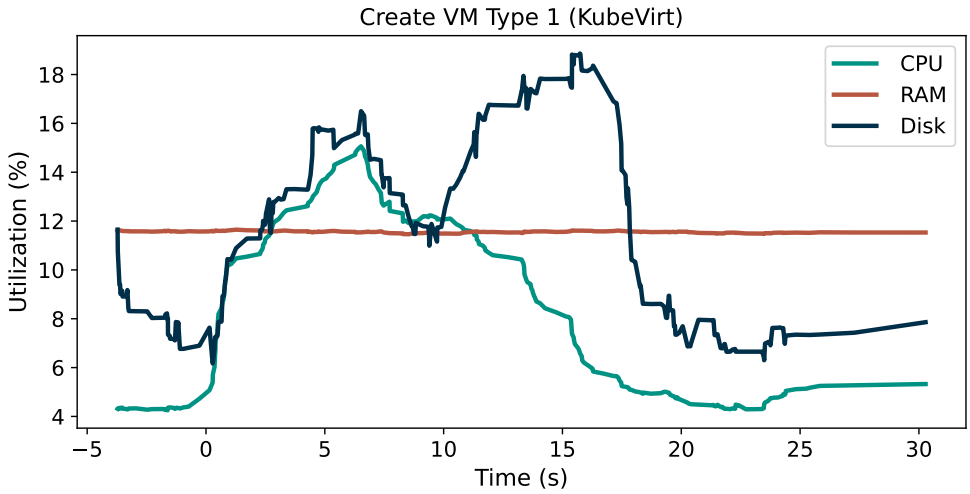

Figure B.1: Resource utilization on the control node when creating a VM type 1 in KubeVirt. CPU and disk utilization follow a similar pattern nearly reaching 20% while memory utilization remains constant at 12% for the duration of the test.

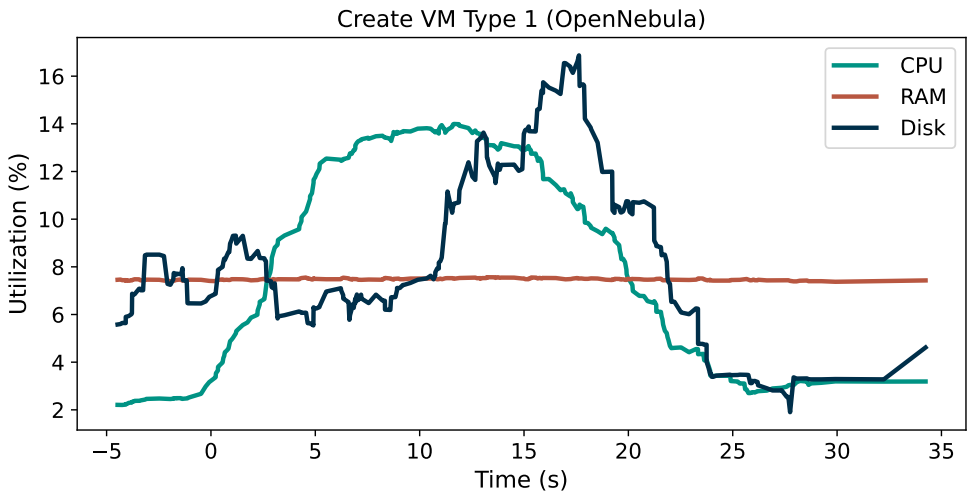

Figure B.2: Resource utilization on the control node when creating a VM type 1 in OpenNebula. CPU and disk utilization both increase and peak after half the duration of the test. Memory utilization remains constant below 8% for the duration of the test.

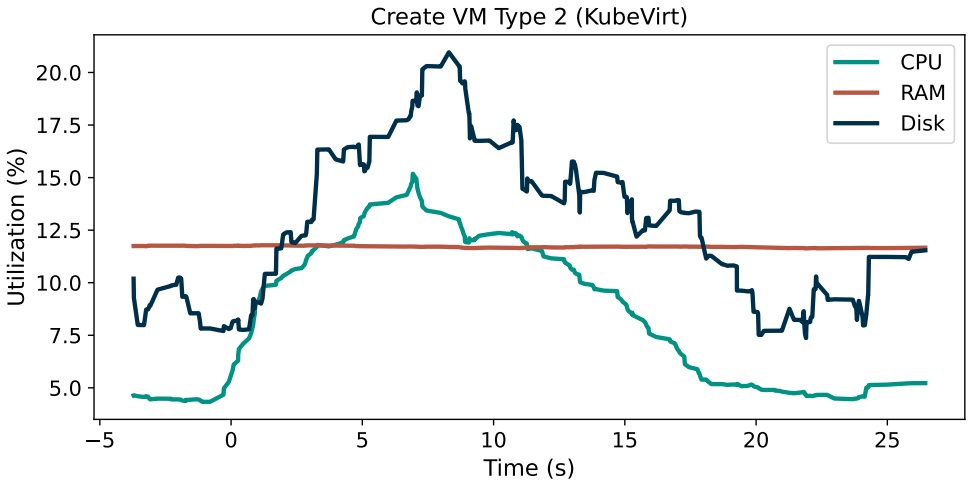

Figure B.3: Resource utilization on the control node when creating a VM type 2 in KubeVirt. The disk utilization is the most prominent among the resources, reaching over 20% utilization. Memory utilization remains constant at 12% for the duration of the test.

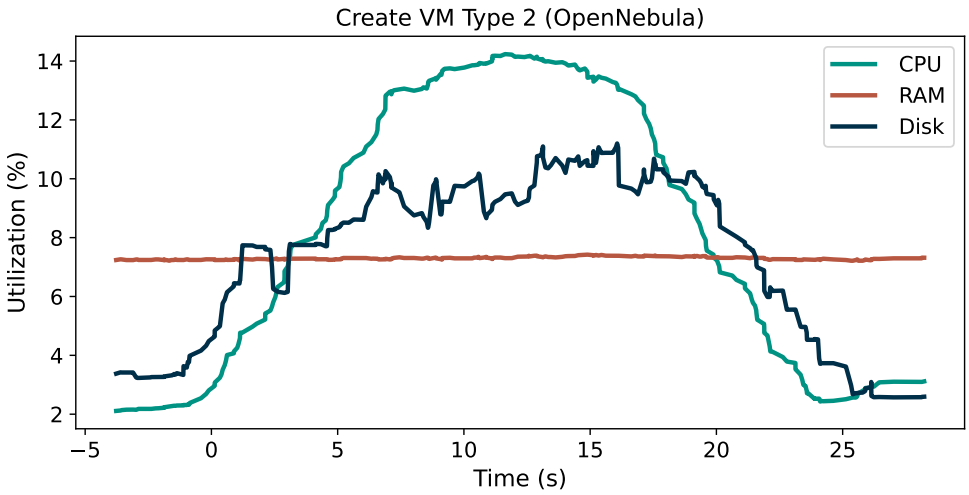

Figure B.4: Resource utilization on the control node when creating a VM type 2 in OpenNebula. Disk utilization plateaus quickly reaching just above 10% utilization while CPU utilization peaks just below 15%. Memory utilization remains constant at 8% for the duration of the test.

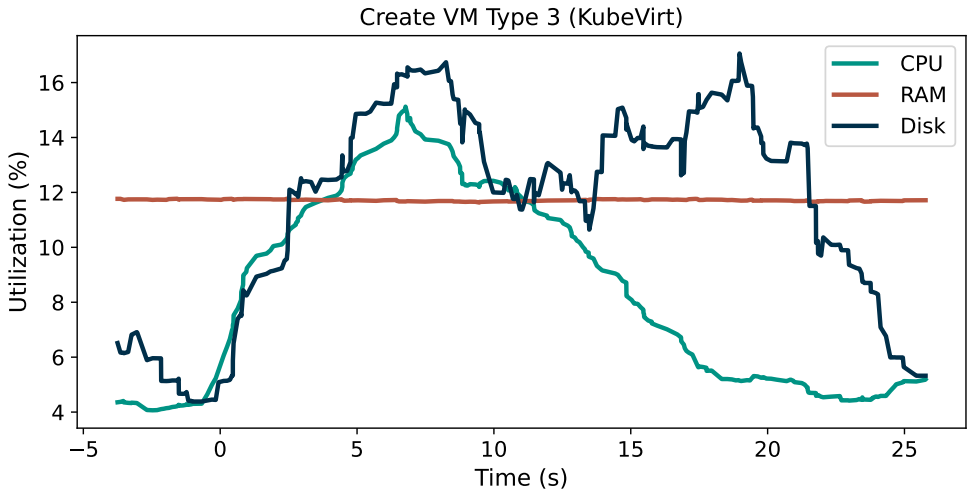

Figure B.5: Resource utilization on the control node when creating a VM type 3 in KubeVirt. CPU and disk utilization initially follow a similar pattern, but disk utilization remains high for the duration of the test. Memory utilization remains constant at 12% for the duration of the test.

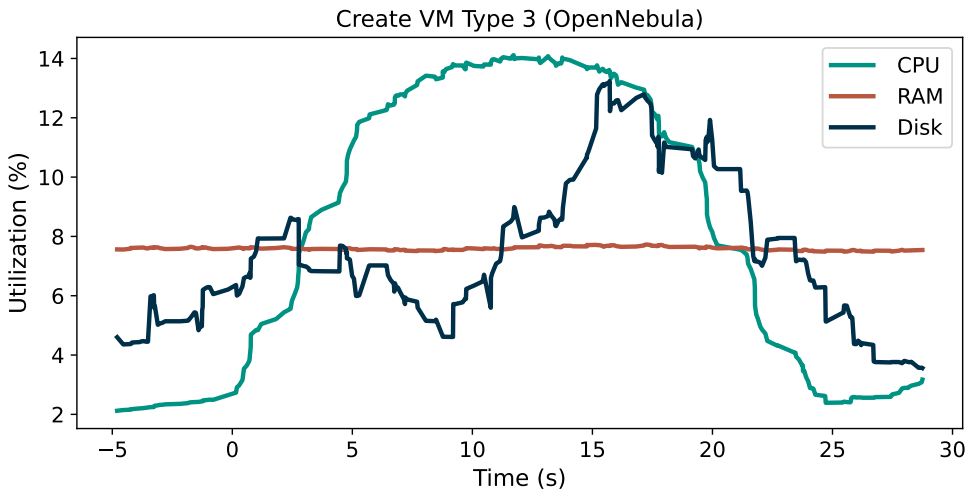

Figure B.6: Resource utilization on the control node when creating a VM type 3 in OpenNebula. CPU utilization is the most prominent among the resources reaching nearly 15% utilization. Memory utilization remains constant at 8% for the duration of the test.

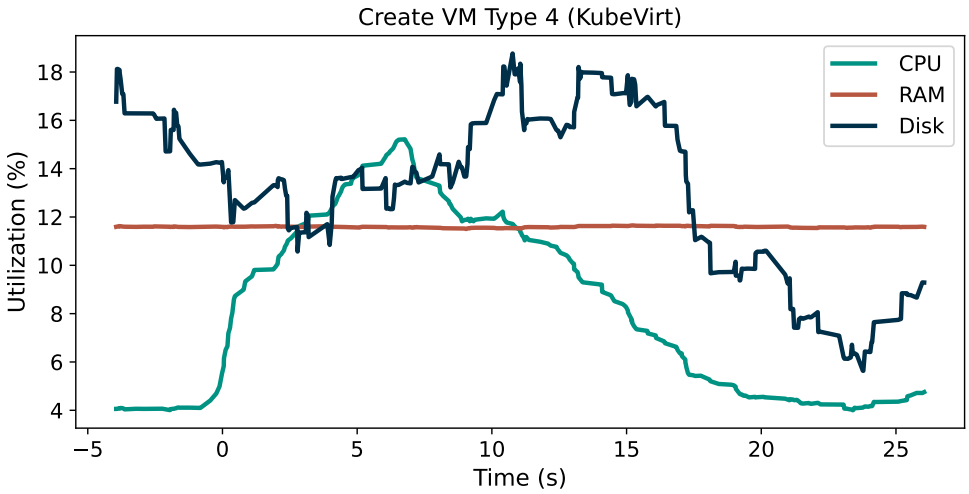

Figure B.7: Resource utilization on the control node when creating a VM type 4 in KubeVirt. CPU utilization peaks quickly while disk utilization is sporadic. The sporadic behavior is not in line with the operations performed during the test. Memory utilization remains constant at 12% for the duration of the test.

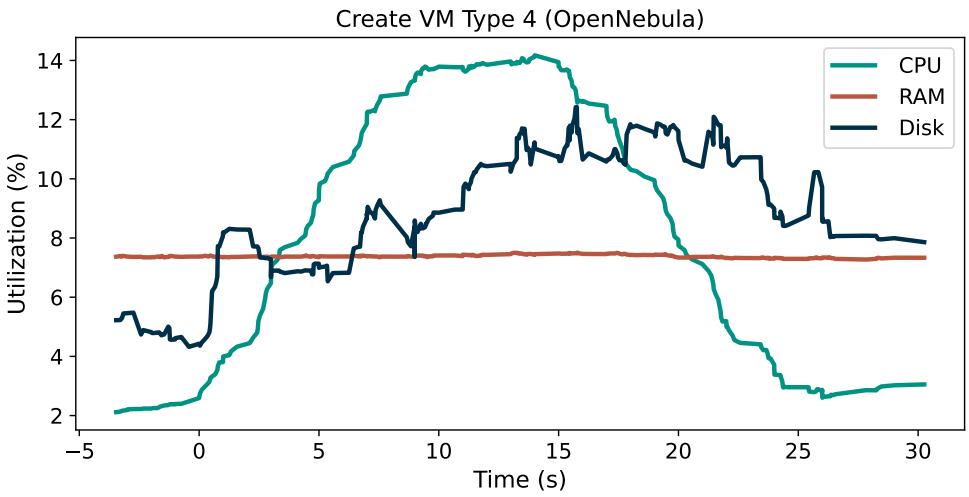

Figure B.8: Resource utilization on the control node when creating a VM type 4 in OpenNebula. CPU utilization is the most prominent among the resources plateauing at nearly 15% utilization. Memory utilization remains constant at 8% for the duration of the test.

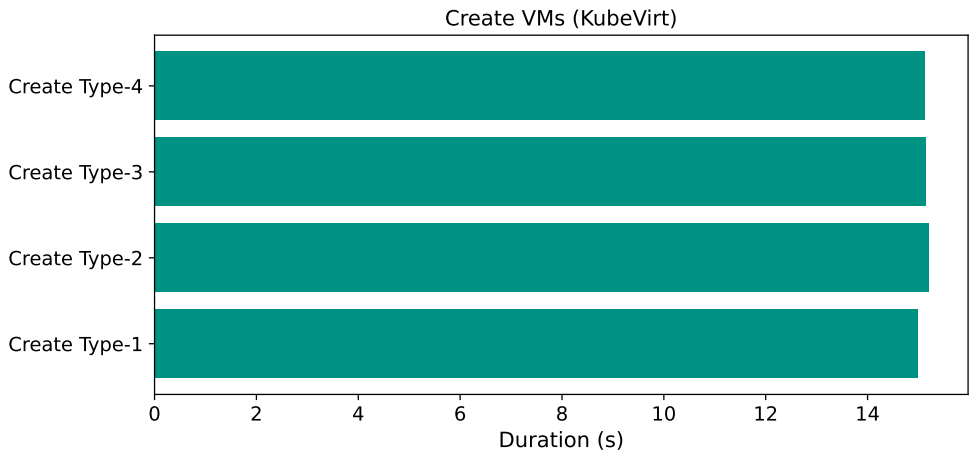

Figure B.9: Duration of creating VMs in KubeVirt. The duration for all types remains constant for each VM type reaching slightly above 14 seconds

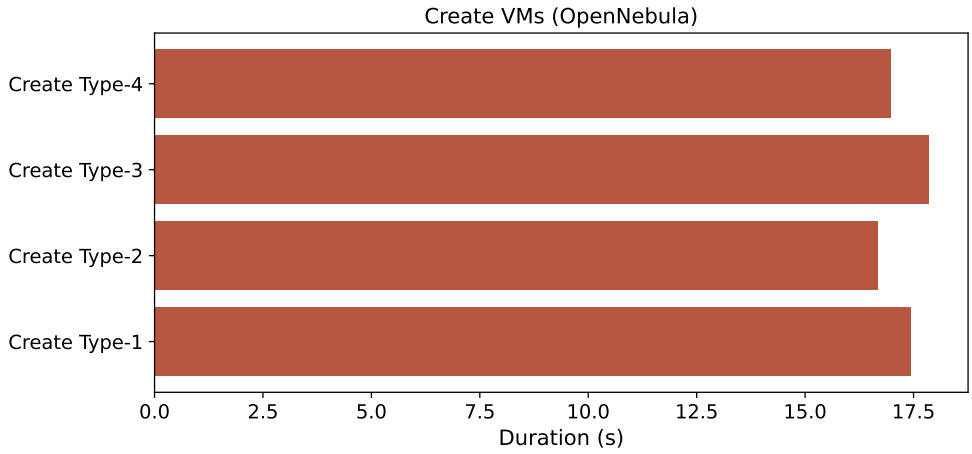

Figure B.10: Duration of creating VMs in OpenNebula. The duration for all types remains constant for each VM type reaching nearly 18 seconds.

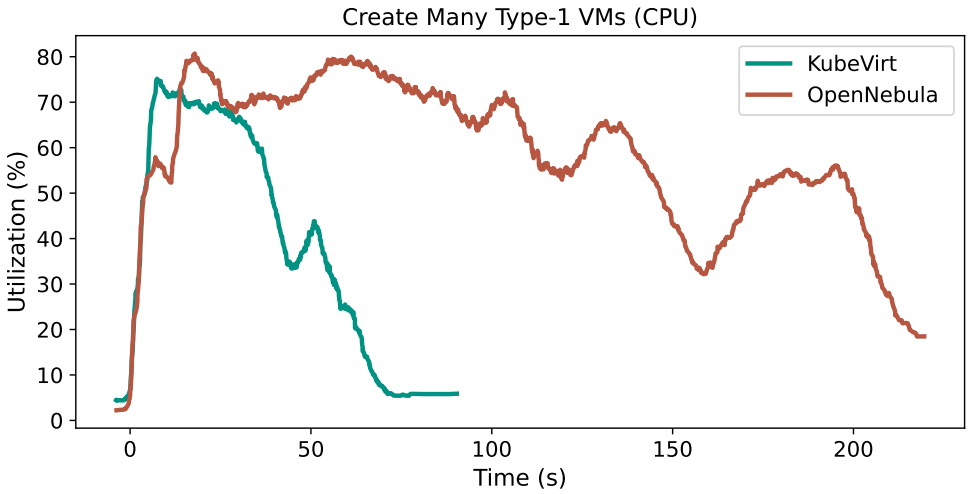

Figure B.11: CPU utilization on the control node when creating 20 type-1 VMs. The CPU utilization peaks quickly for both KubeVirt and OpenNebula due to the burst of requests. KubeVirt reaches a slightly lower peak than OpenNebula and subsides significantly faster. OpenNebula does not manage to return to a utilization level of an idle state within the duration of the test, as evidenced by the utilization not reaching base utilization in the end.

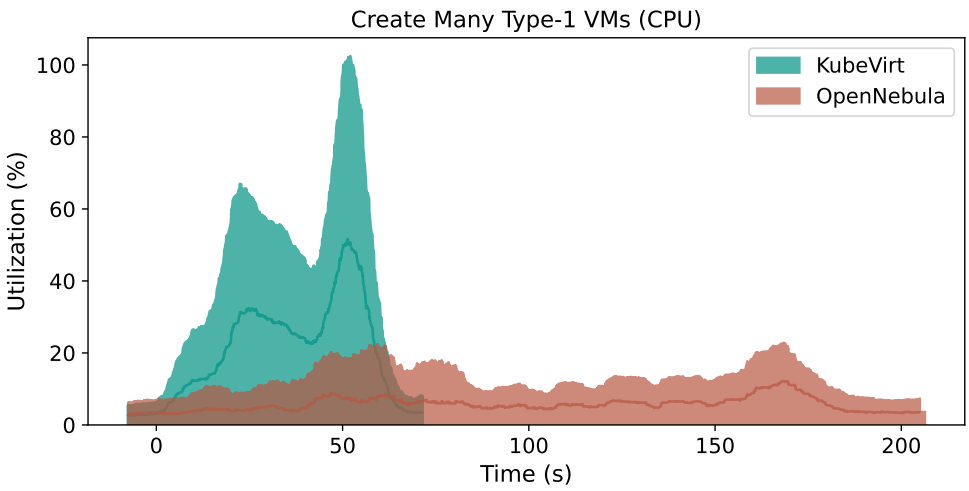

Figure B.12: Cumulative CPU utilization among worker nodes when creating 20 type-1 VMs. The CPU utilization among the worker nodes is vastly different between KubeVirt and OpenNebula, where KubeVirt experiences a short-lived large burst of CPU utilization while OpenNebula barely reaches a cumulative 20% utilization.

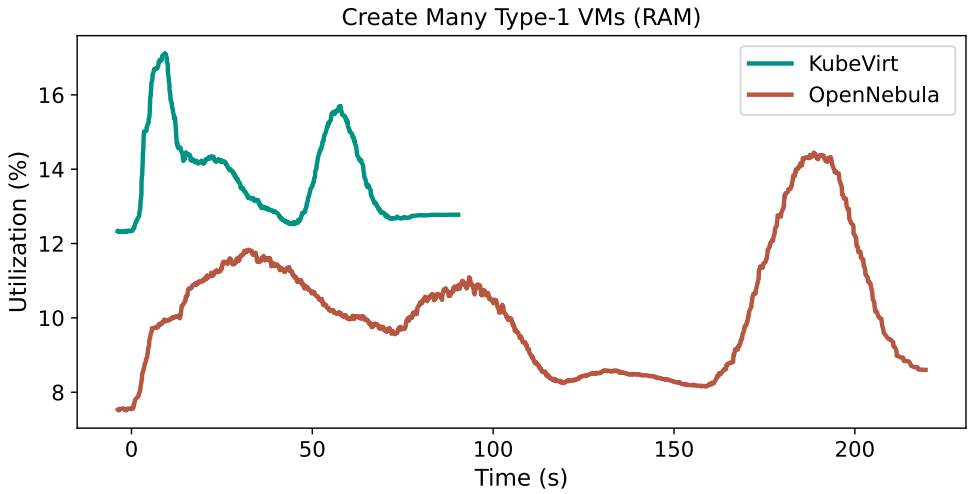

Figure B.13: Memory utilization on the control node when creating 20 type-1 VMs. KubeVirt experiences a burst of memory utilization in the beginning, while OpenNeubla has a more gradual increase before spiking quickly toward the end of the test. While KubeVirt has a larger base memory utilization, OpenNebula experiences a larger induced load from the test, as evidenced by its longer slopes.

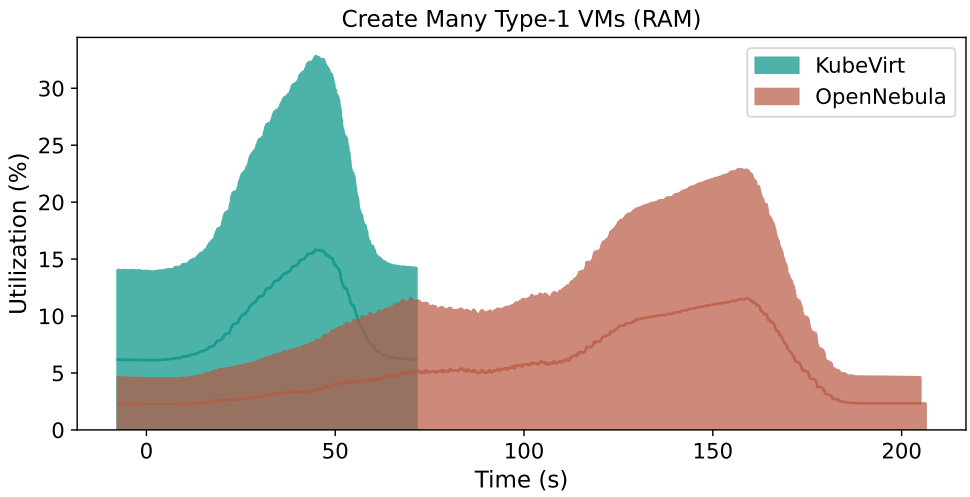

Figure B.14: Cumulative memory utilization among worker nodes when creating 20 type-1 VMs. While KubeVirt experiences a larger memory utilization due to a larger base, both systems have a similar magnitude of the induced load, albeit KubeVirt's load is more short-lived.

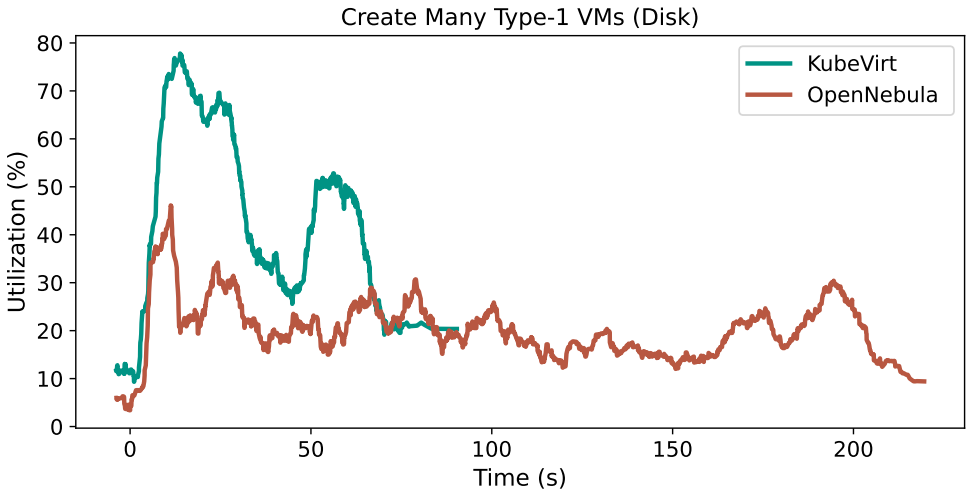

Figure B.15: Disk utilization on the control node when creating 20 type-1 VMs. KubeVirt experiences a larger burst of disk utilization than OpenNebula, while OpenNebula maintains a steady disk utilization throughout the test.

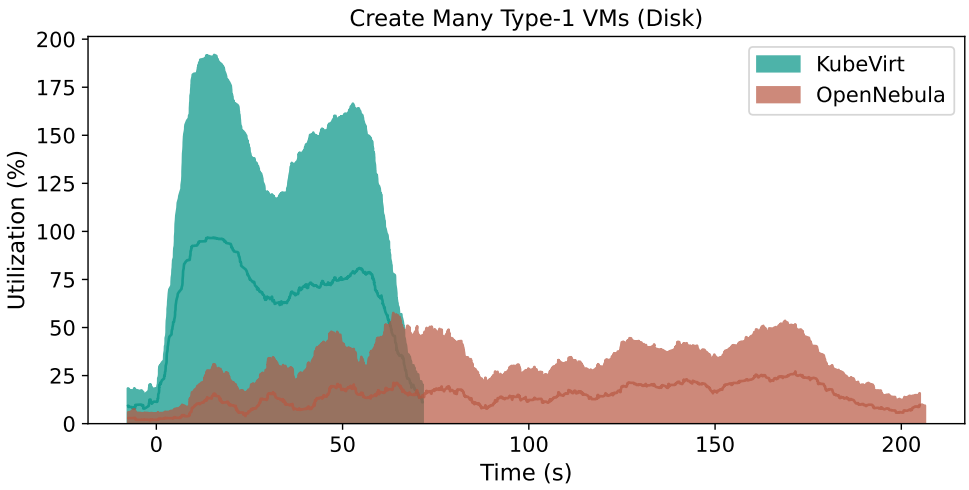

Figure B.16: Cumulative disk utilization among worker nodes when creating 20 type-1 VMs. KubeVirt experiences a larger burst of disk utilization than OpenNebula, while OpenNebula maintains a low and steady disk utilization throughout the test.

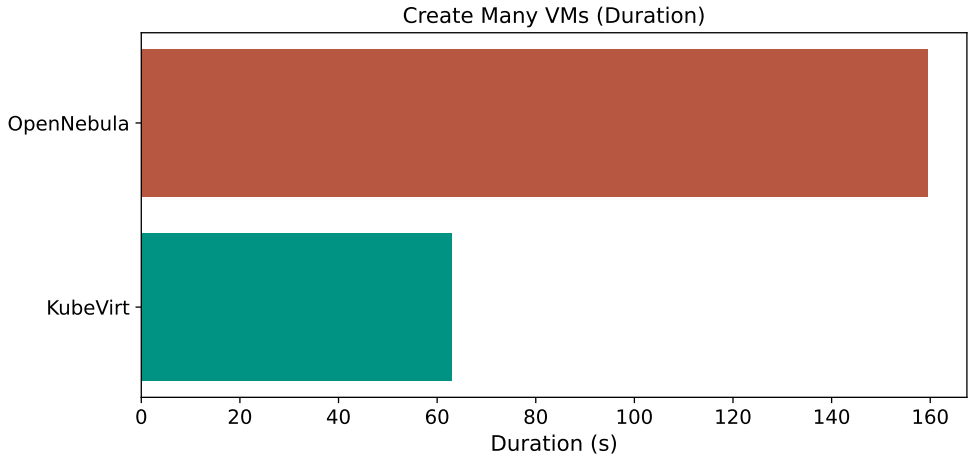

Figure B.17: Duration of creating 20 type-1 VMs. KubeVirt is significantly faster to create 20 type-1 VMs than OpenNebula, taking 67 seconds compared to OpenNebula's 192 seconds.

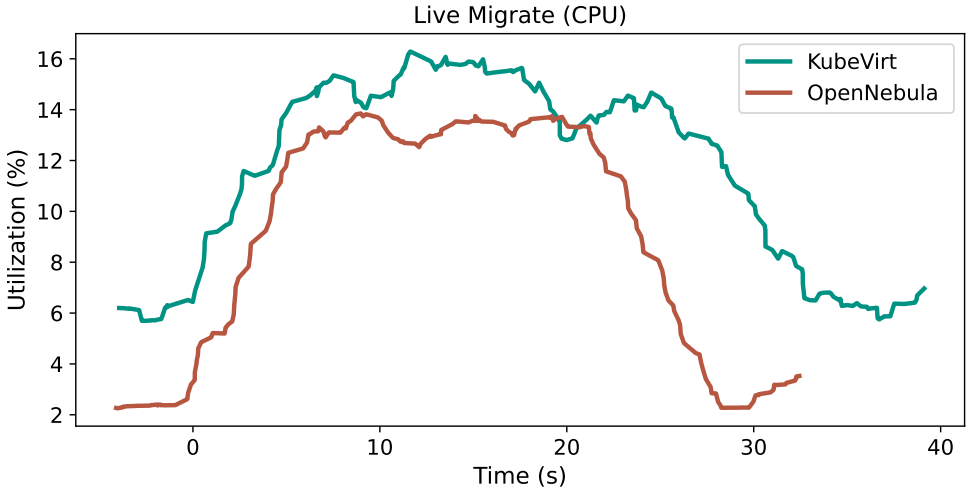

Figure B.18: CPU utilization on the control node when live migrating a type-4 VM. KubeVirt has a slightly larger base utilization than OpenNebula, but the induced load is slightly larger for OpenNebula than KubeVirt. They follow a similar pattern peaking at 16% and 14% respectively.

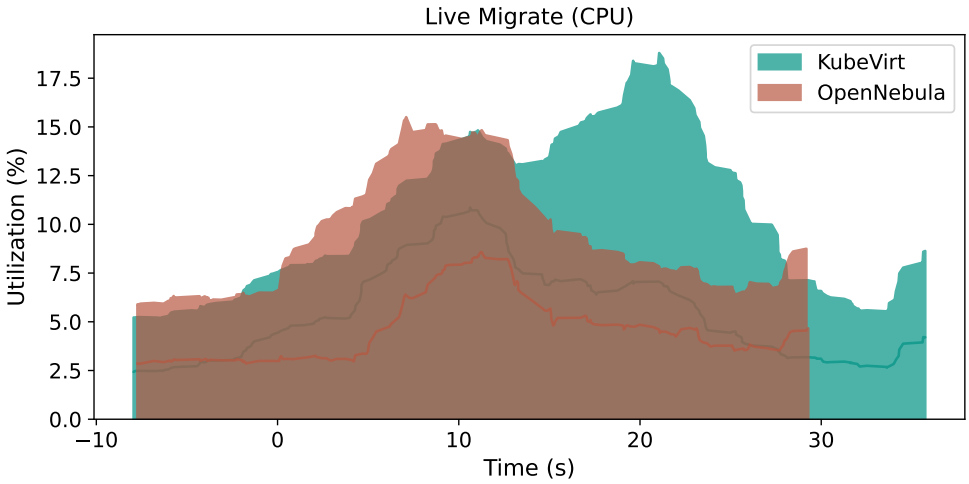

Figure B.19: Cumulative CPU utilization among worker nodes when live migrating a type-4 VM. Both systems follow a similar pattern where OpenNebula experiences a slightly lower utilization than KubeVirt. The induced load is shifted towards the end of the test for KubeVirt since it executes the operation more slowly.

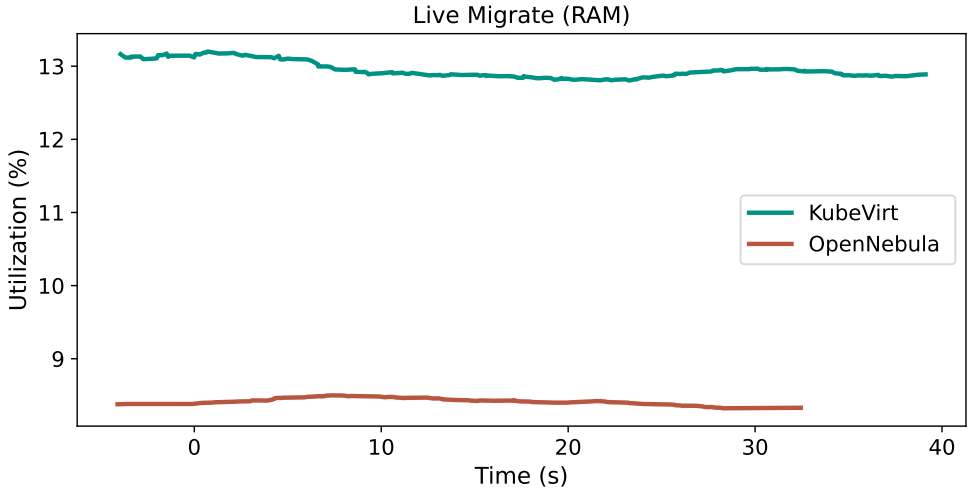

Figure B.20: Cumulative memory utilization on the control node when live migrating a type-4 VM. While KubeVirt has a larger base memory utilization than OpenNebula, the induced load is non-existent for both systems.

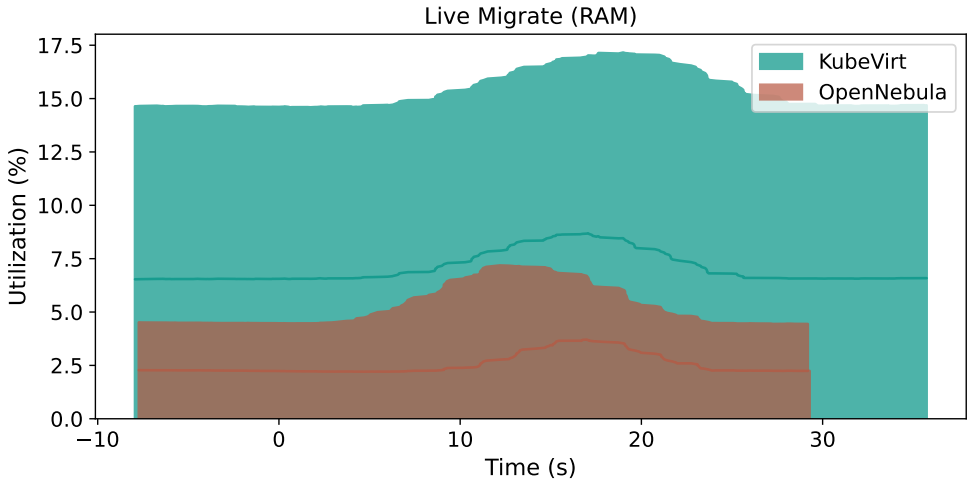

Figure B.21: Cumulative memory utilization among worker nodes when live migrating a type-4 VM. The load is similar for the two systems, where the load is shifted between the two workers over time. The overlap of the peaks between the workers shows that the migration process affects both workers simultaneously during a short period.

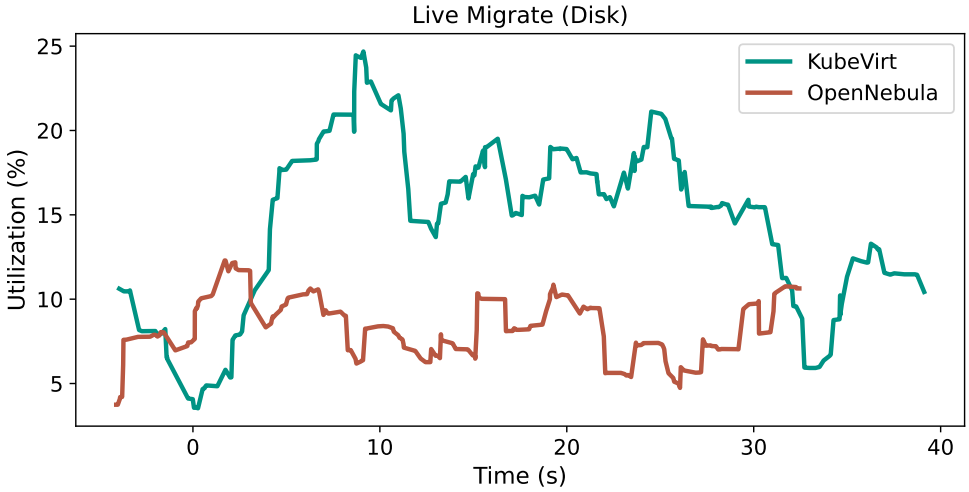

Figure B.22: Disk utilization on the control node when live migrating a type-4 VM. KubeVirt experiences a larger disk utilization than OpenNebula for the entirety of the test. However, the disk usage is sporadic and many peaks are not in line with the operations performed during the test.

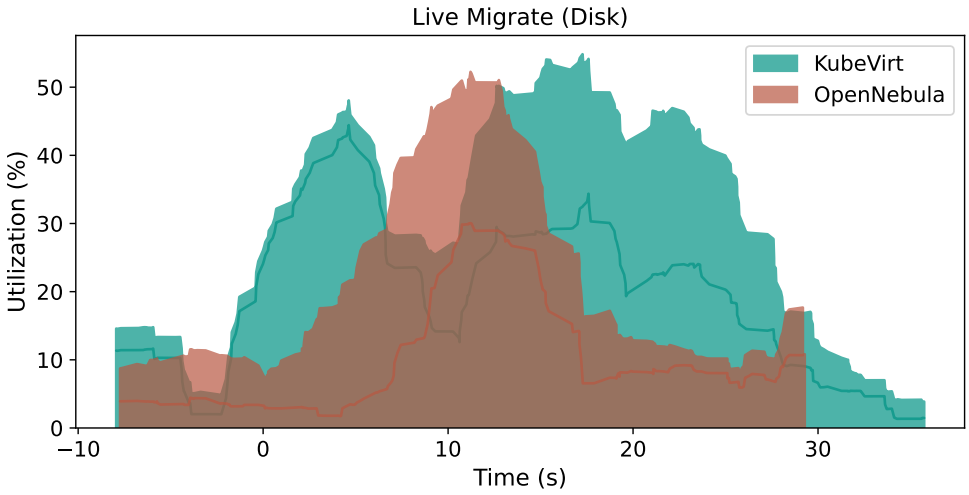

Figure B.23: Cumulative disk utilization among worker nodes when live migrating a type-4 VM. Both KubeVirt and OpenNebula reach a similar peak cumulative disk utilization. However, OpenNebula experiences a shorter plateau of disk utilization than KubeVirt.

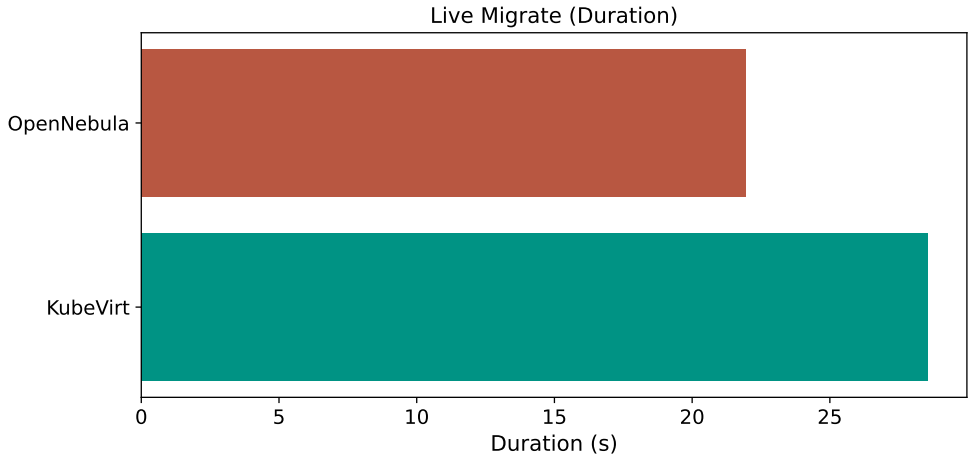

Figure B.24: Duration of live migrating a type-4 VM. KubeVirt is slower than OpenNebula, taking on average 7 seconds longer to complete. The duration is consistent for both systems.

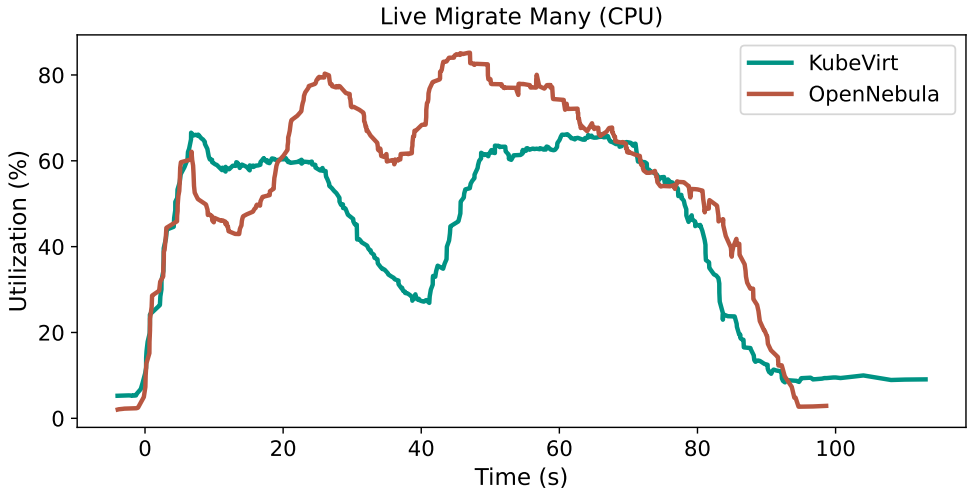

Figure B.25: CPU utilization on the control node when live migrating 10 type-1 VMs. Both systems follow a similar pattern where KubeVirt has a slightly larger base utilization than OpenNebula, but the induced load is larger for OpenNebula.

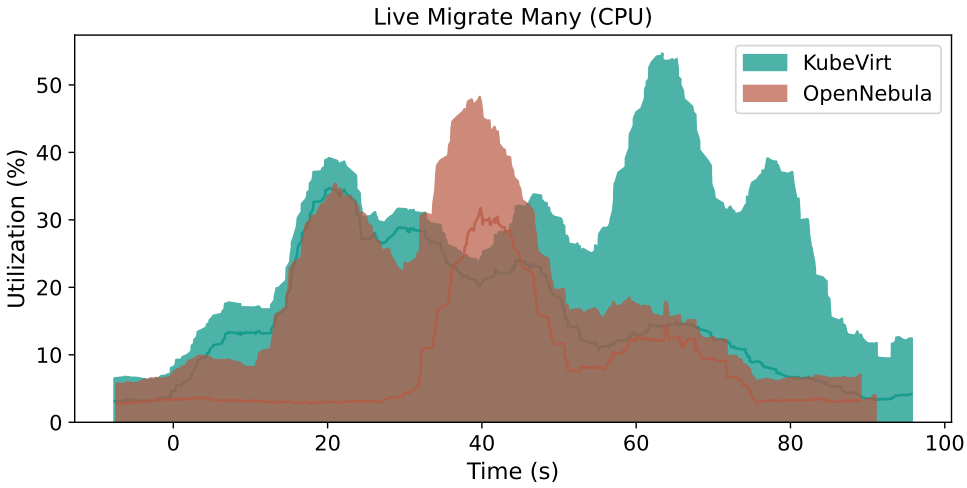

Figure B.26: Cumulative CPU utilization among worker nodes when live migrating 10 type-1 VMs. Both systems experience a similar load for the first half of the test, but KubeVirt experiences a significantly larger load in the second half of the test.

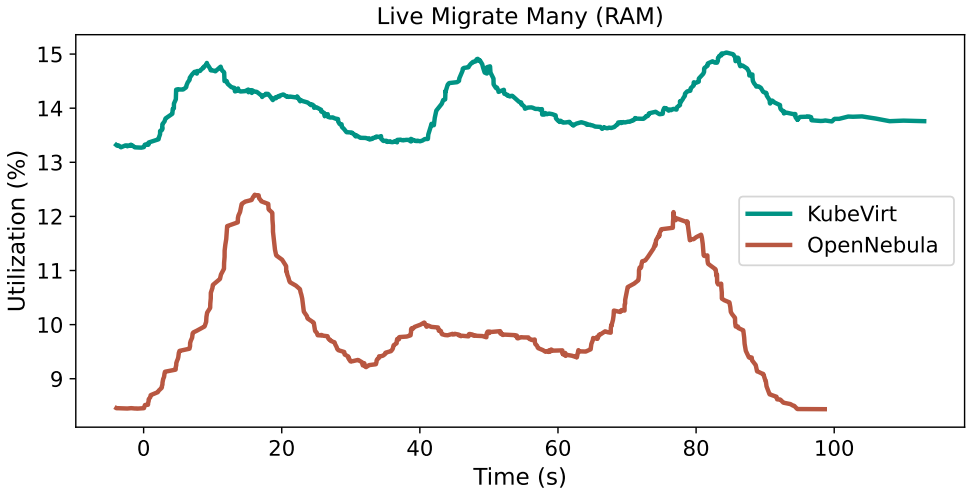

Figure B.27: Memory utilization on the control node when live migrating 10 type-1 VMs. KubeVirt has a larger base memory utilization than OpenNebula, but the induced load is more than double for OpenNebula.

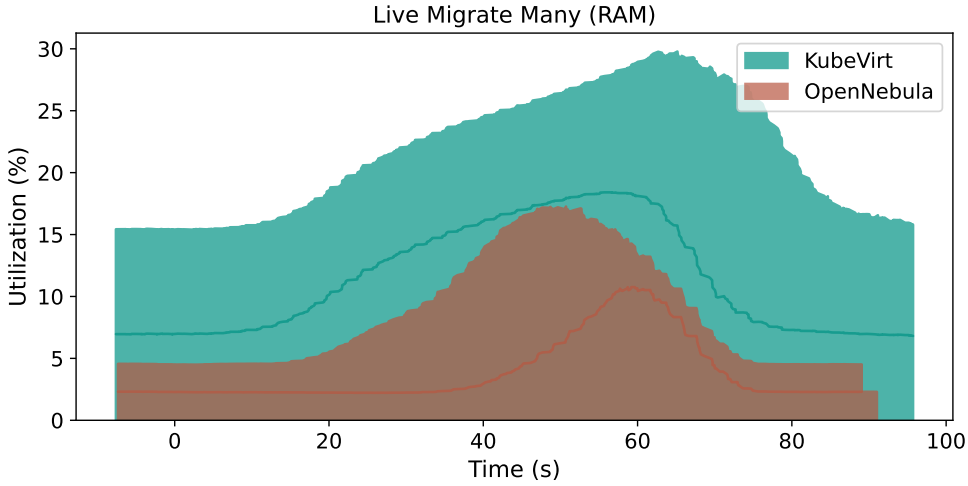

Figure B.28: Cumulative memory utilization among worker nodes when live migrating 10 type-1 VMs. KubeVirt has a larger base memory utilization than OpenNebula, but both systems experience a similar induced load. OpenNebula returns to an idle state faster than KubeVirt.

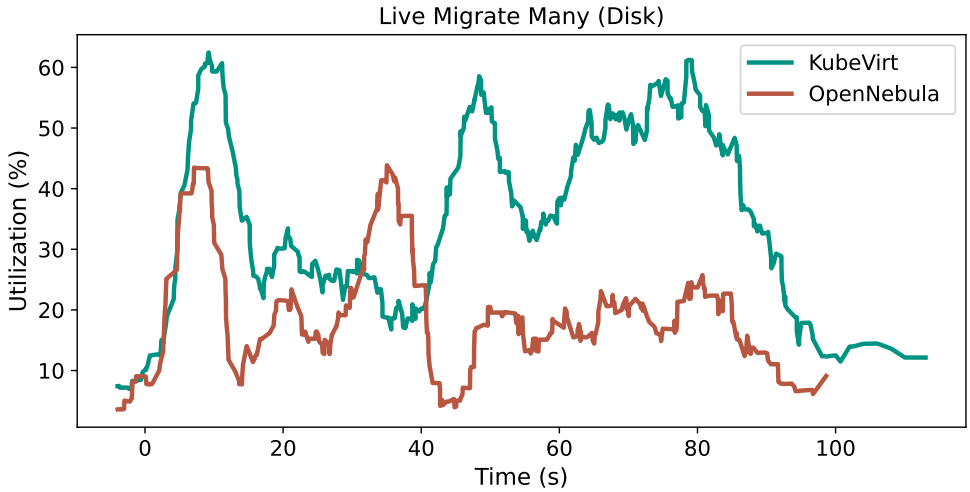

Figure B.29: Disk utilization on the control node when live migrating 10 type-1 VMs. KubeVirt has an overall larger disk utilization than OpenNebula. However, the disk utilization has sporadic peaks that are not in line with the operations performed during the test.

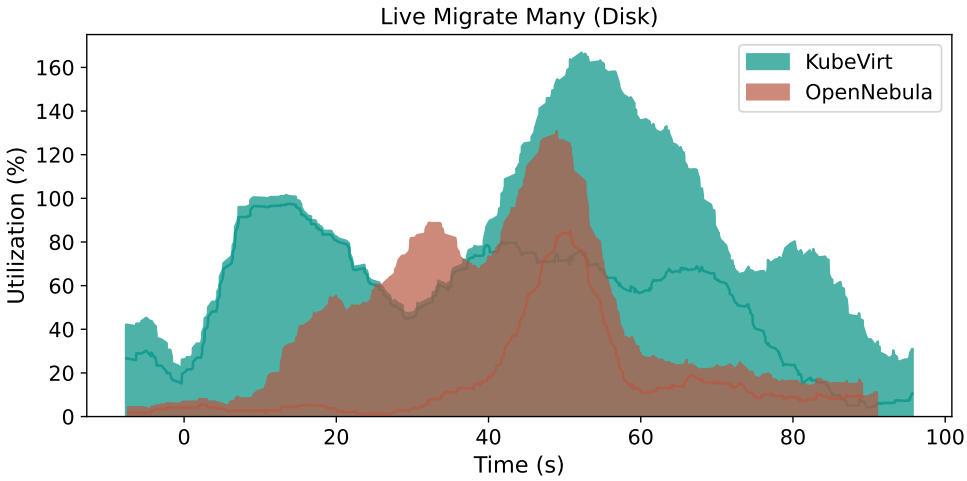

Figure B.30: Cumulative disk utilization among worker nodes when live migrating 10 type-1 VMs. KubeVirt has both a larger base utilization and cumulative disk utilization than OpenNebula. Both systems show a clear pattern of the disk utilization being shifted to a second worker node after half the duration of the test. However, KubeVirt takes a longer time to return to an idle state of the first worker node than OpenNebula.

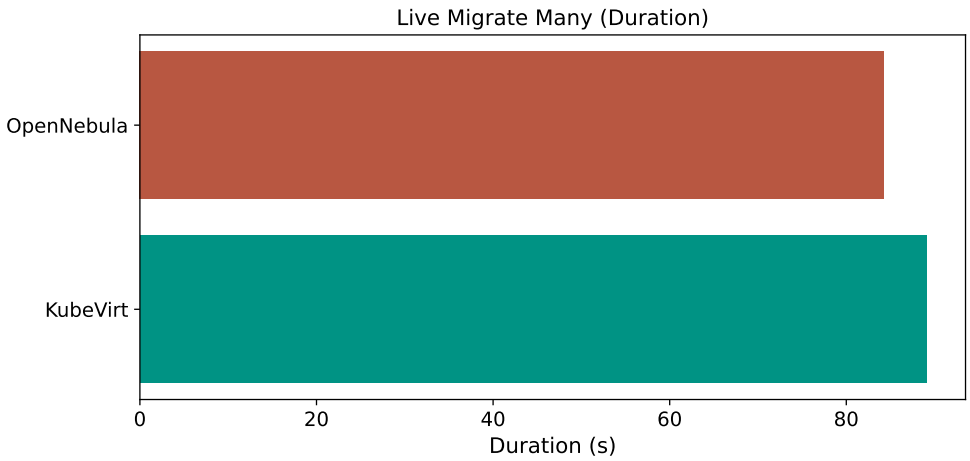

Figure B.31: Duration of live migrating 10 type-1 VMs. KubeVirt is slightly slower than OpenNebula, taking on average 3 seconds longer to complete, 87 seconds compared to OpenNebula's 84 seconds.

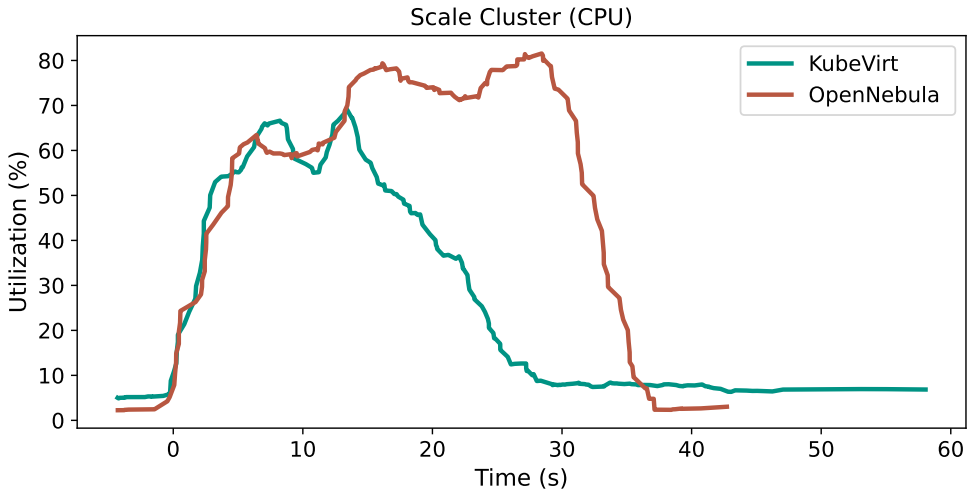

Figure B.32: CPU utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. Both systems are initially identical in CPU utilization, but KubeVirt subsides faster than OpenNebula.

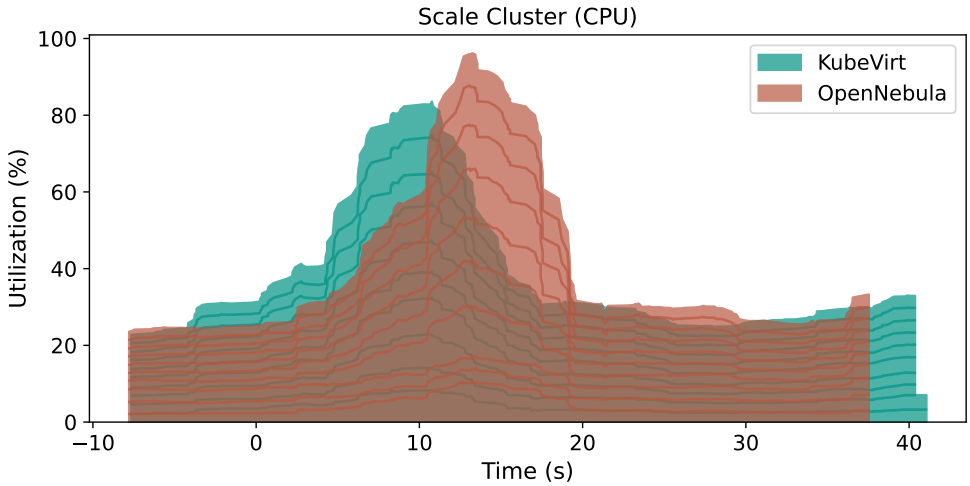

Figure B.33: Cumulative CPU utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. Both the base utilization and induced load are similar between the two systems. However, OpenNebula is slightly faster to complete the scaling operation than KubeVirt.

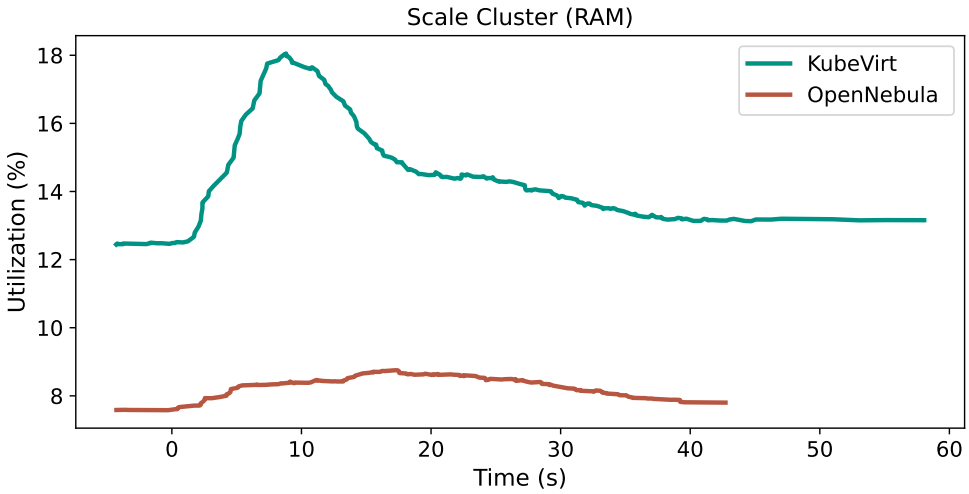

Figure B.34: Memory utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a significantly larger memory utilization than OpenNebula. OpenNebula's memory utilization is barely affected by the scaling operation.

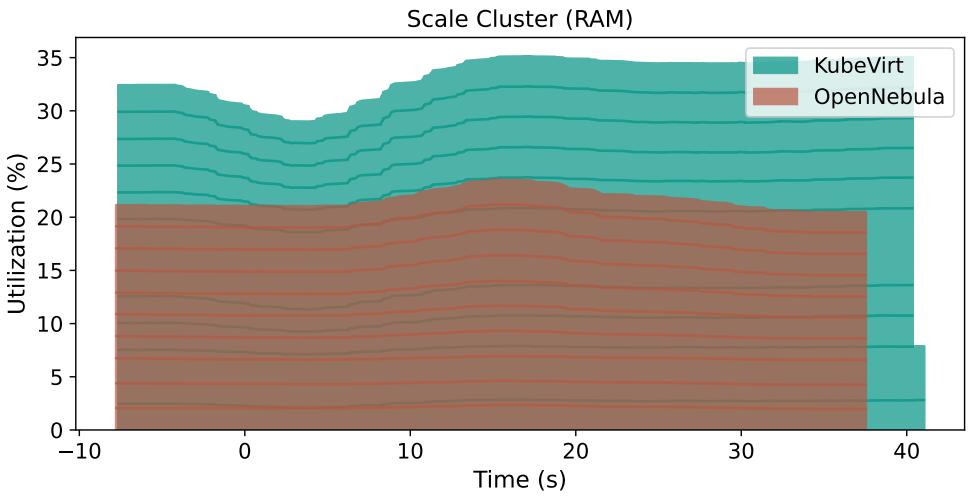

Figure B.35: Cumulative memory utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a slightly larger base memory utilization than OpenNebula, as well as a larger induced load from the scaling operation.

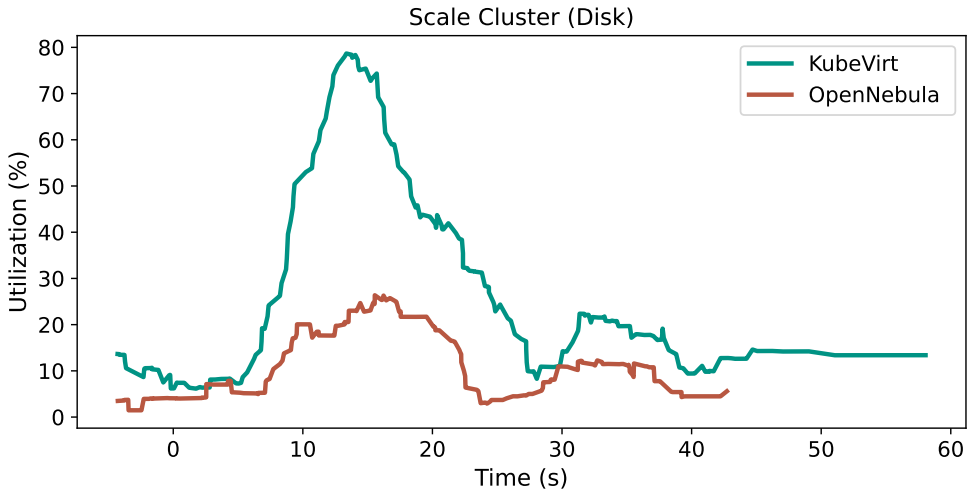

Figure B.36: Disk utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a rapid increase in disk utilization, reaching a peak of over 70% utilization. OpenNebula has a less aggressive increase reaching barely 30% utilization.

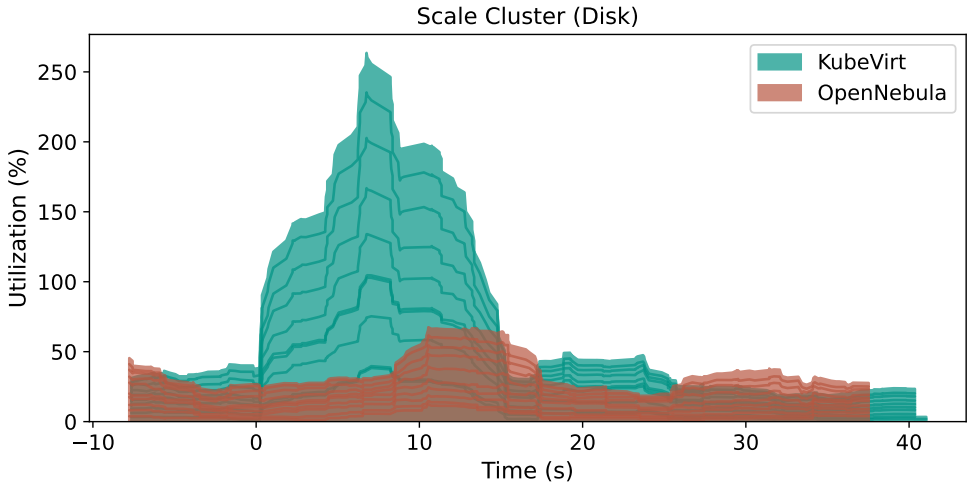

Figure B.37: Cumulative disk utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes. KubeVirt experiences a significantly larger disk utilization than OpenNebula with a burst of disk utilization whereas OpenNebula has a more gradual increase.

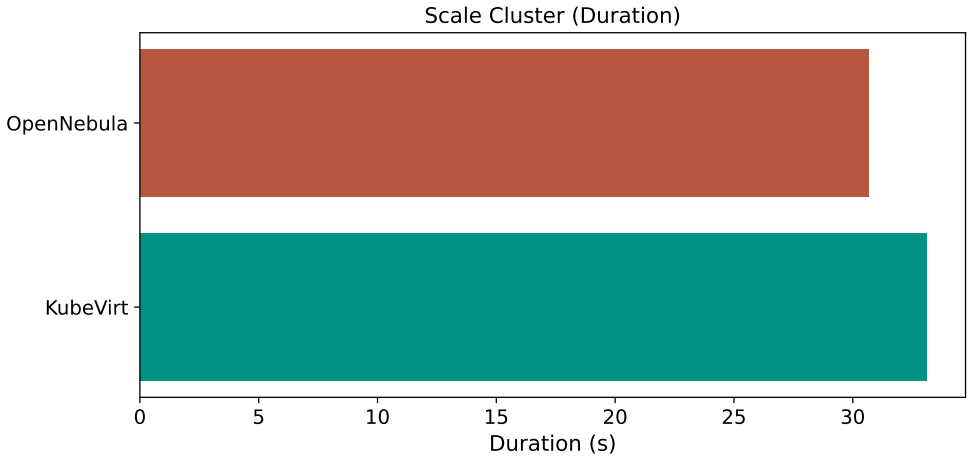

Figure B.38: Duration of scaling the cluster. KubeVirt is slightly slower than OpenNebula, taking on average 3 seconds longer to complete, 33 seconds compared to OpenNebula's 30 seconds.

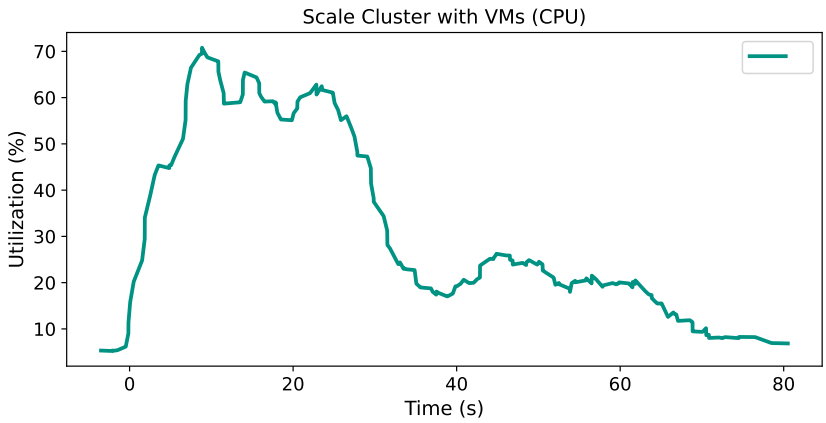

Figure B.39: CPU utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences an initial burst of CPU utilization for the scale-up operation, but a significantly lower utilization for the scale-down operation. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

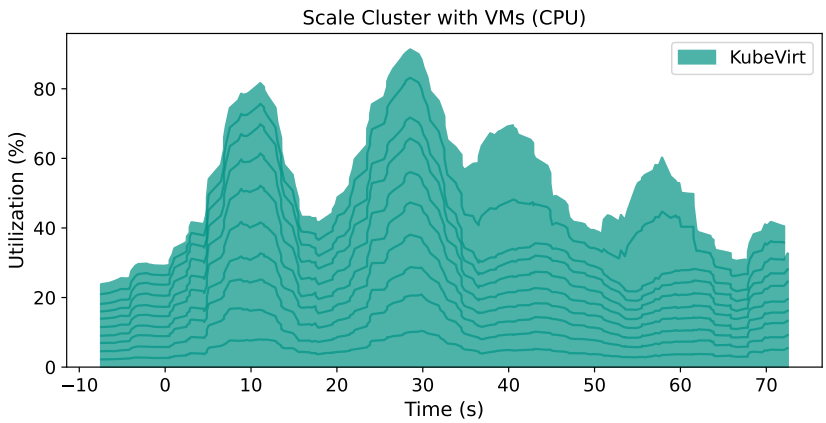

Figure B.40: Cumulative CPU utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt maintains utilization with periodic peaks throughout the test. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

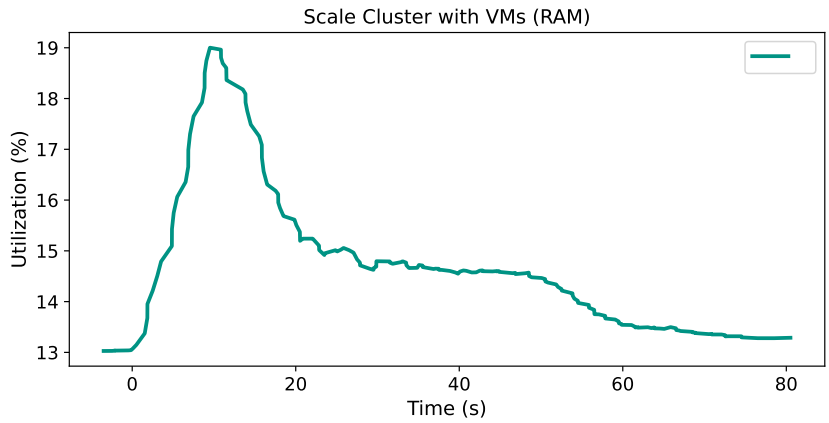

Figure B.41: Memory utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences a large burst of memory utilization for the scale-up operation, but a significantly lower utilization for the scale-down operation that gradually decreases towards the end. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

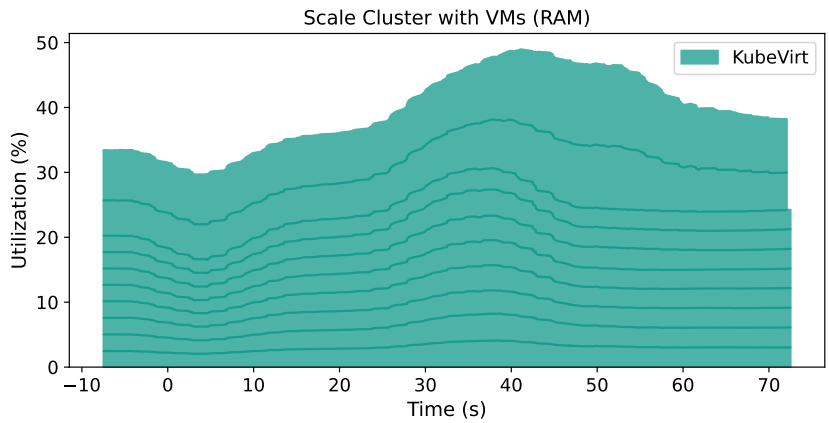

Figure B.42: Cumulative memory utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt maintains a steady memory utilization throughout the test. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

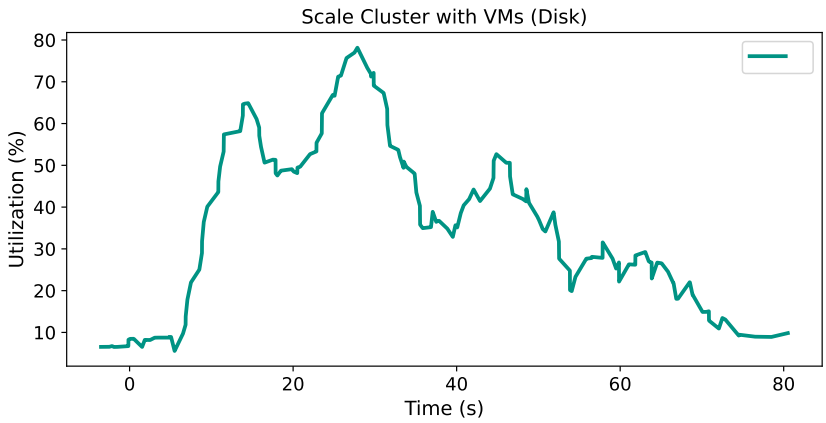

Figure B.43: Disk utilization on the control node when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences a large initial burst of disk utilization that gradually subsides toward the end of the test. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

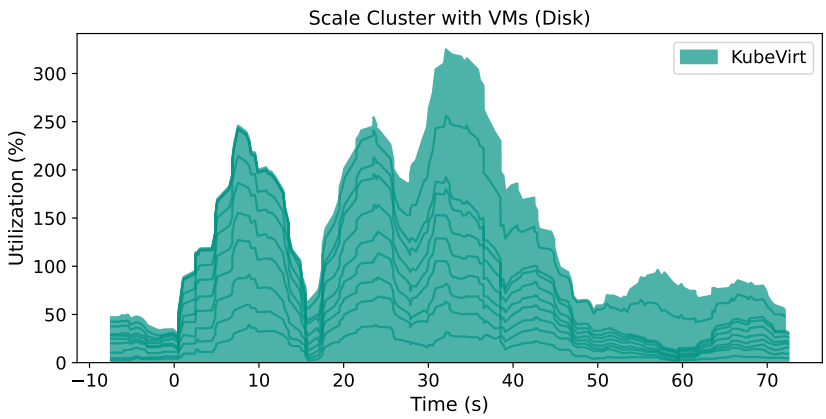

Figure B.44: Cumulative disk utilization among worker nodes when scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt experiences large periodic peaks of disk utilization throughout the test. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

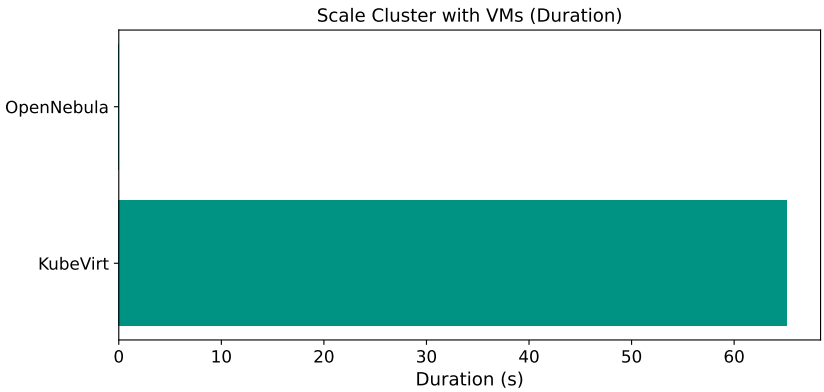

Figure B.45: Duration of scaling the cluster from 2 worker nodes to 10 worker nodes, then back to 2 worker nodes with 1 type-1 VM running on each node. KubeVirt completes the test in just below 70 seconds. OpenNebula required manual intervention to complete the test due to VMs ending up in bad states, which caused it to be disqualified.

Appendix C

Usability Tasks

The following is a replicated version of what was given to the participants for the usability analysis formatted for the appendix. The original version was provided as a separate document to the participants.

Introduction

This is a usability analysis of KubeVirt that involves performing a set of tasks and then reflecting on the experience, where the goal is to evaluate how well KubeVirt works in practice. By participating in the usability analysis you provide valuable feedback and insight that can be used to improve KubeVirt. Your answers will be included as-is in the appendix of the project report. Upon request, the answers can be anonymized.

The following is a brief introduction to KubeVirt, its most relevant features and how it compares to previous VM management systems. If you are already familiar with KubeVirt and its machinery, you can skip this section.

KubeVirt is an extension to Kubernetes that allows for the creation of virtual machines (VMs) and is designed to provide a similar experience to managing any other resource in Kubernetes, such as Pods or Deployments. This means that VMs are defined in YAML files and integrate with the command line tool *kubectl*. KubeVirt defines its resources as a *Custom Resource Definition* (CRD) in Kubernetes, where the most prominent resources are *VirtualMachine* and *VirtualMachineInstance*. The relationship between them is comparable to that of a Deployment and a Pod, where the Deployment defines a desired state and a Pod represents the running state. See References below for more information.

Certain VM operations are not possible using kubectl, such as accessing the VM using SSH or VNC. For this reason, KubeVirt provides a complementary CLI tool called *virtctl*.

Being built on top of Kubernetes allows the VMs to integrate with other resources, including the Service and PersistentVolume resources. For instance, a VM can be exposed to the network using a LoadBalancer Service or manage storage dynamically using a persistent storage provisioner. However, since a VM requires a disk image (such as a QCOW2 image), it means that a PersistentVolume must point to a folder with a disk image in it. For this reason, manually creating a PersistentVolume for a VM can be tedious. Instead, KubeVirt primarily provides two alternatives for managing disks that are presented below. See the References section for more information.

- **containerDisk**: A disk that is created at every boot. It is not persistent and is similar to a container.
- **DataVolume**: A disk that is persistent and initialized when the VM is created. It uses PersistentVolumeClaim and a PersistentVolume under the hood where it injects a disk image.

Basic usage (Task 1/3)

The first task is about creating a VM using KubeVirt. You will need to create a VM and then access it using SSH or VNC. You can use a containerDisk for simplicity.

Manifests

Below is a minimal manifest for creating the VM using a containerDisk. The credentials for the Cirros image are user **cirros** and password **gocubsgo**.

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
spec:
  running: true
  template:
    spec:
      domain:
```

```
      cpu:
        cores: 1
      devices:
        disks:
        - name: containerdisk
          disk:
            bus: virtio
        interfaces:
        - name: default
          masquerade: {}
      resources:
        requests:
          memory: 128M
    networks:
    - name: default
      pod: {}
    volumes:
    - name: containerdisk
      containerDisk:
        image: quay.io/kubevirt/cirros-
           container-disk-demo
```

Steps

1. Create a VM

```
kubectl apply -f my-vm.yaml
```

2. Access the VM through SSH or VNC (credentials vary depending on the image since no CloudInit is used)

```
virtctl ssh cirros@my-vm
virtctl vnc my-vm
```

3. Stop, start and restart the VM

```
virtctl stop my-vm
virtctl start my-vm
virtctl restart my-vm
```

4. Delete the VM

```
kubectl delete vm my-vm
```

Questions

After you have finished the tasks, reflect on the experience. You may use the following questions as a guide, but feel free to elaborate with your own thoughts.

- How intuitive was the process of creating and deleting a VM?
- How straightforward was it to access the VM using SSH and VNC?
- Describe, if any, the issues you encountered during the task. Were the issues related to KubeVirt, documentation, or other?

Maintenance (Task 2/3)

The second task is related to maintenance, where you will create a snapshot of a VM and then restore it. You will also perform a live migration on a VM to move it from one node to another without shutting it down.

Both snapshots and live migrations are treated as a resource by KubeVirt, meaning that the operation will take place in the background after a VirtualMachineSnapshot resource or a VirtalMachineInstanceMigration resource is created. See the References section for more information.

A live migration resource in KubeVirt does not move resources itself and instead uses the Kubernetes scheduler to decide where to place the VM. This means that a LiveMigration resource only tells the scheduler that the VM can be moved but not where to move it. See the References section for more information. The Kubernetes scheduler is sophisticated and supports a wide range of scheduling strategies, but for this task, we can force a move between two nodes by using the *spec.template.spec.nodeSelector* in the VM and editing the labels of the worker nodes in the cluster. To not interfere with other participants, you should create a unique label for you.

A snapshot in KubeVirt works similarly to a snapshot in other VM management systems, where it captures the state of the VM at a specific point in time. However, the snapshot uses Kubernetes VolumeSnapshot, which is a Kubernetes API for taking snapshots of PersistentVolumes. This means that a VM with a containerDisk cannot be snapshotted. Instead, you should use a VM with a DataVolume. See the References section for more information.

Live Migration Manifests

The following is a minimal manifest for creating a VM that runs on a node with label **name: worker-1**. The credentials for the Cirros image are user **cirros** and password **gocubsgo**.

```
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
spec:
  running: true
  template:
    spec:
      nodeSelector:
        # Edit this to use a label of your
           choice
        my-label: some-value
      domain:
        cpu:
          cores: 1
        devices:
          disks:
          - name: containerdisk
            disk:
              bus: virtio
          interfaces:
          - name: default
            masquerade: {}
        resources:
          requests:
            memory: 128M
      networks:
      - name: default
        pod: {}
      volumes:
      - name: containerdisk
        containerDisk:
          image: quay.io/kubevirt/cirros-
             container-disk-demo
```

```yaml
apiVersion: kubevirt.io/v1
kind: VirtualMachineInstanceMigration
metadata:
  name: my-vm-migration
spec:
  vmiName: my-vm
```

Live Migration Steps

1. Ensure worker node 1 has your label (Make sure to replace `my-label` and `some-value` with your own values):

   ```
   kubectl label node usability-worker-1 my-label=some
      -value
   ```

2. Create a VM and wait for it to start. The VM should start on **usability-worker-1**:

   ```
   kubectl apply -f my-vm.yml
   ```

3. (Optional) Check if the VM is running on worker node 1:

   ```
   kubectl get vmis
   ```

4. Create a LiveMigration resource or use virtctl:

   ```
   kubectl apply -f my-vm-migration.yml
   ```

   ```
   virtctl migrate my-vm
   ```

5. Add the same label to worker node 2 and remove it from worker node 1 (Make sure to replace `my-label` and `some-value` with your own values):

   ```
   kubectl label node usability-worker-2 my-label=some
      -value
   kubectl label node usability-worker-1 my-label-
   ```

6. (Optional) Check if the VM is now running on worker node 2:

   ```
   kubectl get vmis
   ```

7. Unlabel worker node 2

```
kubectl label node usability-worker-2 my-label-
```

8. Delete the VM

```
kubectl delete vm my-vm
```

Snapshot Manifests

The following is a minimal manifest for creating a VM with a DataVolume. The credentials for the Cirros image are user **cirros** and password **gocubsgo**.

```yaml
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
spec:
  running: true
  template:
    spec:
      domain:
        devices:
          rng: {}
          disks:
          - disk:
              bus: virtio
            name: datavolume-disk
          interfaces:
          - name: default
            masquerade: {}
        resources:
          requests:
            memory: 1Gi
      networks:
      - pod: {}
        name: default
      volumes:
      - dataVolume:
          name: datavolume
        name: datavolume-disk
  dataVolumeTemplates:
```

```
  - metadata:
      name: datavolume
    spec:
      pvc:
        storageClassName: azurefile-csi-kubevirt
        accessModes:
        - ReadWriteOnce
        resources:
          requests:
            storage: 5Gi
      source:
        registry:
          url: docker://quay.io/kubevirt/cirros-
             container-disk-demo
```

The following is a minimal manifest for creating a VirtualMachineSnapshot resource.

```
apiVersion: snapshot.kubevirt.io/v1alpha1
kind: VirtualMachineSnapshot
metadata:
  name: my-vm-snapshot
spec:
  source:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: my-vm
```

The following is a minimal manifest for restoring a VM from a snapshot.

```
apiVersion: snapshot.kubevirt.io/v1alpha1
kind: VirtualMachineRestore
metadata:
  name: my-vm-restore
spec:
  target:
    apiGroup: kubevirt.io
    kind: VirtualMachine
    name: my-vm
  virtualMachineSnapshotName: my-vm-snapshot
```

Snapshot Steps

1. Create a VM

   ```
   kubectl apply -f my-vm.yml
   ```

2. Create a snapshot of the VM

   ```
   kubectl apply -f my-vm-snapshot.yml
   ```

3. Wait for the snapshot to be ready:

   ```
   kubectl get vmsnapshots
   ```

4. Stop the VM

   ```
   virtctl stop my-vm
   ```

5. Restore the VM from the snapshot

   ```
   kubectl apply -f my-vm-restore.yml
   ```

6. Wait for the VM to be restored

   ```
   kubectl get vmrestore
   ```

7. Delete the resources

   ```
   kubectl delete vmsnapshot my-vm-snapshot
   kubectl delete vm my-vm
   ```

Questions

After you have finished the tasks, reflect on the experience. You may use the following questions as a guide, but feel free to elaborate with your own thoughts.

- How intuitive was the process of migrating a VM from one node to another?
- How straightforward was it to create and restore a snapshot of a VM?
- Describe, if any, the issues you encountered during the task. Were the issues related to KubeVirt, documentation, or other?

Debugging (Task 3/3)

The third task is about debugging a VM that is not accessible through SSH. You will be presented with a VM manifest which, if deployed, will result in a VM that is running but cannot be accessed through SSH. Since the VM is running, you can access it through VNC using virtctl. You can use a containerDisk since the task is not related to persistent storage.

Manifests

The following is a manifest for creating a Ubuntu VM that is not accessible through SSH. Credentials for the Ubuntu image should be user **cloud** and password **cloud**.

```yaml
apiVersion: kubevirt.io/v1
kind: VirtualMachine
metadata:
  name: my-vm
spec:
  running: true
  template:
    spec:
      domain:
        devices:
          disks:
          - name: containerdisk
            disk:
              bus: virtio
          - name: cloudinit
            disk:
              bus: virtio
          rng: {}
        resources:
          requests:
            cpu: 2
            memory: 2Gi
      volumes:
      - name: containerdisk
        containerDisk:
```

```
        image: quay.io/containerdisks/ubuntu
           :22.04
    - name: cloudinit
      cloudInitNoCloud:
        userData: |-
          #cloud-config
          users:
          - name: cloud
            # Generated with `mkpasswd -m sha
               -512 --rounds 4096`
            passwd: $6$rounds=4096$abc
            shell: /bin/bash
            lock-passwd: false
            chpasswd: { expire: False }
            sudo: ALL=(ALL) NOPASSWD:ALL
            ssh_authorized_keys:
            - ssh-ed25519 abc
          ssh_pwauth: True
```

Steps

1. Create a VM

   ```
   kubectl apply -f my-vm.yml
   ```

2. Access the VM through VNC

   ```
   virtctl vnc my-vm
   ```

3. Find out what the issue is.

4. Edit the VM manifest to fix the issue.

5. (Optional) Restart the VM (some changes require a restart)

   ```
   virtctl restart my-vm
   ```

6. Ensure the VM is accessible through SSH. You can use the *–local-ssh* flag if there is a problem with the SSH keys.

   ```
   virtctl ssh cloud@my-vm
   ```

7. Delete the VM

```
kubectl delete vm my-vm
```

Questions

After you have finished the task, reflect on the experience. You may use the following questions as a guide, but feel free to add comparisons with other VM management systems you have used.

- How easy was it to debug and find the issue?
- How easy was it to fix the issue?
- Does the debug process differ from debugging a traditional VM management system?
- Describe, if any, the issues you encountered during the task. Were the issues related to KubeVirt, documentation, or other?

Summary

After you have completed the tasks, reflect on the overall experience of using KubeVirt. You may use the following questions as a guide, but feel free to discuss and compare them with other VM management systems you have used.

- How would you compare the experience of using KubeVirt with other VM management systems you have used?
- How intuitive is the workflow of KubeVirt? Can you use it without reading the documentation?
- How would you describe the overall experience of using KubeVirt?

Thank you for participating in the usability analysis of KubeVirt!

References

- **VirtualMachineInstance**: `https://kubevirt.io/user-guide/virtual_machines/virtual_machine_instances/`
- **Disks**: `https://kubevirt.io/user-guide/virtual_machines/disks_and_volumes`

- **Snapshots**: `https://kubevirt.io/user-guide/operations/snapshot_restore_api/`
- **Live Migration**: `https://kubevirt.io/user-guide/operations/live_migration/`

Appendix D

Usability Analysis Answers

D.1 Participants

- Participant 1: Linus
- Participant 2: Osvald
- Participant 3: Karl
- Participant 4: Sai

D.2 Task 1/3

How intuitive was the process of creating and deleting a VM?

- **Linus**

  The act of doing *kubectl apply -f* and *kubectl delete* is quite neat, but the VM definition is not very neat. It would be quite difficult for me to figure out I need to use *bus: virtio* without having someone else tell me I need to. The *virtctl create vm* interface on the other hand seems to be much easier to use. As referenced from the official documentation.

  At this point, I decided I wanted to try to launch a Debian VM.

  Using *virtctl create vm* however, it was not clear which container disks are generally available or how I can create one (*virtctl create vm –help* and the help for *–volume-containerdisk*). What I'd like is an explanation of what that flag means.

What I'm searching for now is how to create a Debian VM, looking at `https://quay.io/organization/kubevirt` where the example provided was I could not find anything called *debian*. Just trying *virtctl create vm | kubectl apply -f -* did not provide a working VM either, so I can't just modify that to work as I want.

The documentation referred to *quay.io/containerdisks*, which did have some more useful disks but not Debian (only Ubuntu, Fedora and Centos).

Thought: *containerDisks are not a good solution for any workload that requires persistent root disks across VM restarts, so a VM with a containerDisks is probably not what I want for my bog-standard VM.*

At this point, I thought I would just try to look at the standard KubeVirt documentation, hoping that they mention how you run a VM that isn't just a demo. What I found were two guides:

1. KubeVirt Lab 1 (`https://kubevirt.io/labs/kubernetes/lab1.html`)
   - Did not mention anything on how to run my VM.
2. KubeVirt Lab2 (`https://kubevirt.io/labs/kubernetes/lab2.html`)
   - This might be what I want but it is not clear what a CDI is nor what I should use it for, even after clicking through to `https://github.com/kubevirt/containerized-data-importer`.
   - The first step of the guide is to install an operator, which I don't think I want to do just to get started.
   - As an aside it is a bit worrying that they have written this:
     * If the importer pod completes in error, you may need to retry it or specify a different URL to the fedora cloud image. To retry, first delete the importer pod and the DataVolume, and then recreate the DataVolume.
     * That means I can't rely on just creating the resource and having it do what I want. I need to monitor it with my retry logic to be sure it works.
   - This guide specifies how I can provide my SSH keys via cloud-init, which is nice.
   - Trying to follow that guide gave me this error when I ran *kubectl describe datavolumes.cdi.kubevirt.io*

    * Normal ErrClaimNotValid 48s datavolume-import-controller PVC spec is missing accessMode and no storageClass to choose profile
    * While this doesn't seem like an unsurmountable problem I decided to stop here.

- **Osvald**

  Very easy! I like that it's as simple as creating a YAML file and then applying it. Much simpler than any other way I've tried for managing VMs.

How straightforward was it to access the VM using SSH and VNC?

- **Linus**

  Quite, I would have loved to easily be able to integrate the SSH proxy command into my SSH config. Also, I find *virtctl console* nicer than VNC.

- **Osvald**

  Very simple! Simply doing *virtctl ssh cirros@my-vm* and then typing in the password and then I was in!

Describe, if any, the issues you encountered during the task. Were the issues related to KubeVirt, documentation, or other?

- **Linus** I could not figure out how to create a Debian image. The documentation was very lacking as it seemed geared towards how to operate a KubeVirt cluster, not how to use one.

  From the documentation, it also seems that KubeVirt is plagued by the same issues a lot of the Kubernetes ecosystem is: a lot of strange abstractions that aren't well explained (CDI, DataVolume, containerDisks etc.).

- **Osvald**

  Getting VNC to work on Mac was a bit tricky, but I downloaded Tiger VNC and saved the script from (`https://github.com/kubevirt/kubevirt/issues/812#issuecomment-391832335`) as */usr/local/bin/remote-viewer*.

Summary

- **Karl**

  Everything was smooth sailing (except when I didn't read the instructions properly and missed the provided password). This process for managing VMs is much more direct, transparent and reproducible than using a UI-based tool.

- **Sai**

  Personally, from having worked with older virtualization platforms and Kubernetes, I'd prefer to use KubeVirt as my virtualization platform. It is primarily because I'm already storing all my Kubernetes manifest in Git and have the CI setup for that, so why not also create virtual machines in a similar way.

D.3 Task 2/3

How intuitive was the process of migrating a VM from one node to another?

- **Linus**

  Not very, it was unintuitive that the migration succeeded even though nothing was migrated, even though it should have been (I used a *nodeSelector* that didn't exist, so it should not have been running on any node).

  I also found it unintuitive that I needed to create the migration resource after everything was correctly labeled. I would have expected that just updating the *nodeSelector* would move the VM.

- **Osvald**

  Very simple! I once again really like that you just edit YAML files and apply labels to conveniently move VMs between worker nodes. Much simpler than our current way of doing it in OpenNebula.

Describe, if any, the issues you encountered during the task. Were the issues related to KubeVirt, documentation, or other?

- **Linus**

  The documentation mentions that I should look at the status of the VMI, which I did not think to do (regarding the intuitive question above).

  Had I read the documentation I would at least have known that the migration decided to do nothing when I used a non-existent *nodeSelector*.

Once again the documentation deals a lot with how to operate the migration infrastructure and not anything on how one would do a migration operation. Granted it is listed under the *Operations* section of the documentation but I found nothing when searching under the *Virtual Machines* section where I would have expected it to be.

- **Osvald**

  I had to delete the entry from */.ssh/kubevirt_known_hosts* after each time I restarted the VM because *virtctl ssh* complained about a key mismatch.

Summary

- **Linus**

  I found this unintuitive as if it couldn't migrate (nothing matching the *nodeSelector*) I would have expected the migration to fail, not succeed.

  Once again this is also a bit strange, that I have to remove the migration resource and recreate it for something to happen. That means I can't just create a resource and trust KubeVirt will manage it but will need to have logic around the migration.

D.4 Task 3/3

How easy was it to debug and find the issue?

- **Linus**

  Not very, I did not manage to solve all the issues.

  The password issue was easy as not being able to log in was very obvious with *virtctl console*. The SSH issue I solved by accident, the *known-hosts: key mismatch* made me look into the *.ssh/authorized_keys* file and updated that with *virtctl console*. It didn't solve the key mismatch issue but it did allow me to continue with local SSH. I did not manage to solve the key mismatch issue as there is very little in the way of debugging *virtctl ssh*.

- **Osvald**

  Easy! Using *virtctl vnc* I could change the password and then try logging in.

How easy was it to fix the issue?

- **Linus**

  Two of the issues were easy to fix, I appreciated that *virtctl restart* worked as I did not think cloud-init would update the configs.

- **Osvald**

  Very easy, I just generated a new password.

Does the debug process differ from debugging a traditional VM management system?

- **Linus**

  I'm not sure what a traditional VM management system refers to. It is quite similar to the OpenNebula command lines. I appreciated that *virtctl console* was so easy to use.

- **Osvald**

  I've used IPMI which is similar to the VNC session in this case.

Describe, if any, the issues you encountered during the task. Were the issues related to KubeVirt, documentation, or other?

- **Linus**

  I could not (easily) find documentation about how *virtctl ssh* worked so I could not figure out one of the issues. There seems to be no documentation for *virtctl*. Is it the same *virtctl* as the one from OpenShift? The fact that *kubectl ssh* broke without even a verbose option is worrying.

- **Osvald**

  The most difficult thing was to get the VNC viewer working on Mac, but after that, it was smooth sailing.

Summary

- **Linus**

  A "key mismatch" error is difficult to debug because there are no good debug options for *virtctl ssh*.

- **Karl**

  Debugging the issue was easy because the manifest uses standard constructs that are easily googleable.

D.5 Summary

How would you compare the experience of using KubeVirt with other VM management systems you have used?

- **Linus**

  I like the *virtctl* tool (when it worked) and like the idea of declaring my VMs. However, the system is very complicated with a lot of parts that it isn't clear how they fit together.

  The documentation is also very lacking. The fact that you manage the system with two different tools, both *virtctl* and *kubectl* is unnecessary and confusing. I find it telling that I didn't even manage to run a Debian VM.

- **Osvald**

  It seems to be much easier to deploy and manage machines! I want to start using this in production for our build pipelines and internal clusters.

How intuitive is the workflow of KubeVirt? Can you use it without reading the documentation?

- **Linus**

  Not at all. I would not have been able to use it without the documentation. I could barely use it with the documentation.

- **Osvald**

  Well, I had the example files you provided which were simple. But the CLI tools also gave good help.

How would you describe the overall experience of using KubeVirt?

- **Linus**

  The idea is good but the implementation is severely lacking. I could see the potential and it seemed very promising but from this experience, KubeVirt did not deliver.

  One could probably build something sensible with it but I feel the barrier of entry is very high.

- **Osvald**

  Great! I want more!

- **Karl**

  For developers with previous Kubernetes experience, it looks to be a very intuitive way to handle virtual machines. In an environment where Kubernetes clusters are already part of the workflow, it is obvious how aspects such as automation and access control can be transferred to apply also to the VM infrastructure.

  Since I was just trying it out, I haven't had any real issues that I need to debug yet.

- **Sai**

  I've been using KubeVirt for several months and I like it. Specifically, the cloud native approach where everything is API accessible, which makes it easier to build tooling around and the community has done a great job of that as expected. Due to the fact it's APIs first, it opens up many approaches to manage virtual machines. Personally, from having worked with older virtualization platforms and Kubernetes, I'd prefer to use KubeVirt as my virtualization platform.

  I'd say I like this relatively new take on how to deal with virtual machines. At the same time, it may not be for everyone. In the end, it depends on what problems you are trying to solve and the need.